

University of Sfax
Faculty of Economics and Management Sciences
Department of Applied Computer Science in Management

GRADUATION PROJECT REPORT

For the obtainment of the

Applied Bachelor's Degree in Computer Science Applied to Management

---

PETIT SUIVI

Prepared by:
Mahmoud Kammoun

Supervised by:
Marwa Graja

Jury Members:
Monia Ben Brahim
Emna Feki

Academic Year 2025 – 2026

Contents

General Introduction **3**

1 Project Definition and Analysis of Existing Solutions **4**
- **1.1 Introduction** . . . **4**
- **1.2 Mission Definition** . . . **4**
  - **1.2.1 Presentation of the Host Organization** . . . **4**
  - **1.2.2 Presentation of the Application** . . . **5**
  - **1.2.3 Objectives to be Achieved** . . . **6**
- **1.3 Project Context** . . . **7**
- **1.4 Analysis of Existing Applications** . . . **7**
  - **1.4.1 Brightwheel Application** . . . **7**
  - **1.4.2 Kinderpedia Application** . . . **8**
  - **1.4.3 Synthesis and Comparison** . . . **9**
- **1.5 Problem Statement and Proposed Solution** . . . **10**
- **1.6 Conclusion** . . . **10**

2 Requirements Gathering **11**
- **2.1 Introduction** . . . **11**
- **2.2 Identification of System Actors** . . . **11**
- **2.3 Context of the Information System** . . . **11**
- **2.4 Identification of Use Cases** . . . **12**
  - **2.4.1 Use Case Diagram** . . . **12**
  - **2.4.2 Use Case Summary Table** . . . **13**
- **2.5 Textual Description of Use Cases** . . . **15**
  - **2.5.1 1. Parent Use Cases** . . . **15**
  - **2.5.2 2. Teacher Use Cases** . . . **18**
  - **2.5.3 3. Teacher and Parent Use Cases** . . . **24**
  - **2.5.4 4. Admin Use Cases** . . . **26**
  - **2.5.5 5. All Actors Use Cases** . . . **50**
- **2.6 Conclusion** . . . **52**

3 Analysis and Design **53**
- **3.1 Introduction** . . . **53**
- **3.2 Realization of Use Cases** . . . **53**
  - **3.2.1 Parent: Track Children - Scheduled Activities** . . . **54**
  - **3.2.2 Parent: Track Children - Attendance Status** . . . **54**
  - **3.2.3 Parent: Track Children - Acquired Competences** . . . **55**
  - **3.2.4 Parent: View Photos** . . . **55**
  - **3.2.5 Parent: Submit Registration Form** . . . **56**
- **3.3 Domain Model Construction (Class Diagram)** . . . **57**
  - **3.3.1 Detailed Class Definitions** . . . **58**
- **3.4 Future Software Architecture** . . . **71**
- **3.5 Physical Data Schemas** . . . **72**

3.5.1 User & Access Management . . . 73
3.5.2 Registration & Finance . . . 74
3.5.3 People & Logistics . . . 75
3.5.4 Nutrition & Health . . . 77
3.5.5 Educational Planning . . . 79
3.6 Conclusion . . . 81

4 Implementation 82
4.1 Introduction . . . 82
4.2 Development Environment . . . 82
4.2.1 Hardware Environment . . . 82
4.2.2 Software Environment and Technologies . . . 82
4.2.3 Deployment and Infrastructure Tools . . . 83
4.3 Technical Implementation of Use Cases . . . 84
4.3.1 Parent Use Cases . . . 84
4.3.2 Teacher Use Cases . . . 91
4.3.3 Parent and Teacher Shared Use Cases . . . 97
4.3.4 Administrator Use Cases . . . 101
4.3.5 System API Endpoints . . . 110
4.4 Conclusion . . . 112

General Conclusion 113

Bibliography 114

Webography 115

General Introduction

In any educational institution, the Information System aims to facilitate the establishment and implementation of pedagogical and administrative management strategies. It must support the execution of daily activities such as enrollment management and activity monitoring while ensuring an efficient flow of information between parents, teachers, and the administration.

The system is designed to meet professional requirements and consists of a set of hardware and software resources used to collect, store, and process data related to children's development. One of the greatest assets of a preschool institution lies in this information, which is often heterogeneous, concerning the medical, educational, and behavioral monitoring of each child.

Despite this wealth of information, the absence of an integrated computerized tool makes it difficult to obtain a comprehensive and coherent overview of student progress and administrative management. Furthermore, manual access to information stored across different media, including paper files, allergy lists, and attendance registers, makes data handling complex and inefficient.

It is within this context that we examine the case of preschool management. Given the significant volume and sensitivity of the information handled in these institutions, we were tasked with automating their Information System as part of a final year project entitled Petit Suivi.

The objective of this project is to develop a comprehensive management platform that facilitates the efficient use of service resources, such as teachers, classes, and meal plans, to carry out daily tasks. The project also aims to strengthen communication between parents and the school, ensure accurate tracking of each child's development, and optimize work organization through innovative tools, particularly artificial intelligence for behavioral monitoring.

To accomplish the required work, we adopted a design methodology based on the Rational Unified Process and the Unified Modeling Language. The results of our design work are presented in this report, which is structured into four distinct chapters.

In the first chapter, Business Modeling, we analyze the existing system by highlighting the objectives to be achieved. We define the scope of the study and examine in detail the business processes within the educational institution. The second chapter, Requirements Gathering, focuses on identifying the actors of the future application, including administrators, teachers, and parents, while developing the context model and describing the use case model. In the third chapter, we analyze user requirements, design use case scenarios, and define the application classes, the system architecture, and the physical database schema. Finally, the fourth chapter, Implementation, describes the development environment involving Laravel, Flutter, and React, presents the technical realization of each use case with interface screenshots, and documents the system API endpoints.

This report concludes with a general summary of the work accomplished and proposes future improvements to further enhance the application.

Chapter 1

Project Definition and Analysis of Existing Solutions

1.1 Introduction

Business modeling is an effective way to manage the complexity of a project and ensure its overall consistency. This phase is essential to achieve a shared understanding among stakeholders and to clearly define the problem to be addressed.

This stage begins with an analysis of the existing system and a precise definition of our mission. It also involves identifying the project scope, describing business processes, and determining the information flows involved.

1.2 Mission Definition

Our mission within the framework of this project is to design and develop a comprehensive platform, along with its mobile application, aimed at facilitating the daily management of preschool institutions.

This solution enables administrators, teachers, and parents to efficiently manage:

- Enrollments and user accounts
- Financial tracking and monthly payments
- Child monitoring and pedagogical competence evaluations
- Activities, events, and meal orchestration (including dietary exceptions)
- Notifications and real-time interactions related to school life

The following subsections present the host organization, the application, and the objectives to be achieved.

1.2.1 Presentation of the Host Organization

The host organization is an educational institution specializing in early childhood education, providing preschool and kindergarten services. Its mission is to ensure high-quality pedagogical support and effective communication with families.

To address growing needs in enrollment management, detailed child monitoring, financial tracking, activity and meal coordination, and instant internal communication, the institution seeks to modernize its processes.

The **Petit Suivi** project therefore aims to implement:

- A comprehensive web platform for administrators
- A dedicated mobile application for parents and teachers

This solution simplifies daily management, improves pedagogical child monitoring, and strengthens interaction between the institution, families, and educational staff.

1.2.2 Presentation of the Application

The **Petit Suivi** application is a school management platform dedicated to preschool institutions.

It addresses three major needs:

For Administration:

- Centralized management of users, classes, and enrollments
- Payment tracking and registration fees management
- Configuration of system parameters and global contacts
- Meal orchestration and detailed student allergy handling
- Custom event planning and validation
- Activity scheduling and competence criterion management
- Advanced reporting and financial data visualization

For Teachers:

- Class and child attendance tracking
- Reviewing planned activities for their assigned classes
- Evaluating child competences and submitting behavioral reports
- Capturing and sending photos to parents
- Viewing notifications

For Parents:

- Submitting child registrations and updating medical information
- Monitoring child attendance, activities, and competence evaluations
- Viewing and downloading shared photos
- Reviewing monthly payment history and receipts
- Receiving real-time system notifications
- Managing profiles and accessing support services

The application is structured into two complementary components:

1. Mobile Component

The mobile application primarily targets parents and teachers.
Main features include:

- Authentication (login/registration) with validation workflow
- Role-based dashboard (parent/teacher)
- Child monitoring (attendance, activities, specific medical alerts)
- Teacher tools (class management, competence evaluations, reporting)
- Parent tools (payment history, notifications, profile management)
- Real-time photo gallery sharing
- Support and contact interface

2. Web Administration Component

The administrative web platform includes:

- **Dashboard:** global indicators and operational statistics
- **User Management:** administration of teacher and parent accounts
- **System Settings:** global configurations, fees, and contacts
- **Class Management:** levels, capacities, and teacher assignments
- **Enrollment Management:** request validation, filtering, and assignment
- **Payment Management:** tracking transactions, monthly fees, and generating receipts
- **Activity & Event Management:** planning, calendar assignment, and execution tracking
- **Meal Orchestration:** weekly planning, catalog updates, and allergy handling
- **Reporting:** financial indicators, payment monitoring, and dietary exception tracking

1.2.3 Objectives to be Achieved

The Petit Suivi application aims to:

- Centralize the management of enrollments, payments, classes, activities, and meals
- Ensure child safety through strict tracking of medical / dietary needs (allergies) and real-time attendance
- Reduce manual administrative tasks and parameter configurations
- Streamline daily operations and financial reporting

- Improve pedagogical monitoring through precise competence evaluations
- Strengthen communication between administration, staff, and families via photo sharing and notifications
- Provide reliable indicators and comprehensive dashboards for optimal decision-making

1.3 Project Context

The management of educational institutions, particularly in early childhood, entails complex administrative and pedagogical challenges. Currently, many local preschools rely on traditional, paper-based methods or disjointed digital tools (such as separate messaging groups, spreadsheets, and paper notebooks). This lack of centralization often leads to inefficiencies: high administrative burdens, fragmented parent-teacher communication, difficulties tracking payments, and challenges in maintaining comprehensive records of child development.

In this context, there is a growing need for digital transformation. Institutions require comprehensive solutions that bring administrators, teachers, and parents onto a single, intuitive platform to streamline operations and enhance the overall educational experience.

1.4 Analysis of Existing Applications

This section presents a comparative study of similar applications already available on the market. The objective is to analyze existing solutions in order to understand their strengths and weaknesses, identify best practices, and highlight the added value that the Petit Suivi application will provide.

Studying existing platforms allows us to:

- Identify commonly implemented features.
- Detect functional or technical limitations.
- Evaluate usability and ergonomics.
- Determine opportunities for innovation and improvement.

Two major applications have been selected for analysis: Brightwheel and Kinderpedia, both widely used in preschool and early childhood education management.

1.4.1 Brightwheel Application

Brightwheel is one of the leading childcare management software platforms, originally developed in the United States. It offers an all-in-one platform covering attendance tracking, billing, staff management, and parent communication. This solution proposes several advanced features:

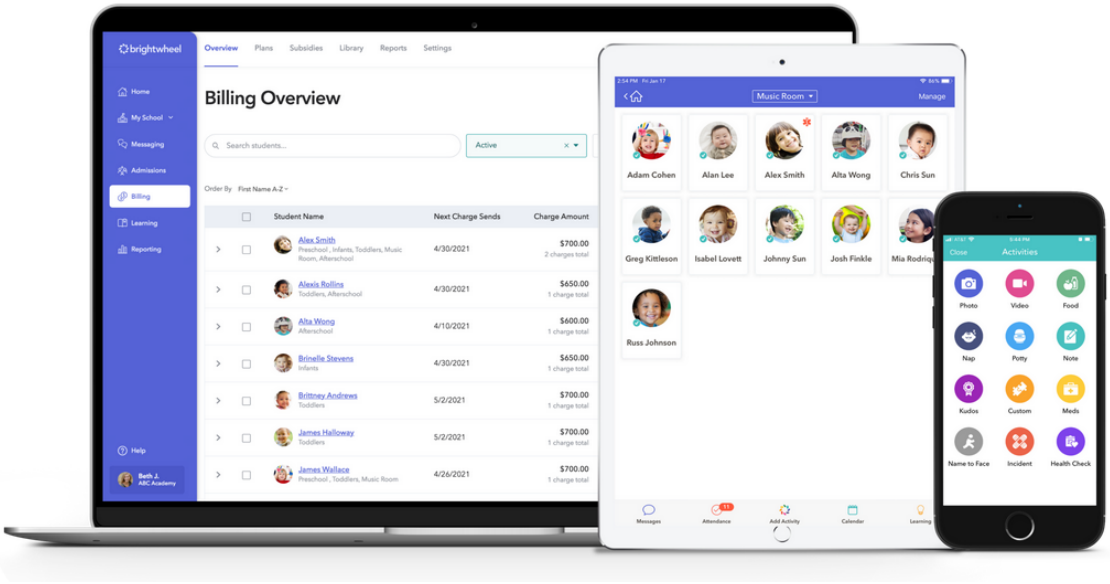


Figure 1.1: Brightwheel platform interface

- **Comprehensive Features:** Includes robust billing and automated invoicing tools to streamline financial management.
- **Real-Time Updates:** Enables excellent real-time photo and video sharing, ensuring parents stay connected.
- **Ease of Use:** Provides a very intuitive mobile interface tailored specifically for teachers' daily tasks.

Although Brightwheel offers a complete range of features adapted to early childhood management, several factors limit its direct adoption for our host institution:

- **Cost limitations:** High recurring subscription costs designed for international markets represent a significant, ongoing financial burden that can be prohibitive for local nurseries.
- **Customization limitations:** The platform is difficult to adapt to specific local administrative procedures, unique class structures, and targeted pedagogical competency frameworks required by our curriculum.
- **Data Control limitations:** Relying on external, international servers implies less control over sensitive private data and documents, a crucial aspect for the institution.

1.4.2 Kinderpedia Application

Kinderpedia is an award-winning European school management software focusing heavily on family engagement, automated invoicing, and digital class management. It stands out by providing deeply integrated communication modules with several key features:

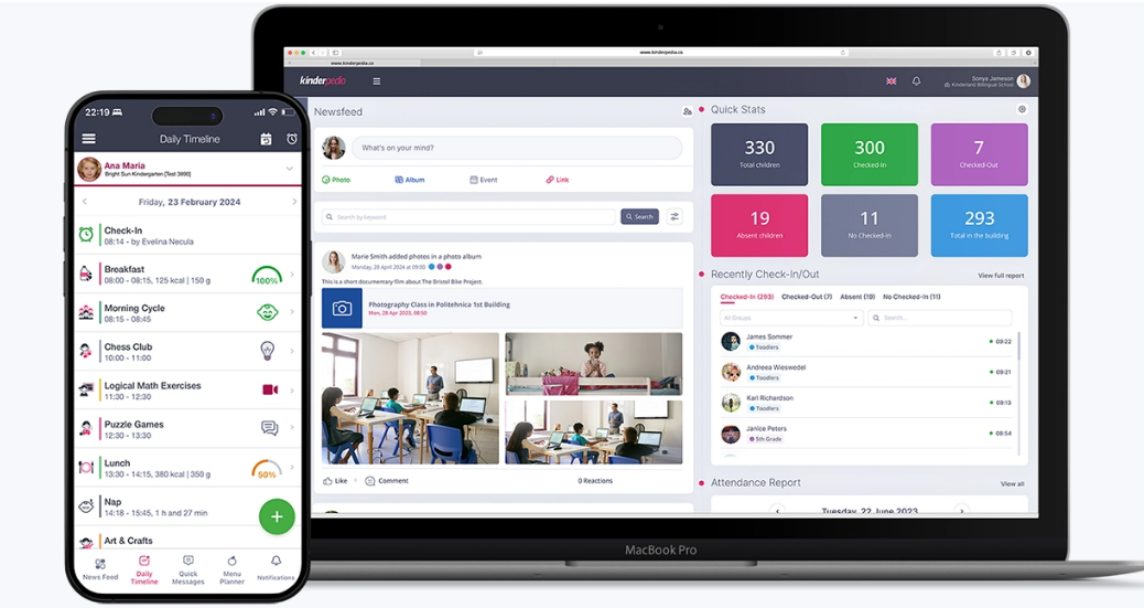


Figure 1.2: Kinderpedia platform interface

- **Multi-language Support:** Offers an interface available in several languages, making it accessible to diverse staff and parents.
- **Family Engagement:** Features dedicated tools designed to keep parents heavily involved in tracking a child's academic and behavioral progress.
- **Automated Reports:** Generates comprehensive and excellent attendance and financial reports for administrators.

While Kinderpedia is highly regarded in the European market, certain aspects restrict its suitability for our specific needs:

- **Platform Complexity:** The abundance of advanced modules makes the platform somewhat bloated for small-to-medium nurseries, resulting in a steep learning curve and overly complex onboarding for staff.
- **Fixed Workflows:** It lacks the precise flexibility required to fully customize meal tracking processes and detailed behavioral evaluations according to purely local operational standards.
- **Pricing Model:** The structure heavily charges per student or requires expensive premium tiers to access the necessary reporting functionalities.

1.4.3 Synthesis and Comparison

Based on the analysis of these platforms, both Brightwheel and Kinderpedia provide strong communication, attendance, and administrative features. However, they remain limited in terms of local customization, deployment cost, and full control over institutional data, which reinforces the relevance of developing Petit Suivi as a solution tailored to the host institution.

1.5 Problem Statement and Proposed Solution

The critical analysis of the existing systems highlights several limitations. While solutions like Brightwheel and Kinderpedia offer rich functionalities, they rely on expensive, recurring subscription models targeted at European and North American markets. Furthermore, these platforms lack the flexibility to adapt to the specific administrative workflows, customized meal planning, and localized pedagogical tracking methods required by our institution. Finally, relying on third-party international cloud solutions raises concerns regarding long-term data ownership and localization.

Proposed Solution: To address these challenges, we propose the development of **Petit Suivi**. This custom-tailored solution will provide a robust, unified platform consisting of a web-based administration panel and a dedicated mobile application for parents and teachers. By developing this system in-house, the institution will benefit from:

- **Total Ownership:** Elimination of third-party licensing fees and full control over data hosting and security.
- **Tailored Functionality:** Workflows specifically designed around the institution's actual operations, from customized enrollment forms to local competency tracking.
- **Centralized Communication:** A seamless digital bridge replacing fragmented communication channels, offering real-time notifications, absence alerts, and photo sharing.

1.6 Conclusion

In this chapter, we established the general framework of the project. We clearly defined the mission, identified the primary actors, and outlined the system's core objectives. Through a critical study of existing applications, we gathered valuable insights into the market's current capabilities and limitations. These conclusions justify our decision to develop a fully custom solution. With the context and requirements clearly defined, the next chapter will focus on the detailed functional requirements and the specifications of the Petit Suivi system.

Chapter 2

Requirements Gathering

2.1 Introduction

This chapter focuses on the requirements gathering phase, which is a fundamental step in any software development process. By analyzing the needs of all stakeholders, we define the functional scope of the **Petit Suivi** system.

We begin by identifying the key actors who interact with the platform, then model the information system context. Next, we identify and classify all use cases using a UML use case diagram, and finally provide detailed textual descriptions for each use case scenario.

2.2 Identification of System Actors

An actor represents any external entity whether a person, a device, or another system that interacts with the system under study. The Petit Suivi platform identifies three primary actors:

- **Parent:** Accesses the system through the mobile application. Parents can register themselves and their children, track child activities, attendance, and competence evaluations, view photos, manage their profile, consult payment history, receive notifications, and contact nursery support.
- **Teacher:** Accesses the system through the mobile application. Teachers can view planned activities for their assigned class, manage class attendance, evaluate child competences, send photos to parents, manage their profile, view notifications, and contact support.
- **Administrator (Admin):** Accesses the system through the web administration panel. The admin manages all aspects of the nursery including: user accounts (parents and teachers), class management, enrollment approvals, payment tracking, activity and competence management, meal planning, event management, system configuration, and report generation.

The following table summarizes each actor alongside their role and primary interactions with the system:

2.3 Context of the Information System

The context of the system is represented by a context diagram that shows the interactions between the domain of study and its exchanges with the environment (Figure 2.1).

Table 2.1: System Actors and Their Roles

Actor	Interface	Description
Parent	Mobile App	Registers accounts, tracks children (activities, attendance, competences), views photos, consults payment history, manages profile, and contacts support.
Teacher	Mobile App	Views and tracks planned activities, manages class attendance, evaluates child competences, sends photos, manages profile, and views notifications.
Administrator	Web Panel	Manages users, classes, enrollments, payments, activities, meals, events, system parameters, and generates reports via the administrative dashboard.

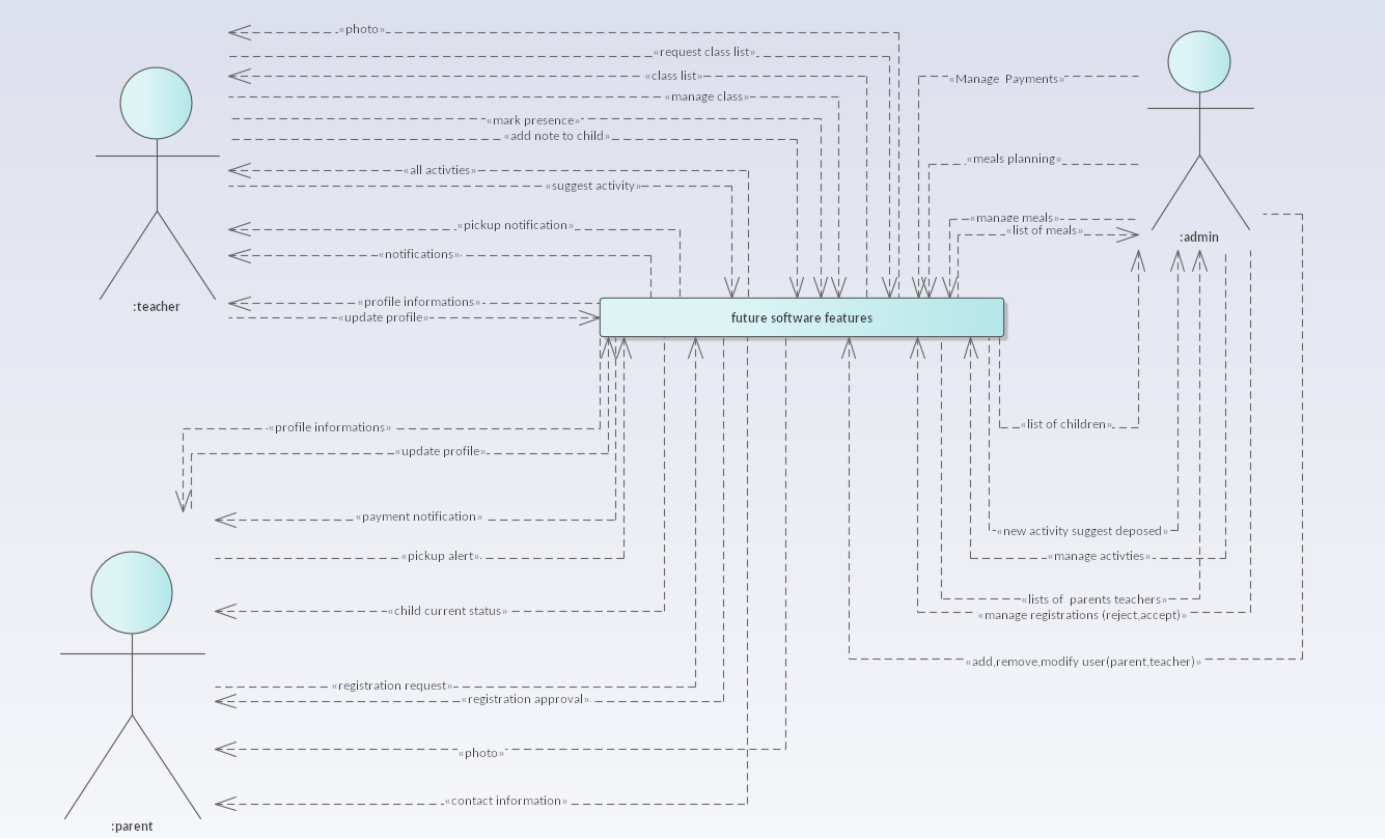


Figure 2.1: Context Diagram of the Information System

2.4 Identification of Use Cases

This section presents the global use case diagram of the Petit Suivi system, identifying all functional interactions between actors and the system.

2.4.1 Use Case Diagram

Figure 2.2 presents the global use case diagram, illustrating the relationships between the three actors (Parent, Teacher, and Administrator) and the system's functionalities. All

use cases include the “Authenticate” use case as a precondition.

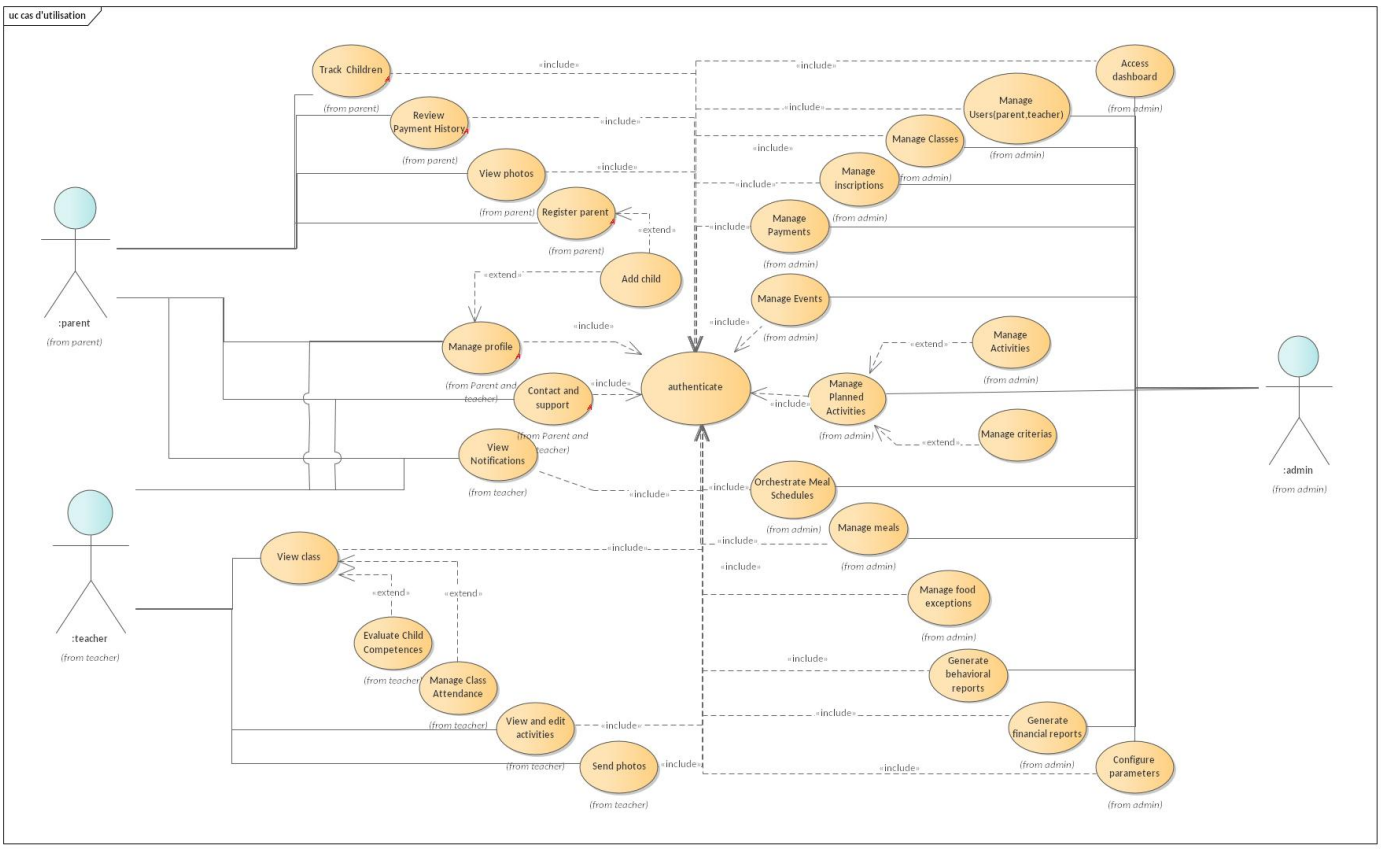


Figure 2.2: Global Use Case Diagram of the Petit Suivi System

2.4.2 Use Case Summary Table

The following table provides an overview of all identified use cases, organized by actor:

Table 2.2: Summary of Use Cases by Actor

Actor	Use Case	Brief Description
Parent	Track Children	View activities, attendance status, and competence evaluations of enrolled children.
	View Photos	Browse and download photos shared by teachers.
	Register Parent	Create a new parent account with personal and authentication information.
	Add Child	Register a new child with medical information and enrollment details.
	Review Payment History	Consult monthly payment records.
	View Class	View assigned classes and select a class to access its details.
Teacher		Continued on next page

Table 2.2 – continued from previous page

Actor	Use Case	Brief Description
	View and Edit Activities	Browse and manage planned activities for the assigned class.
	Send Photos	Upload and share photos with parents.
	Manage Class Attendance	Record daily presence/absence for all students.
	Evaluate Child Competences	Assess competences and submit behavioral reports.
Parent, Teacher	View Notifications	Browse and read system notifications.
	Contact and Support	Access nursery contact details (phone, email, address).
	Manage Profile	View personal information and account details.
	Authenticate	Log in to the mobile application via email and password.
Administrator	Access Dashboard	View aggregated KPIs, charts, and statistics.
	Manage Activities	Create, modify, and delete activities.
	Manage Criterias	Add or remove competence criteria.
	Manage Payments	Record monthly payments and registration fees.
	Manage Inscriptions	Approve, reject, or archive enrollment requests with class assignment.
	Manage Classes	Create, modify, and archive classes with teacher assignments.
	Manage Users	Create, modify, approve, reject, or archive parent and teacher accounts.
	Orchestrate Meal Schedules	Compose weekly menus, save/load templates.
	Manage Meals	Add or remove meals from the catalog.
	Manage Planned Activities	Assign/unassign activities on the academic calendar.
	Configure Parameters	Update global system settings (fees, contacts, AI).
	Manage Events	Create, delete, and mark events as executed.
	Manage Food Exceptions	Consult dietary exceptions.
	Generate Behavioral Reports	Analyze and view behavioral signalement data.
	Generate Financial Reports	Analyze and view financial payment data.

Continued on next page

Table 2.2 – continued from previous page

Actor	Use Case	Brief Description
	Authenticate	Log in to the web panel via email and password.

2.5 Textual Description of Use Cases

This section provides a detailed textual description for each use case, following a structured format that specifies the actors involved, preconditions, the basic flow (step-by-step interaction between the actor and the system), and alternate or exception flows where applicable.

2.5.1 1. Parent Use Cases

Use Case: 1.1 Track Children

Scenario: Consult today's scheduled activities for a child
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent submits their authentication credentials.
2	System	The system authenticates the user.
3	System	The system retrieves the assigned children along with their active inscriptions.
4	System	The system displays the registered children to the Parent.
5	Actor	The Parent selects a specific child.
6	System	The system extracts the assigned class from the child's active inscription.
7	System	The system fetches the daily plan and today's scheduled activities for the class.
8	System	The system renders a list of today's scheduled activities.

Scenario: Check daily attendance status of a child
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent submits their authentication credentials.
2	System	The system authenticates the user.
3	System	The system retrieves the assigned children along with their active inscriptions.
4	System	The system displays the registered children to the Parent.
5	Actor	The Parent selects a specific child.
6	System	The system fetches the monthly attendance records directly using the child's ID.
7	System	The system renders a monthly calendar view .

Scenario: Review acquired competences of a child
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent submits their authentication credentials.
2	System	The system authenticates the user.
3	System	The system retrieves the assigned children along with their active inscriptions.
4	System	The system displays the registered children to the Parent.
5	Actor	The Parent selects a specific child.
6	System	The system fetches the child's evaluations directly using the child's ID.
7	System	The system categorizes and renders the competences into groups by activity with status badges.

Use Case: 1.2 View Photo

Scenario: Browse photo gallery of a child
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent submits their authentication credentials.
2	System	The system authenticates the user.
3	System	The system retrieves the assigned children along with their active inscriptions.
4	System	The system displays the registered children to the Parent.
5	Actor	The Parent selects a specific child's card.
6	System	The system fetches all photos shared with this parent for the selected child.
7	System	The system renders a gallery grid view of the child's photos, providing a "Download" button for each photo.
8	Actor	The Parent scrolls through the gallery to browse the available photos.
9	Actor	The Parent taps the "Download" button on a specific photo.
10	System	The system downloads the full-resolution image binary to the device's local storage.
11	System	The system displays a brief "Photo saved" confirmation toast at the bottom of the screen.

Use Case: 1.3 Register Parent

Scenario: Submit registration form with parent and children information (new account)
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent opens the application for the first time and taps the "S'inscrire" (Register) button.
2	System	The system displays the parent account creation form with fields for personal details and credentials.
3	Actor	The Parent fills in all their personal details, creates a password, and taps the "Suivant" button.
4	System	The system validates the input fields client-side.
5	System	The system displays the child information form with fields for basic child details, class type, and payment method.
6	Actor	The Parent fills in the child's details and selects the preferred options.
7	System	The system displays the medical form section requesting allergies, conditions, and emergency contacts.
8	Actor	The Parent fills in the medical information, and taps "Soumettre".
9	System	The system creates the Parent account record configured with a "Pending" approval status.
10	System	The system iteratively creates each Child record securely linked to the new parent.
11	System	The system creates an Inscription record with a "Pending" status and calculates the initial fees.
12	System	The system stores the submitted medical form data and links it to the inscription.
13	System	The system displays a success screen indicating the account is awaiting administration approval.

Scenario: Reinscribe an existing child for a new academic year (existing account)
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Parent submits their authentication credentials.
2	System	The system authenticates the user.
3	System	The system loads the Parent Dashboard and displays a banner indicating the reinscription period is open.
4	Actor	The Parent taps the "Réinscrire" (Reinscribe) button next to an existing child's profile on the Dashboard.
5	System	The system fetches the child's existing profile and previously submitted medical form data.
6	System	The system displays a reinscription form pre-filled with the current information.
7	Actor	The Parent reviews the information, updates outdated details, selects the payment method for the new year, and taps "Confirmer".
8	System	The system validates the updated payload server-side.
9	System	The system creates a NEW Inscription record with a "Pending" status for the upcoming academic year, linked to the existing Child ID.
10	System	The system stores the updated medical form data and links it directly to the newly created Inscription.
11	System	The system displays a success toast indicating the reinscription is awaiting administration approval.

Use Case: 1.4 Review Payment History

Scenario: Consult monthly payment records of a child
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent logs into the mobile application.
2	System	The system authenticates the Parent and displays the Parent Dashboard.
3	Actor	The Parent taps on the "Paiements" (Payments) section from the bottom navigation or Dashboard menu.
4	System	The system retrieves all payment records associated with the parent's children.
5	System	The system displays a payment overview screen showing: the total annual fee, the total amount paid to date, and the remaining balance, with a visual progress bar indicating overall payment completion.
6	Actor	The Parent scrolls down to view the detailed monthly payment breakdown.

Use Case: 1.5 Add Child (Extended)

This is an extended use case invoked from Use Case 1.3 (Register Parent) during initial registration, and from Use Case 3.2 (Manage Profile) when adding additional children.

Scenario: Register a new child with medical and enrollment information
Path Type: Basic Path

Step	Assignee	Action
1	System	The system displays the child registration form with fields: Child Full Name, Date of Birth, Gender, Preferred Class Type (Maternelle/Préscolaire), and Payment Method.
2	Actor	The Parent fills in the child's personal details and selects the preferred class type and payment method.
3	System	The system displays the medical form section with fields for allergies, chronic conditions, medications, dietary restrictions, and document upload.
4	Actor	The Parent fills in the medical information and taps "Soumettre" (Submit).
5	System	The system creates the child record linked to the Parent ID, creates a new Inscription with "Pending" status, and stores the medical form in the database.
6	System	The system displays a success message confirming the enrollment request has been submitted and is awaiting administration approval.

2.5.2 2. Teacher Use Cases

Use Case: 2.1 View and Edit Activities

Scenario: Browse all planned activities of a class for the current academic year
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher opens the mobile application on their device.
2	System	The system displays the Login Screen requesting CIN and Password.
3	Actor	The Teacher enters their CIN and password and taps "Se Connecter" (Log In).
4	System	The system validates the credentials via '/login/teacher', verifies the teacher's active status, issues a session token, and redirects to the Teacher Dashboard.
5	System	The system displays the Teacher Dashboard showing the assigned class(es) and a summary of today's scheduled activities.
6	Actor	The Teacher taps on their assigned class card to view class details.
7	System	The system retrieves the classes assigned to this teacher for the current academic planning year.
8	Actor	The Teacher navigates to the "Activités" (Activities/Planning) section within the class view.
9	System	The system displays a calendar-style list of all planned activities for the academic year.
10	Actor	The Teacher applies a date filter by selecting a specific month from the month picker dropdown.
11	System	The system filters and refreshes the activity list to show only activities scheduled for the selected month.
12	Actor	The Teacher taps on a specific activity to view its full details.
13	System	The system displays the activity detail card showing: title, description, type, date, associated competence criteria, and an option to mark the activity status as "Executed" or "Not Executed".

Use Case: 2.2 Send Photos

Scenario: Upload and share photos for a specific child, an entire class, or a selected group of children
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher logs into the mobile application with their CIN and password.
2	System	The system authenticates the Teacher and displays the Teacher Dashboard.
3	Actor	The Teacher navigates to the "Photos" section from the bottom navigation or class view.
4	System	The system displays the photo management screen showing previously uploaded photos and an "Ajouter" (Upload) floating action button.
5	Actor	The Teacher taps the "Ajouter" button to initiate a new photo upload.
6	System	The system opens the device's media picker interface, allowing the Teacher to choose between the camera or the photo gallery.
7	Actor	The Teacher selects one or multiple photos from the gallery or captures a new photo using the camera.
8	System	The system displays the selected photo(s) as thumbnails in a preview area, along with a recipient selection interface.
9	Actor	The Teacher selects the target audience by choosing one of three options: "Toute la classe" (entire class), a specific child from a searchable dropdown, or multiple children via checkboxes.
10	Actor	The Teacher optionally adds a caption or description text for the photo(s) and taps the "Partager" (Share) button.
11	System	The system submits the image file(s), selected recipients (child IDs or class ID), and the caption text.
12	System	The system uploads and stores the image(s) on the server, creates 'ChildPhoto' and 'ChildPhotoRecipient' records in the database linking each photo to the selected children.
13	System	The system triggers push notifications to the relevant parents informing them that new photos are available for their child.
14	System	The system displays a "Photos partagées avec succès" success toast and refreshes the photo gallery to show the newly uploaded entries.

Use Case: 2.3 Manage Class Attendance

Scenario: Record daily attendance for all children in a class
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher logs into the mobile application.
2	System	The system authenticates the Teacher and displays the Teacher Dashboard with their assigned class.
3	Actor	The Teacher navigates to the "Présence" (Attendance) section for their class.
4	System	The system retrieves the full list of enrolled children in the class.
5	System	The system displays a student roster for the current date, listing each child's name and photo avatar, with toggle switches defaulting to "Absent" status.
6	Actor	The Teacher begins marking attendance by toggling each child's status switch to "Présent" (Present) for children who are in class.
7	Actor	The Teacher leaves the toggle in the "Absent" position for children who are not present, and optionally taps on an absent child's row to add an absence reason note.
8	System	The system validates in real-time that no duplicate attendance record exists for the same child on the same date.
9	Actor	The Teacher reviews the complete attendance list to verify correctness, then taps the "Enregistrer la Présence" (Save Attendance) button.
10	System	The system processes the bulk attendance data: each child's ID, the date, and their presence status.
11	System	The system saves all attendance records for the specific date in the 'presence' table in the database.
12	System	The system displays a "Présence enregistrée avec succès" success notification toast, and updates the attendance view to show the saved records with green/red status indicators.

Use Case: 2.4 Evaluate Child Competences

Scenario: Assess each competence of an activity as acquired or needs reinforcement
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher logs into the mobile application.
2	System	The system authenticates the Teacher and displays the Teacher Dashboard.
3	Actor	The Teacher navigates to their assigned class and selects a specific child from the student roster.
4	System	The system displays the child's profile page with sections for attendance, evaluations, and behavioral reports.
5	Actor	The Teacher taps on the "Évaluations" (Evaluations) tab.
6	System	The system fetches today's planned activities and their associated competence criteria.
7	System	The system displays the list of today's activities, each expandable to reveal its individual competence criteria with evaluation toggle buttons.
8	Actor	The Teacher taps on an activity to expand it, then selects a competence criterion and marks it as "Acquis" (Acquired) by tapping the green checkmark button.
9	Actor	The Teacher selects another competence criterion and marks it as "À Renforcer" (Needs Reinforcement) by tapping the orange warning button.
10	Actor	The Teacher repeats the evaluation process for all competence criteria across all today's activities for this child.
11	Actor	The Teacher taps the "Soumettre les Évaluations" (Submit Evaluations) button to save all assessments.
12	System	The system submits all evaluation data, storing each competence's status linked to the child, the activity, and the evaluation date.
13	System	The system refreshes the evaluation view showing the saved assessments with color-coded status badges and displays a "Évaluations enregistrées" success confirmation toast.

Scenario: Submit a behavioral report for a child
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher logs into the mobile application.
2	System	The system authenticates the Teacher and displays the Teacher Dashboard.
3	Actor	The Teacher navigates to their assigned class and selects a specific child from the student roster.
4	System	The system displays the child's profile page.
5	Actor	The Teacher taps the "Ajouter un Signalement" (Add Behavioral Report) button.
6	System	The system displays a behavioral report form with: a text area for the observation description, a severity level selector (dropdown with levels such as "Info", "Avertissement", "Urgent"), and a date field defaulting to today.
7	Actor	The Teacher writes a detailed observation describing the child's behavior.
8	Actor	The Teacher selects a severity category from the dropdown and verifies the date.
9	Actor	The Teacher taps the "Envoyer le Signalement" (Send Report) button.
10	System	The system submits the child ID, description text, severity level, and date.
11	System	The system saves the signalement record in the database, linking it to the child, the teacher, and the date.
12	System	The system triggers a notification to the Administrator via the Web Panel alerting them of the new behavioral report.
13	System	The system returns the Teacher to the child's profile page and displays a "Signalement envoyé avec succès" success toast.

Use Case: 2.5 View Class

Scenario: View assigned class details and student roster
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher logs into the mobile application.
2	System	The system authenticates the Teacher and displays the Teacher Dashboard with their assigned class(es).
3	Actor	The Teacher taps on their assigned class card to view class details.
4	System	The system retrieves the class information including: class name, type, academic year, and the full list of enrolled children.
5	System	The system displays the class detail view showing: the class name, type, the total number of enrolled students, and a scrollable student roster with each child's name and photo avatar.
6	Actor	The Teacher scrolls through the student roster to review all enrolled children.
7	Actor	The Teacher taps on a specific child's name in the roster.
8	System	The system navigates to the child's profile page, providing access to attendance tracking, competence evaluations, and behavioral report submission.

2.5.3 3. Teacher and Parent Use Cases

Use Case: 3.1 Contact and Support

Scenario: Consult nursery contact information and address
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The User (Teacher or Parent) opens and logs into the mobile application.
2	System	The system authenticates the user and displays their respective Dashboard (Parent or Teacher).
3	Actor	The User taps on the hamburger menu icon or navigates to the "À Propos" / "Support" section from the bottom navigation bar.
4	System	The system displays the Support & Contact page layout with multiple information sections.
5	System	The system fetches the nursery's global configuration parameters including contact details.
6	System	The system populates and displays the nursery's full contact information: official name (company_name), physical address (kindergarten_address), phone number (contact_phone), and email (contact_email).
7	Actor	The User taps on the phone number displayed on the screen.
8	System	The system invokes the device's native phone dialer with the nursery's phone number pre-filled, ready to call.
9	Actor	The User taps on the email address displayed.
10	System	The system opens the device's default email client with the nursery's email address pre-filled in the "To" field.

Use Case: 3.2 Manage Profile

Scenario: View personal profile details
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The User (Teacher or Parent) logs into the mobile application.
2	System	The system authenticates the user and displays their Dashboard.
3	Actor	The User taps on their Avatar icon or "Profil" menu item in the navigation drawer.
4	System	The system retrieves the authenticated user's profile data from the database.
5	System	The system displays the Profile page with the user's personal information: full name, CIN, email address, phone number, address, role label, and account creation date.
6	Actor	The User reviews their displayed personal information for accuracy.
7	Actor	The User scrolls down to view additional sections such as linked children (for Parents) or assigned classes (for Teachers).
8	System	The system displays the associated entities: for a Parent, it shows a list of their registered children with enrollment status; for a Teacher, it shows their assigned class name and student count.
9	Actor	The User taps on a specific linked entity (child or class) to navigate to its detail view.
10	System	The system navigates to the corresponding child tracking view or class overview.

Scenario: Update account password
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The User navigates to the "Profil" page and taps the "Changer le Mot de Passe" (Change Password) button.
2	System	The system displays a password change form with three fields: "Mot de passe actuel" (Current Password), "Nouveau mot de passe" (New Password), and "Confirmer le nouveau mot de passe" (Confirm New Password).
3	Actor	The User types their current password in the first field.
4	Actor	The User types the new desired password and confirms it by retyping it in the confirmation field.
5	System	The system performs client-side validation: checks that the new password meets minimum length requirements, and that the "New Password" and "Confirm" fields match exactly.
6	Actor	The User taps the "Mettre à Jour" (Update) button.
7	System	The system submits the current password and the new password.
8	System	The system verifies server-side that the current password matches the hashed password stored in the database for the authenticated user.
9	System	If the current password is correct, the system hashes the new password using bcrypt and updates the user's password record in the 'accounts' table.
10	System	The system invalidates the current session token, forcing the user to log in again with their new credentials.
11	System	The system redirects the user to the Login Screen and displays a "Mot de passe mis à jour avec succès. Veuillez vous reconnecter." success toast.

Use Case: 3.3 View Notifications

Scenario: Browse and read received notifications
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The User (Teacher or Parent) opens the mobile application and logs in.
2	System	The system authenticates the User and displays the Dashboard.
3	System	The system fetches the unread notifications count and displays a red badge with the unread count on the Notification Bell icon in the top navigation bar.
4	Actor	The User taps the Notification Bell icon.
5	System	The system retrieves the full list of notifications.
6	System	The system displays a scrollable list of notifications sorted by date (newest first), with unread notifications highlighted with a bold font and a colored left-border indicator.
7	Actor	The User scrolls through the list and taps on a specific unread notification.
8	System	The system marks the notification as read in the database.
9	System	The system displays the full notification content in an expanded view, showing: the title, the detailed message body, the timestamp, and the sender or event type.
10	Actor	The User taps the "Tout marquer comme lu" (Mark All as Read) button at the top of the notifications list.
11	System	The system updates all unread notifications to "read" status.
12	System	The system refreshes the notification list UI, removing bold styling from all items, and resets the badge count on the Bell icon to zero.

2.5.4 4. Admin Use Cases

Use Case: 4.1 Access Dashboard

Scenario: View aggregated statistics across all academic years
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Administrator opens a web browser and navigates to the Web Panel URL.
2	System	The system displays the Admin Portal login page with Email and Password fields.
3	Actor	The Admin enters their secure email and password credentials and clicks the "Se Connecter" (Log In) button.
4	System	The system validates the credentials against administrative permissions, and issues a Bearer token.
5	System	The system redirects the Admin to the main Dashboard page and compiles live statistics.
6	System	The system displays the Dashboard with aggregated KPI cards showing: total number of registered children, total number of active teachers, total number of pending inscriptions, total collected revenue, and outstanding payment amounts.
7	Actor	The Admin reviews the summary statistics cards at the top of the Dashboard.
8	System	The system generates and renders interactive charts below the KPI cards: a payment status distribution chart, an enrollment trend chart, and a class occupancy overview.
9	Actor	The Admin hovers over a specific chart segment to view detailed metric tooltips.
10	System	The system displays a tooltip overlay with the precise numerical values and percentages for the hovered chart element.
11	Actor	The Admin selects a different academic year from the year filter dropdown to compare historical data.
12	System	The system refetches statistics for the selected planning year and dynamically updates all KPI cards and charts to reflect the filtered data.

Use Case: 4.2 Manage Activities

Scenario: Create a new activity
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel and navigates to the "Activités" section from the sidebar menu.
2	System	The system fetches all existing activities and displays them in a DataGrid table with columns: Name, Description, Type, and Actions.
3	Actor	The Admin reviews the current list of activities and clicks the "Ajouter une Activité" (Add New Activity) button.
4	System	The system opens a modal dialog with a blank activity creation form containing fields: Title (text input), Description (multi-line text area), and Type (dropdown selector with options like "Éducative", "Sportive", "Artistique").
5	Actor	The Admin enters the activity title, writes a detailed description, and selects the activity type from the dropdown.
6	System	The system validates the form inputs client-side: ensures the title is not empty and does not exceed the maximum character length.
7	Actor	The Admin clicks the "Enregistrer" (Save) button.
8	System	The system creates the activity with the title, description, and type data.
9	System	The system validates server-side that no activity with the same title already exists in the database for the current year.
10	System	The system creates the activity record in the 'activities' table and assigns it a unique ID.
11	System	The system closes the modal, refreshes the activities DataGrid to include the new entry at the top, and displays a "Activité créée avec succès" success toast notification.

Scenario: Delete an existing activity
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Activités" section from the sidebar menu.
2	System	The system displays the full list of existing activities in a DataGrid table.
3	Actor	The Admin identifies an outdated or unused activity and clicks the "Supprimer" (Delete) icon button in the Actions column next to that activity's row.
4	System	The system checks whether the activity is currently assigned to any day in the planning calendar for the current academic year.
5	System	If the activity IS scheduled: The system displays an error alert: "Suppression impossible: cette activité est actuellement planifiée. Veuillez d'abord la retirer du planning."
6	System	If the activity is NOT scheduled: The system displays a confirmation pop-up dialog: "Êtes-vous sûr de vouloir supprimer cette activité ? Cette action est irréversible."
7	Actor	The Admin reads the confirmation message and clicks the "Confirmer la Suppression" (Confirm Delete) button.
8	System	The system removes the activity from the database.
9	System	The system also cascades the deletion to remove any associated competence criteria linked to this activity.
10	System	The system refreshes the activities DataGrid, removing the deleted entry, and displays a "Activité supprimée" success toast.

Scenario: Modify activity details
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Activités" section and reviews the activity list.
2	System	The system displays the existing activities in the DataGrid.
3	Actor	The Admin clicks the "Modifier" (Edit) icon button next to a specific activity.
4	System	The system opens a modal dialog populated with the activity's current data: title, description, and type.
5	Actor	The Admin modifies the activity title.
6	Actor	The Admin updates the description text to provide more detailed instructions and clicks "Mettre à Jour" (Update).
7	System	The system validates the updated fields: checks that the new title is not empty and doesn't conflict with another existing activity's title.
8	System	The system updates the activity record in the database.
9	System	The system saves the modifications, closes the modal dialog, refreshes the DataGrid to reflect the changes, and displays a "Activité mise à jour" success toast.
10	System	The system propagates the name/description changes so that teachers viewing planned activities see the updated information.

Use Case: 4.3 Manage Payments

Scenario: Record a monthly payment as received
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel and navigates to the "Paiements" (Payments) section from the sidebar.
2	System	The system sends parallel GET requests to '/admin/payments', '/admin/inscriptions', '/admin/parameters', and '/admin/plannings' to load all payment data.
3	System	The system displays the Payments interface with summary statistic cards at the top (Payés, Partiels, En attente, Mensuel, Annuel) and a DataGrid table below listing all children with approved inscriptions.
4	Actor	The Admin uses the "Année Scolaire" (School Year) dropdown filter to select the current academic year.
5	System	The system filters the DataGrid to show only children enrolled in the selected academic year.
6	Actor	The Admin locates a specific child using the DataGrid search/filter toolbar and clicks the "Détails & Paiements" button in the Actions column.
7	System	The system opens a detailed payment dialog showing: the child's name, class, total annual amount, amount paid so far, remaining balance, a visual progress bar, the payment method (Monthly/Annual), and a monthly payment schedule with checkboxes for each month.
8	Actor	For a monthly payment: The Admin clicks the checkbox next to the next unpaid month in the schedule.
9	System	The system displays a confirmation dialog: "Confirmer le paiement de [amount] TND pour [month] ?" with "Confirmer" and "Annuler" buttons.
10	Actor	The Admin clicks "Confirmer" to validate the payment.
11	System	The system submits the amount, date, and target month.
12	System	The system records the transaction in the database, updates the payment status (from "En attente" to "Partiel" or from "Partiel" to "Payé" if fully paid), and recalculates the remaining balance.
13	System	The system refreshes the payment dialog and the DataGrid row, updates the progress bar, and marks the paid month's checkbox as green with a "Payé le [date]" label.
14	System	The system enables a "Reçu" (Receipt) button next to the paid month, allowing the Admin to generate and print a payment receipt.

Scenario: Mark registration fees as paid
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Paiements" section from the sidebar menu.
2	System	The system displays the Payments DataGrid with all children and their payment information.
3	Actor	The Admin locates a child whose registration fee ("Frais d'Inscription") has not yet been paid, identified by an unchecked checkbox in the "Frais Inscription" column.
4	Actor	The Admin clicks the checkbox in the "Frais Inscription" column for that child's row.
5	System	The system marks the fee as checked.
6	System	The system updates the 'frais_inscription_amount' field in the inscription record, setting it to the globally configured registration fee value from the parameters table.
7	System	The system performs an optimistic UI update, immediately showing the checkbox as checked and displaying the fee amount in the payment history.
8	Actor	The Admin clicks "Détails & Paiements" to verify the registration fee appears in the transaction history.
9	System	The system displays the payment detail dialog, showing the "Frais Annuel d'Inscription" entry at the top of the transaction history with the fee amount highlighted in blue.
10	System	The system provides a "Reçu" (Receipt) button next to the registration fee entry for generating a printable receipt.

Use Case: 4.4 Manage Inscriptions

Scenario: Approve a pending inscription request
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel and navigates to the "Inscriptions" section from the sidebar menu.
2	System	The system displays the inscriptions management interface with summary statistics (Total, Approuvées, En attente, Rejetées, Archivées) and a DataGrid table.
3	Actor	The Admin identifies a pending inscription in the DataGrid, indicated by an orange " En attente" status badge in the "Décision" column.
4	Actor	The Admin clicks the "Voir dossier" (View File) button in the "Fiche médicale" column to review the child's medical form.
5	System	The system opens a medical form review dialog displaying all submitted medical information: allergies, chronic conditions, medications, dietary restrictions, emergency contacts, and uploaded documents.
6	Actor	The Admin reviews the medical form for completeness, closing the dialog once satisfied.
7	Actor	The Admin clicks the decision dropdown in the "Décision" column and selects " Approuver" (Approve).
8	System	The system opens a class assignment dialog titled " Assigner une classe", fetching available classes via GET '/admin/classes' and filtering by the child's preferred type (Maternelle/Préscolaire).
9	System	The system displays a dropdown listing available classes with their current enrollment count and capacity.
10	Actor	The Admin selects the appropriate class from the dropdown and clicks "Confirmer".
11	System	If AI medical analysis is enabled: The system triggers an AI scan of the medical form via POST '/admin/inscriptions/{childId}/rescan-medical', extracting dietary restrictions and health notes.
12	System	The system displays an AI Review dialog showing the extracted dietary comment and health comment, with editable text fields allowing the Admin to review and correct the AI's output before finalizing.
13	Actor	The Admin reviews the AI-generated summaries, makes any corrections if needed, and clicks "Approuver et Enregistrer" (Approve & Save).
14	System	The system saves the AI comments to the database, changes the inscription status to "Approved", assigns the selected class to the child, generates the payment records for the academic year, and sends a notification to the Parent.
15	System	The system refreshes the inscriptions DataGrid, updating the row to show a green " Approuvé" status badge and the assigned class name in a green chip.

Scenario: Reject a pending inscription request
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Inscriptions" section from the sidebar.
2	System	The system displays the inscriptions list with pending, approved, and rejected entries.
3	Actor	The Admin locates a pending inscription and clicks the "Voir dossier" button to review the submitted medical form and child details.
4	System	The system displays the complete medical form and child profile in a review dialog.
5	Actor	The Admin identifies issues with the submission and closes the review dialog.
6	Actor	The Admin clicks the decision dropdown and selects " Rejeter" (Reject).
7	System	The system displays a confirmation dialog: "Êtes-vous sûr de vouloir rejeter cette inscription ?"
8	Actor	The Admin clicks "Confirmer" to proceed with the rejection.
9	System	The system updates the status to rejected.
10	System	The system changes the inscription status to "Rejected" in the database and sends a notification to the Parent informing them of the rejection.
11	System	The system refreshes the DataGrid, updating the row to show a red " Rejeté" status badge, and updates the summary statistics counters.

Scenario: Archive or unarchive an inscription
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Inscriptions" section.
2	System	The system displays the inscriptions list.
3	Actor	The Admin clicks the "Voir dossier" button or action menu to access archiving options.
4	Actor	The Admin selects "Archiver" or "Désarchiver".
5	System	The system checks if the assigned class is archived or if the planning is archived (for unarchiving).
6	System	If unarchiving to an archived class/planning, the system may display a warning or block the action.
7	System	Otherwise, the system toggles the $`$is$_a rchived`$ $`status of the inscription`$.
8	System	The system refreshes the DataGrid, updating the rows in active and archived tabs.
9	System	The system displays a success toast confirming the action.

Use Case: 4.5 Manage Classes

Scenario: Create a new class
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Administrator logs into the Web Panel with secure credentials.
2	System	The system authenticates the Admin and displays the main Dashboard.
3	Actor	The Admin clicks on the "Gestion" (Management) menu in the sidebar and selects the "Classes" sub-item.
4	System	The system displays the full list of existing classes in a DataGrid table with columns: Name, Type, Teacher, Capacity, Students Enrolled, and Actions.
5	Actor	The Admin reviews the current classes and clicks the "Ajouter une Classe" (Add New Class) button.
6	System	The system displays a class creation form/modal with fields: Class Name (text input), Class Type (dropdown with options fetched from '/admin/class-types', e.g., "Maternelle", "Préscolaire"), Assigned Teacher(s) (searchable dropdown populated from the teachers list), and Maximum Capacity (numeric input).
7	Actor	The Admin enters the class name, selects the class type, assigns a teacher from the dropdown, and sets the maximum capacity.
8	Actor	The Admin clicks the "Valider" (Validate) button to submit the form.
9	System	The system validates the input: verifies that the class name does not already exist for the current academic year, checks that the assigned teacher is not already leading another class in a conflicting time slot.
10	System	The system creates the class record in the 'school_classes' table, assigning a unique ID.
11	System	The system refreshes the classes DataGrid to include the new entry and displays a "Classe créée avec succès" success notification toast.

Scenario: Archive or unarchive an existing class
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Classes" section and reviews the class list.
2	System	The system displays all classes in the DataGrid.
3	Actor	The Admin clicks the action menu next to a specific class row and selects "Archiver" or "Désarchiver".
4	System	The system displays a confirmation dialog warning that archiving a class removes it from active lists.
5	Actor	The Admin clicks "Confirmer" (Confirm).
6	System	The system toggles the 'is$_{a}rchived$'$statusoftheclassrecordinthedatabase$.
7	System	The system refreshes the DataGrid, moving the class to the respective tab (Active or Archived).
8	System	The system displays a success toast confirming the status update.

Scenario: Modify class details
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Administrator logs into the Web Panel.
2	System	The system authenticates the Admin and navigates to the Dashboard.
3	Actor	The Admin navigates to the "Classes" section from the sidebar and reviews the class list.
4	System	The system displays all registered classes in a DataGrid table.
5	Actor	The Admin clicks the "Modifier" (Edit) icon next to a specific class.
6	System	The system opens a modal/form populated with the class's current data: name, type, assigned teacher(s), and capacity.
7	Actor	The Admin modifies the class name and assigns a different teacher from the dropdown.
8	Actor	The Admin adjusts the capacity value and clicks the "Mettre à Jour" (Update) button.
9	System	The system validates: checks if the current enrollment count exceeds the new capacity number, and ensures the newly assigned teacher is not already leading another class at the same time.
10	System	If validation passes: The system saves the modified attributes to the database.
11	System	The system closes the modal, refreshes the DataGrid to reflect the updated values, and displays a "Classe mise à jour avec succès" success message.

Use Case: 4.6 Configure Params

Scenario: Update system configuration parameter values
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel and navigates to the "Paramètres" (Settings/Configuration) page from the sidebar menu.
2	System	The system fetches all customizable global parameters.
3	System	The system displays a configuration form with all editable parameters organized in labeled sections, including: "Frais d'inscription" (Registration Fee), "Company Name", "Kindergarten Address", "Contact Phone", "Contact Email", "AI Enabled" toggle, and meal plan pricing values.
4	Actor	The Admin reviews the current parameter values displayed in the form.
5	Actor	The Admin modifies the "Frais d'inscription" value by typing a new amount.
6	Actor	The Admin updates the "Contact Phone" field with a new number and toggles the "AI Enabled" parameter.
7	System	The system performs client-side validation: verifies that numeric fields contain positive integers and that required text fields are not empty.
8	Actor	The Admin clicks the "Enregistrer les Paramètres" (Save Configurations) button.
9	System	The system saves all modified parameter key-value pairs.
10	System	The system updates the parameter records in the 'parameters' table and applies the new configuration globally for all future queries and calculations across the application.
11	System	The system displays a "Paramètres enregistrés avec succès" success toast and refreshes the form to show the saved values.

Scenario: Archive current academic year
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel and navigates to the "Paramètres" (Settings) page.
2	System	The system displays the configuration form, including the Planning section with an option to archive the current year.
3	Actor	The Admin clicks the "Archiver l'année scolaire" button.
4	System	The system displays a confirmation dialog warning that this action is irreversible and affects active inscriptions and classes.
5	Actor	The Admin clicks "Confirmer" to proceed.
6	System	The system validates that there are no pending inscriptions blocking the archive process.
7	System	The system archives the current planning year, and cascades the archive status to all active classes and inscriptions associated with it.
8	System	The system displays a success toast showing the number of classes and inscriptions archived.

Use Case: 4.7 Manage Users (Parent, Teacher)

Scenario: Create a new teacher account
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Enseignants" (Teachers) directory from the sidebar menu.
2	System	The system displays the list of all registered teachers in a DataGrid with columns: Name, CIN, Email, Phone, Status, and Actions.
3	Actor	The Admin clicks the "Ajouter un Enseignant" (Add Teacher) button.
4	System	The system displays a teacher creation form with fields: Full Name, CIN, Email, Phone Number, Address, and an initial temporary password.
5	Actor	The Admin fills in the teacher's personal information, sets an initial temporary password, and clicks "Enregistrer" (Save).
6	System	The system validates the input: checks for CIN and email uniqueness against the existing database records.
7	System	The system processes the teacher data.
8	System	The system creates the Teacher account in the 'accounts' table, hashes the temporary password, and sets the account status to "Active".
9	System	The system sends an email to the teacher's email address containing their login credentials (CIN and temporary password) with instructions to change the password on first login.
10	System	The system refreshes the teachers DataGrid to include the new entry and displays a "Enseignant créé avec succès" success toast.

Scenario: Archive or unarchive a teacher account
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Enseignants" directory and reviews the teacher list.
2	System	The system displays all teachers in the DataGrid.
3	Actor	The Admin clicks the action menu next to a specific teacher's row and selects "Archiver" (Archive) or "Désarchiver" (Unarchive).
4	System	The system checks the current status of the teacher account.
5	System	If archiving: The system displays a confirmation dialog warning that archiving will prevent the teacher from logging in.
6	Actor	The Admin clicks "Confirmer" (Confirm).
7	System	The system toggles the 'is$_a rchived$'$statusoftheteacherrecord.$
8	System	If archived, the system revokes their authentication token and prevents future logins to the mobile application.
9	System	The system refreshes the DataGrid, moving the teacher entry to the respective tab (Active or Archived).
10	System	The system displays a success toast confirming the new status.

Scenario: Modify parent or teacher information
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to either the "Parents" or "Enseignants" directory from the sidebar.
2	System	The system displays the corresponding user list in a DataGrid.
3	Actor	The Admin uses the search bar or DataGrid filter to locate a specific user and clicks the "Modifier" (Edit) icon next to their row.
4	System	The system retrieves the user's full profile data from the database and opens an editable form dialog pre-populated with their current information.
5	Actor	The Admin updates the outdated fields.
6	Actor	The Admin clicks the "Mettre à Jour" (Update) button.
7	System	The system validates the updated fields: checks for email/CIN uniqueness if changed, ensures required fields are not empty.
8	System	The system saves the updated data.
9	System	The system commits the changes to the database and closes the edit dialog.
10	System	The system refreshes the DataGrid to reflect the updated information and displays a "Informations mises à jour" success toast.

Scenario: Approve a pending parent account
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Parents" directory from the sidebar menu.
2	System	The system displays all parent accounts in a DataGrid, with "Pending" accounts highlighted with an orange status badge.
3	Actor	The Admin identifies a parent with "En Attente" (Pending) status and reviews their profile details by clicking "Voir" (View).
4	System	The system displays the parent's full profile: name, CIN, email, phone, address, and their children's inscription information.
5	Actor	The Admin verifies the information is complete and valid, then navigates back to the parents list.
6	Actor	The Admin clicks the approval action dropdown and selects "Approuver" (Approve) for the parent account.
7	System	The system updates the status to approved.
8	System	The system updates the parent's account status to "Approved", enabling them to fully access the mobile application.
9	System	The system sends a notification to the Parent informing them that their account has been approved and they can now log in.
10	System	The system refreshes the DataGrid, updating the parent's row to show a green "Approuvé" status badge.

Scenario: Reject a pending parent account
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Parents" directory from the sidebar.
2	System	The system displays the parent accounts list with pending entries highlighted.
3	Actor	The Admin identifies a parent with questionable or incomplete registration and clicks the approval action dropdown.
4	Actor	The Admin selects "Rejeter" (Reject) from the dropdown.
5	System	The system displays a confirmation dialog: "Êtes-vous sûr de vouloir rejeter ce compte parent ?"
6	Actor	The Admin confirms the rejection by clicking "Confirmer".
7	System	The system updates the status to rejected.
8	System	The system updates the parent's account status to "Rejected" in the database, preventing login access.
9	System	The system sends a notification to the Parent informing them of the rejection.
10	System	The system refreshes the DataGrid and updates the summary statistics counters.

Scenario: Archive or unarchive a parent account
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Parents" directory from the sidebar.
2	System	The system displays the parent accounts list.
3	Actor	The Admin clicks the action menu next to a specific parent and selects "Archiver" or "Désarchiver".
4	System	The system displays a confirmation dialog warning that archiving will prevent the parent from logging in.
5	Actor	The Admin confirms the action by clicking "Confirmer".
6	System	The system toggles the 'is$_{a}rchived'statusoftheparent'saccountinthedatabase.$
7	System	If archived, the system prevents future login access for the parent.
8	System	The system refreshes the DataGrid, moving the parent to the respective tab, and updates the summary statistics counters.

Use Case: 4.8 Manage Meals

Scenario: Add a new meal to the catalog
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel.
2	Actor	The Admin navigates to the "Repas" (Meal Settings) section from the sidebar menu.
3	System	The system displays the full list of existing meals in the meal catalog DataGrid.
4	Actor	The Admin reviews the current meal catalog and clicks the "Ajouter un Repas" (Add New Meal) button.
5	System	The system displays a meal creation form/modal with a text input field for the meal name and optionally a description field.
6	Actor	The Admin enters the name of the new meal in the text input field.
7	System	The system performs client-side validation: ensures the meal name is not empty and does not exceed the character limit.
8	Actor	The Admin clicks the "Enregistrer" (Save) button.
9	System	The system saves the meal name.
10	System	The system verifies server-side that the meal name does not already exist in the database (case-insensitive uniqueness check).
11	System	If unique: The system creates the new meal record in the 'meals' table and assigns it a unique ID.
12	System	The system automatically refreshes the meal catalog list, showing the new entry, and displays a "Repas ajouté avec succès" success toast.

Scenario: Remove a meal from the catalog
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin logs into the Web Panel and navigates to the "Repas" section.
2	System	The system displays the full meal catalog in a DataGrid table.
3	Actor	The Admin identifies a meal that is no longer offered and clicks the "Supprimer" (Delete/Remove) icon button in the Actions column.
4	System	The system verifies whether the meal is currently assigned to any active "Weekly Menu" planning by checking the 'menu_plannings' table.
5	System	If the meal IS currently used in a weekly menu: The system blocks the deletion and displays an error message: "Suppression impossible: ce repas est actuellement utilisé dans un menu hebdomadaire. Veuillez d'abord le retirer du planning."
6	System	If the meal is NOT used in any menu: The system displays a confirmation pop-up: "Êtes-vous sûr de vouloir supprimer le repas '[MealName]' ? Cette action est irréversible."
7	Actor	The Admin reads the confirmation and clicks "Confirmer" to proceed.
8	System	The system deletes the meal record.
9	System	The system removes the meal record from the database, along with any associated food exception entries linked to this meal.
10	System	The system refreshes the meal catalog DataGrid, removing the deleted entry, and displays a "Repas supprimé" success toast.

Use Case: 4.9 Orchestrate Meal Schedules

Scenario: Compose a weekly meal plan
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Menu Hebdomadaire" (Weekly Menu) section from the sidebar.
2	System	The system displays a calendar grid interface for the current week (Monday through Friday), with each day showing an empty meal slot.
3	Actor	The Admin selects the target week using the week navigation arrows or date picker.
4	System	The system loads any existing meal assignments for the selected week and populates the grid accordingly.
5	Actor	The Admin clicks on a specific day slot to assign a meal.
6	System	The system opens a dropdown menu listing all available meals from the meal catalog, fetched from the database.
7	Actor	The Admin selects a meal from the dropdown.
8	System	The system temporarily locks the selected meal into the Monday slot on the UI grid, displaying the meal name with a remove icon.
9	Actor	The Admin repeats the meal selection process for Tuesday through Friday, completing the entire week's meal plan.
10	Actor	The Admin reviews the complete weekly plan and clicks the "Publier le Menu" (Publish Plan) button.
11	System	The system saves the meal-to-date assignments for each day of the week.
12	System	The system inserts the meal schedule records into the database, linking each meal to the specific date.
13	System	The system broadcasts the updated weekly menu to Parents via push notifications in the mobile application and displays a "Menu publié avec succès" success toast.

Scenario: Save a meal plan as a reusable template
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin composes a complete weekly meal plan in the "Menu Hebdomadaire" calendar grid.
2	Actor	The Admin clicks the "Sauvegarder comme Modèle" (Save as Template) button.
3	System	The system displays a dialog prompt requesting a Template Name with a text input field.
4	Actor	The Admin enters a descriptive template name and clicks "Enregistrer" (Save).
5	System	The system validates that the template name is not empty and does not duplicate an existing template name.
6	System	The system serializes the 5-day meal configuration and saves it in the 'menu_plannings' table with the given template name.
7	System	The system displays a "Modèle enregistré avec succès" success toast.
8	Actor	The Admin verifies the template appears in the template list by navigating to the template management section.
9	System	The system displays the saved template in the templates list with its name, creation date, and a preview of the assigned meals.
10	System	The system enables "Charger" (Load), "Modifier" (Edit), and "Supprimer" (Delete) action buttons next to the template entry.

Scenario: Load a previously saved meal plan template
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to a blank upcoming week in the "Menu Hebdomadaire" calendar.
2	System	The system detects empty meal slots for the selected week and displays a "Charger un Modèle" (Load Template) button prominently in the interface.
3	Actor	The Admin clicks the "Charger un Modèle" button.
4	System	The system displays a list of all saved templates with their names and meal summaries.
5	Actor	The Admin selects "Semaine Santé 1" from the template list.
6	System	The system retrieves the template data and auto-populates all 5 daily slots on the calendar UI with the stored meal assignments.
7	Actor	The Admin visually verifies the loaded meals for each day, making any desired adjustments.
8	Actor	The Admin clicks the "Publier" (Publish) button to commit the loaded schedule.
9	System	The system applies the loaded (and possibly adjusted) meal plan to the selected week's dates in the database.
10	System	The system saves the schedule, notifies parents via the mobile app, and displays a "Menu publié avec succès" success toast.

Scenario: Delete a saved meal plan template
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the meal template management section.
2	System	The system displays the list of all saved meal plan templates.
3	Actor	The Admin identifies an outdated or unused template and clicks the "Supprimer" (Delete) icon next to it.
4	System	The system displays a confirmation dialog: "Êtes-vous sûr de vouloir supprimer le modèle '[TemplateName]' ?"
5	Actor	The Admin clicks "Confirmer" to proceed with the deletion.
6	System	The system deletes the meal plan template.
7	System	The system removes the template record from the database.
8	System	The system refreshes the template list and displays a "Modèle supprimé" success toast.
9	Actor	The Admin verifies the template is no longer listed.
10	System	The system confirms the template list is updated, with no impact on any previously published menus that were derived from this template.

Use Case: 4.10 Manage Planned Activities

Scenario: Assign or unassign activities for each day across the academic year
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Planning" section from the sidebar and selects a specific class from a class dropdown filter.
2	System	The system fetches the planning data for the selected class and displays a monthly calendar layout, with each day showing its assigned activities (if any).
3	Actor	The Admin uses the month navigation arrows to navigate to the desired month in the academic year.
4	System	The system loads and displays the planning data for the selected month, showing activity cards in each day's slot.
5	Actor	The Admin clicks on a specific date cell in the calendar to add an activity.
6	System	The system displays a side-panel or dropdown listing all available activities from the activity catalog.
7	Actor	The Admin selects an activity from the side-panel and drags it or selects it to assign it to the clicked date slot.
8	System	The system registers the activity-to-date mapping and displays the activity card in the calendar cell with the activity name and type icon.
9	Actor	The Admin decides to remove a previously assigned activity and clicks the "" (remove) icon on an activity card on a different date.
10	System	The system unassigns the activity from that date, removing the card from the calendar cell and freeing up the time block.
11	Actor	The Admin clicks "Sauvegarder le Planning" (Save Planning) after making all desired changes.
12	System	The system saves all planning changes to the database and silently synchronizes the updates with the Teachers' planned activities views in the mobile app.
13	System	The system displays a "Planning mis à jour" success toast.

Use Case: 4.11 Manage Events

Scenario: Create a new event
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Événements" (Events) section from the sidebar.
2	System	The system displays the events management interface with a list of past and upcoming events in a DataGrid or timeline view.
3	Actor	The Admin clicks the "Créer un Événement" (Create Event) button.
4	System	The system displays an event creation form with fields: Event Title, Date (date picker), Type (dropdown with options such as "Visite Médicale", "Sortie Scolaire", "Réunion Parents", "Fête"), and Assigned Class(es) (multi-select dropdown).
5	Actor	The Admin enters the event title, selects the date, chooses the event type, and assigns the relevant class(es).
6	Actor	The Admin optionally adds a description or notes about the event.
7	Actor	The Admin clicks "Soumettre" (Submit) to create the event.
8	System	The system validates the input: ensures the date is in the future, the title is not empty, and at least one class is assigned.
9	System	The system saves the event record in the 'events' table, linking it to the selected class(es) via the 'event_notification_target' table.
10	System	The system sends notifications to the teachers of the assigned classes, alerting them of the upcoming event.
11	System	The system refreshes the events list and displays a "Événement créé avec succès" success toast.

Scenario: Delete an existing event
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin navigates to the "Événements" section.
2	System	The system displays the list of all events (pending and executed).
3	Actor	The Admin identifies an event that needs to be cancelled and clicks the "Supprimer" (Delete) icon next to it.
4	System	The system checks whether the event has already been marked as "Executed".
5	System	If the event is already executed: The system displays a warning: "Cet événement a déjà été exécuté et ne peut pas être supprimé, uniquement archivé."
6	System	If the event is still "Pending": The system displays a confirmation dialog: "Êtes-vous sûr de vouloir supprimer l'événement '[EventTitle]' ?"
7	Actor	The Admin clicks "Confirmer" to proceed.
8	System	The system deletes the event record and its notification targets from the database.
9	System	The system sends cancellation notifications to the teachers of the affected classes.
10	System	The system refreshes the events list, removing the deleted entry, and displays a "Événement supprimé" success toast.

Scenario: Mark an event as executed
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Événements" section.
2	System	The system displays past and upcoming scheduled events, with past events that are still "Pending" highlighted.
3	Actor	The Admin identifies an event whose date has passed and is still marked as "En Attente" (Pending).
4	Actor	The Admin clicks the "Marquer comme Exécuté" (Mark as Done) button next to the event.
5	System	The system displays a brief confirmation: "Confirmer que l'événement '[EventTitle]' a été exécuté ?"
6	Actor	The Admin clicks "Confirmer".
7	System	The system updates the event status from "Pending" to "Executed" in the database.
8	System	The system records the execution date and the Admin who confirmed the completion.
9	System	The system refreshes the events list, moving the event to the "Historique" (History) section with an "Exécuté" status badge in green.
10	System	The system displays a "Événement marqué comme exécuté" success toast.

Use Case: 4.12 Manage Food Exceptions

Scenario: Consult the list of children with dietary exceptions
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Exceptions Alimentaires" (Dietary Exceptions/Food Exceptions) module from the sidebar menu.
2	System	The system queries all medical profiles for registered allergies, food intolerances, or restricted diets.
3	System	The system generates and displays a comprehensive report in a DataGrid table listing: each child's name, their assigned class, the AI-generated dietary comment summarizing restrictions, and the specific food items flagged as exceptions.
4	Actor	The Admin reviews the list, scanning for children with critical dietary needs.
5	Actor	The Admin uses the DataGrid search/filter tools to filter by class or by specific dietary keyword.
6	System	The system dynamically filters and displays only the matching records.
7	Actor	The Admin clicks on a specific child's row to expand or view the detailed dietary breakdown.
8	System	The system displays the full dietary exception details: the original AI analysis comment, the list of flagged meals with reasons, and an option to override or edit the exceptions.
9	Actor	The Admin clicks the "Exporter" (Export) or "Imprimer" (Print) button to generate a printable report for the kitchen staff.
10	System	The system generates a formatted PDF or printable view containing each child's name, class, photo, and specific dietary needs, organized by class for easy distribution.

Use Case: 4.13 Generate Behavioral Reports

Scenario: Analyze behavioral reports (SIGNALEMENTS COMPORTEMENTAUX)
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Rapports" (Reports) section from the sidebar and selects the "Signalements Comportementaux" (Behavioral Analytics) tab.
2	System	The system displays the behavioral reports analytics interface with date range filters and class filters.
3	Actor	The Admin selects a date range using the start and end date pickers.
4	System	The system sends a query to compile all submitted behavioral signalements within the specified date range.
5	System	The system generates and renders statistical charts: a bar chart showing the frequency of signals per class, a pie chart showing the distribution by severity level ("Info", "Avertissement", "Urgent"), and a timeline trend chart.
6	Actor	The Admin hovers over the bar chart segments to view detailed metrics.
7	System	The system displays tooltip overlays with precise numerical counts and percentages.
8	Actor	The Admin applies a class filter by selecting a specific class from the dropdown to drill down into its signals.
9	System	The system dynamically updates all charts to show only the data for the selected class.
10	Actor	The Admin scrolls down to view the detailed signalement list below the charts.
11	System	The system displays a detailed DataGrid table listing each individual signalement with columns: Child Name, Date, Severity, Description, and Teacher who submitted it.

Scenario: View behavioral reports history (SIGNALEMENTS COMPORTEMENTAUX)
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Rapports" section and selects the "Historique des Signalements" (Behavioral History) tab.
2	System	The system retrieves all behavioral signalements from the database, ordered by date (most recent first).
3	System	The system displays a DataGrid table with columns: Child Name, Class, Date, Severity Level (with color-coded badges), Description excerpt, and Teacher.
4	Actor	The Admin uses the column filters to sort by severity level.
5	System	The system filters the DataGrid to display only the matching entries.
6	Actor	The Admin clicks on a specific signalement row to view the full details.
7	System	The system opens a detail dialog showing: the complete observation text, the child's name and class, the date and time, the severity level, and the submitting teacher's name.
8	Actor	The Admin reviews the report details and closes the dialog.
9	Actor	The Admin uses the date range picker to filter signalements for a specific academic period.
10	System	The system refetches and displays signalements within the selected date range, updating the DataGrid.

Use Case: 4.14 Manage Criterias

Scenario: Add a competence criterion to an activity
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Activités" section and locates the target activity.
2	Actor	The Admin clicks the "Voir Détails" (View Details) icon to open the activity's details view.
3	System	The system displays the activity detail panel showing: the title, description, type, and a list of currently associated competence criteria (if any).
4	Actor	The Admin reviews the existing competences and clicks the "Ajouter une Compétence" (Add Competence) button.
5	System	The system displays a text input field below the existing competence list, with a placeholder text: "Saisir la description de la compétence..."
6	Actor	The Admin types the competence description and clicks the "Ajouter" (Add) button.
7	System	The system validates that the competence text is not empty and not a duplicate of an existing competence for this activity.
8	System	The system creates a new 'Criteria' record in the database, linked to the activity via the activity ID.
9	System	The system dynamically updates the competence list on the screen to display the newly added criterion with a delete icon next to it.
10	System	The system displays a brief "Compétence ajoutée" confirmation message inline.

Scenario: Remove a competence criterion from an activity
Path Type: Alternate Flow

Step	Assignee	Action
1	Actor	The Admin opens an existing activity's detail view by clicking "Voir Détails".
2	System	The system displays the activity's full details including the list of associated competence criteria.
3	Actor	The Admin identifies an obsolete or incorrect competence and clicks the "Supprimer" (Trash) icon button next to it.
4	System	The system checks whether any existing evaluation records in the database reference this competence criterion.
5	System	If evaluations reference this competence: The system blocks the deletion and displays an error message: "Suppression impossible: cette compétence a déjà été évaluée pour certains enfants."
6	System	If no evaluations reference this competence: The system displays a simple confirmation: "Supprimer cette compétence ?"
7	Actor	The Admin clicks "Confirmer" to proceed with the deletion.
8	System	The system removes the competence criterion from the database.
9	System	The system refreshes the competence list dynamically, removing the deleted entry from the UI.
10	System	The system displays a "Compétence supprimée" confirmation message.

Use Case: 4.15 Generate Financial Reports

Scenario: Analyze financial reports (RAPPORTS FINANCIERS)
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Rapports" section and selects the "Rapports Financiers" (Financial Overview) tab.
2	System	The system displays the financial analytics interface with an academic year selector dropdown.
3	Actor	The Admin selects the current academic year from the year filter.
4	System	The system aggregates collected payments per month versus expected payments.
5	System	The system generates and renders a bar chart displaying: total expected revenue per month, total collected revenue per month, and the gap between the two, using color-coded bars (green for collected, grey for expected).
6	Actor	The Admin hovers over a specific month's bar to view the exact collected vs. expected amounts in a tooltip.
7	System	The system displays the tooltip with precise TND values.
8	Actor	The Admin applies a filter to view only specific months by clicking on the month axis labels.
9	System	The system dynamically updates the chart to zoom into the selected month's data, showing a detailed breakdown by payment status (Paid, Pending).
10	System	The system displays summary KPI cards above the chart: Total Revenue Collected, Total Outstanding, Collection Rate percentage, and Number of Fully Paid Students.
11	Actor	The Admin reviews the overall financial health indicators and identifies months with low collection rates for follow-up action.

Scenario: View financial reports history (RAPPORTS FINANCIERS)
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Admin navigates to the "Rapports" section and selects the "Historique Financier" (Financial History) tab.
2	System	The system displays a historical view of all financial transactions across academic years.
3	Actor	The Admin selects a specific academic year from the year filter dropdown.
4	System	The system displays a detailed monthly breakdown table.
5	System	The system shows a DataGrid with columns: Month, Expected Amount, Collected Amount, Outstanding Amount, Number of Paid Students, Number of Pending Students, and Collection Rate %.
6	Actor	The Admin clicks on a specific month row to drill down into individual student payment details.
7	System	The system displays detailed lists of: parents who paid fully, parents who paid partially, and parents with no payment for that month.
8	Actor	The Admin reviews the detailed per-parent breakdown for the selected month.
9	Actor	The Admin navigates back to the main financial history view to compare months side by side.
10	System	The system restores the full monthly breakdown table, maintaining the selected academic year filter.

2.5.5 5. All Actors Use Cases

Use Case: 5.1 Authenticate

Scenario: Parent login via mobile application
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Parent opens the mobile application on their smartphone.
2	System	The system checks for any existing session token in local storage. Finding none (or an expired token), it displays the Login Screen with CIN and Password input fields and a "Se Connecter" (Log In) button.
3	Actor	The Parent enters their CIN (National ID number) in the CIN field and their password in the Password field.
4	Actor	The Parent taps the "Se Connecter" button.
5	System	The system validates the provided CIN and password credentials.
6	System	The system verifies the credentials against the database: checks that the CIN exists, the password hash matches, the account role is "Parent", and the account approval status is "Approved".
7	System	If credentials are invalid: The system displays an error message below the login form: "CIN ou mot de passe incorrect."
8	System	If the account is not yet approved: The system navigates to a "Waiting for Approval" screen informing the Parent that their account is under review.
9	System	If authentication succeeds: The system issues a Sanctum Bearer token, stores it in secure local storage, and retrieves the parent's profile data.
10	System	The system redirects the Parent to the Parent Dashboard, displaying their children's profiles and the available navigation menu.

Scenario: Teacher login via mobile application
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Teacher opens the mobile application on their device.
2	System	The system checks for any existing session token. Finding none, it displays the Login Screen with CIN and Password fields.
3	Actor	The Teacher enters their CIN and password and taps the "Se Connecter" button.
4	System	The system processes the login with the provided credentials.
5	System	The system validates the Teacher's credentials: verifies the CIN exists, the password hash matches, the account role is "Teacher", and the account status is "Active" (not archived or deactivated).
6	System	If credentials are invalid or the account is inactive: The system displays an appropriate error message.
7	System	If authentication succeeds: The system issues a Bearer token and retrieves the teacher's profile data including their assigned class information.
8	System	The system fetches the teacher's assigned class(es) via the class-teacher relationship in the database.
9	System	The system routes the Teacher to the Teacher Dashboard with access restricted to their assigned class, displaying today's activities, student count, and pending attendance.
10	System	The system loads the notification bell with any unread notifications count for the teacher.

Scenario: Admin login via web panel
Path Type: Basic Path

Step	Assignee	Action
1	Actor	The Administrator opens a web browser and navigates to the Web Panel URL.
2	System	The system serves the React Admin SPA and displays the Admin Portal login page with Email and Password input fields and a "Se Connecter" button.
3	Actor	The Admin enters their secure email and password credentials in the respective fields.
4	Actor	The Admin clicks the "Se Connecter" button to submit the login form.
5	System	The system validates the email and password, protected by rate limiting (max 10 attempts per minute).
6	System	The system validates the credentials against the database: checks that the email exists, the password hash matches, and the account has administrative privileges (role = "admin").
7	System	If credentials are invalid: The system displays an error alert: "Email ou mot de passe incorrect."
8	System	If authentication succeeds: The system issues a Bearer token, stores it in the browser's local storage via the AuthContext, and fetches the admin's profile data.
9	System	The system grants access to the Admin Dashboard and generates the main dashboard statistics.
10	System	The system renders the full Admin Dashboard with the sidebar navigation (Inscriptions, Paiements, Classes, Activités, Planning, Événements, Repas, Paramètres, Rapports), KPI cards, and interactive charts.

2.6 Conclusion

In this chapter, we conducted a thorough requirements gathering process for the Petit Suivi system. We identified the three principal actors, Parent, Teacher, and Administrator, and mapped their interactions through a comprehensive use case diagram encompassing more than twenty use cases.

Each use case was then described in detail using a structured textual format that specifies the involved actors, basic flows, and alternate flows. This documentation serves as the functional specification baseline that will guide the system design and implementation phases covered in the subsequent chapters.

Chapter 3

Analysis and Design

3.1 Introduction

At this stage of our study, we cross the boundary that separates the functional world from the object world, formally beginning an analysis that allows us to identify the system's objects.

Analysis is a very important step during the implementation of a computerized system. It mainly consists of elaborating a domain class model that allows a smooth transition to true object modeling and illustrates the dynamics of the system by formalizing the scenarios of its use cases.

3.2 Realization of Use Cases

Sequence diagrams present the cooperation between the different objects from a temporal point of view. These diagrams are distinctly suited to model the dynamic aspects of our complex use case scenarios. Below, we formalize the sequence diagrams for the previously identified use case scenarios across the connected platforms.

3.2.1 Parent: Track Children - Scheduled Activities

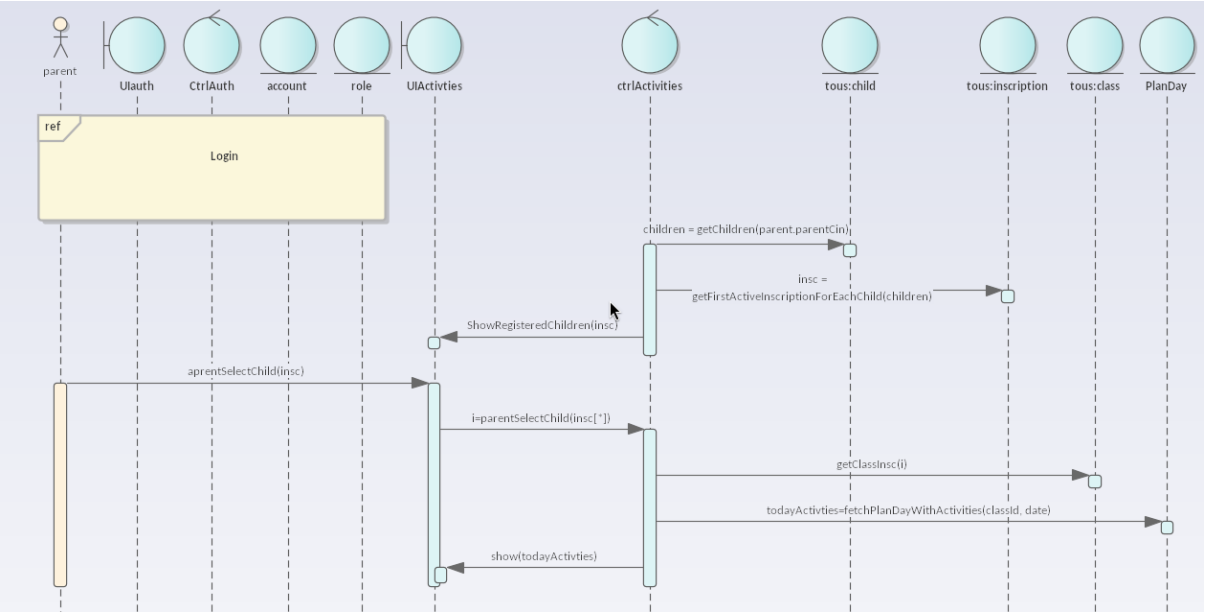


Figure 3.1: Sequence Diagram: Consult today's scheduled activities

3.2.2 Parent: Track Children - Attendance Status

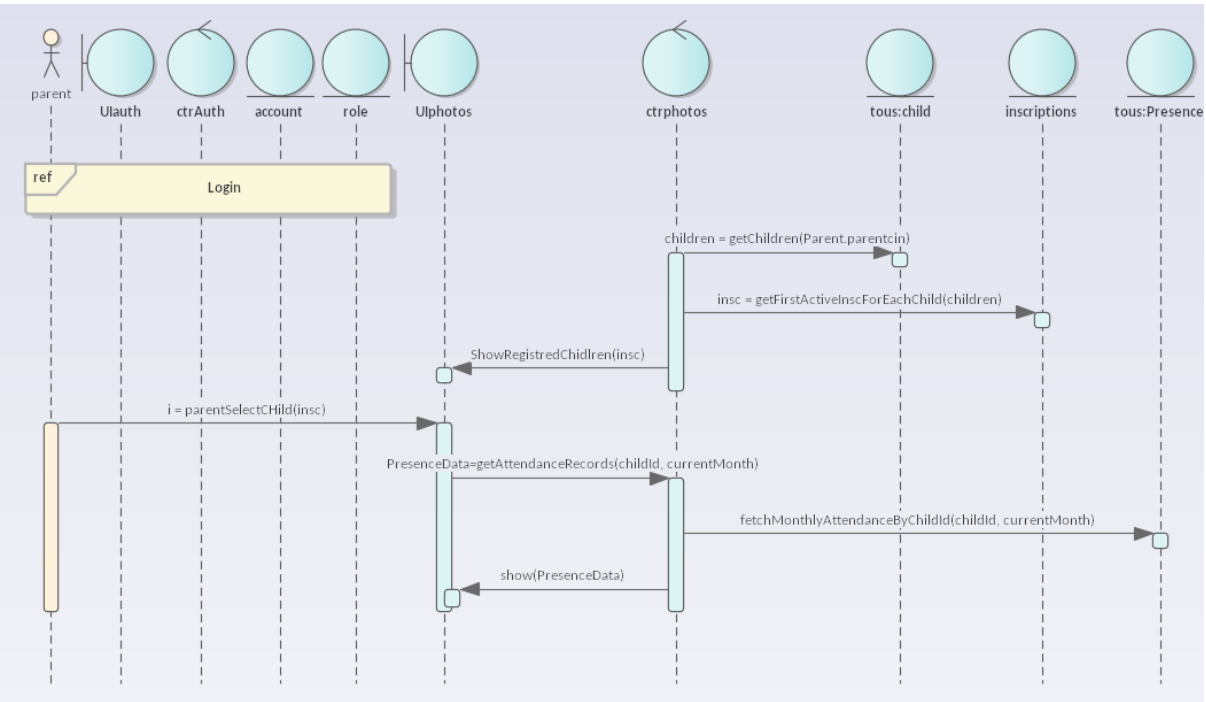


Figure 3.2: Sequence Diagram: Check daily attendance status

3.2.3 Parent: Track Children - Acquired Competences

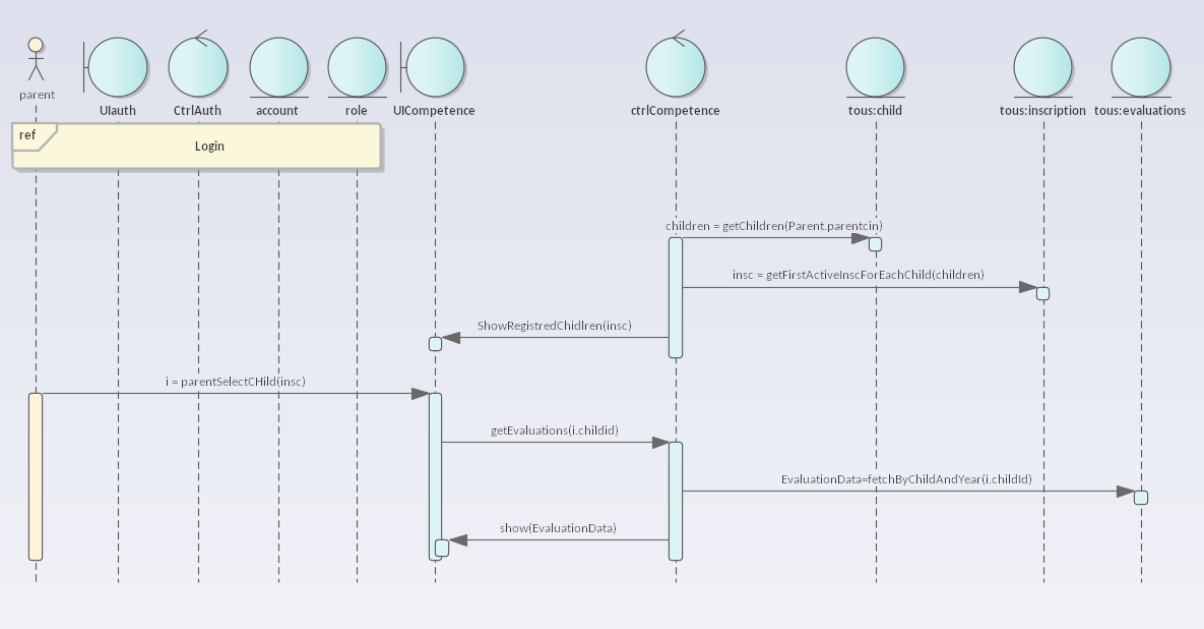


Figure 3.3: Sequence Diagram: Review acquired competences

3.2.4 Parent: View Photos

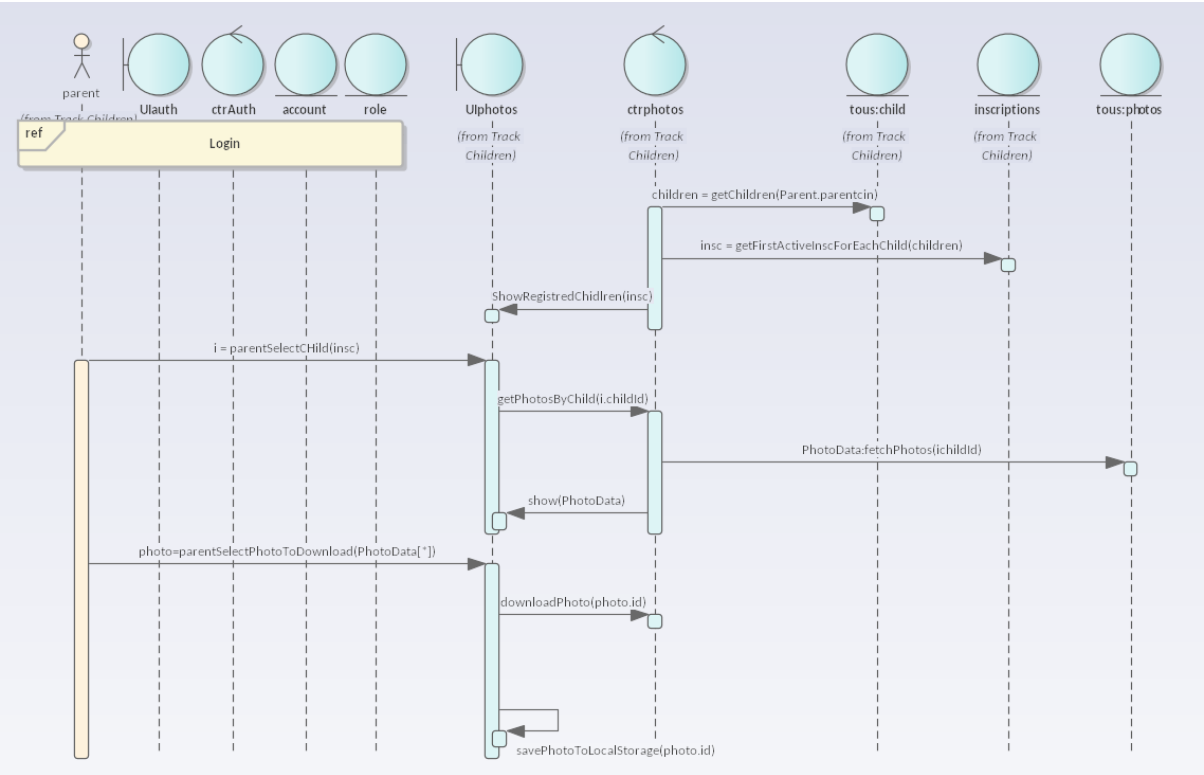


Figure 3.4: Sequence Diagram: Browse photo gallery

3.2.5 Parent: Submit Registration Form

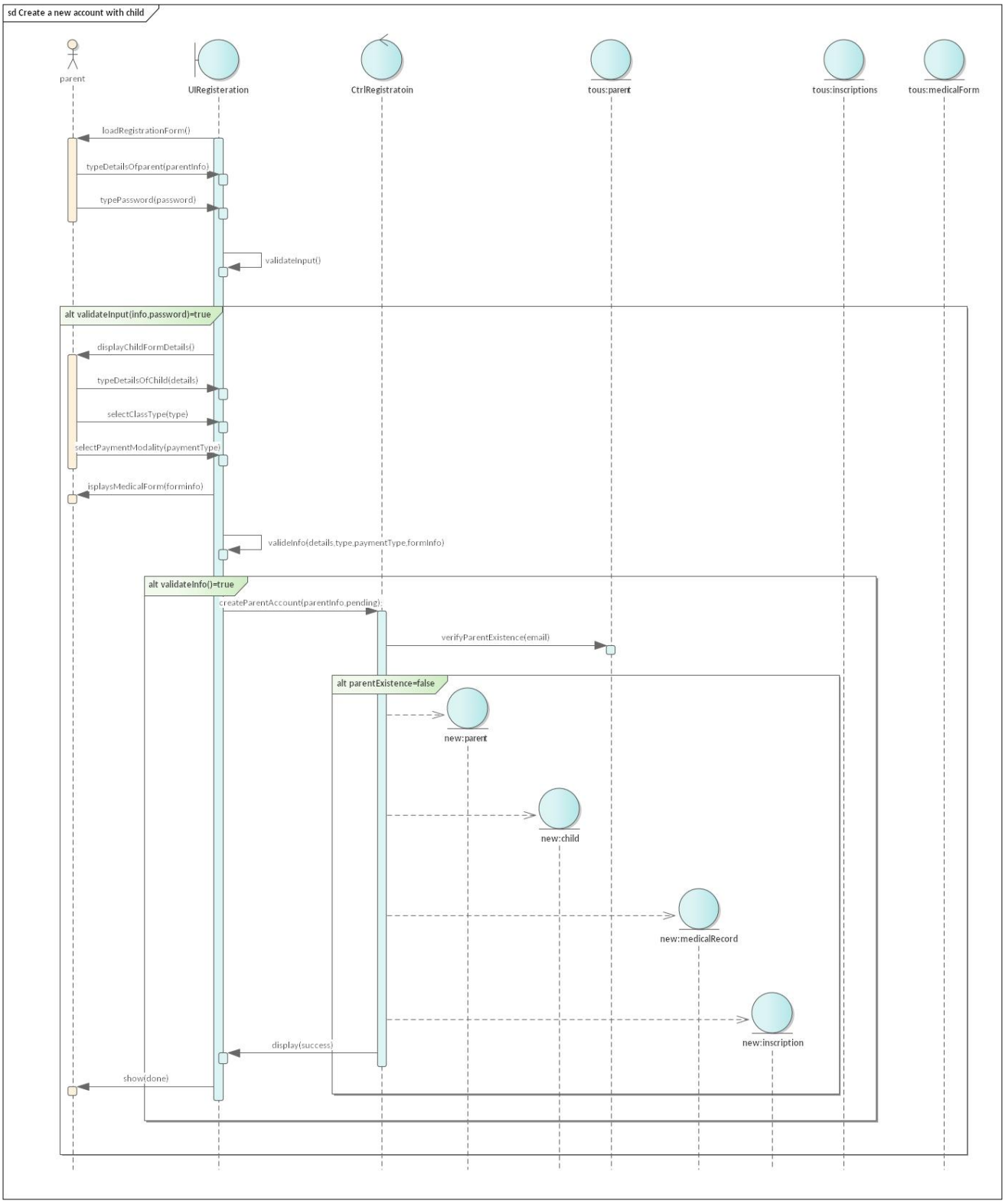


Figure 3.5: Sequence Diagram: Submit registration form

3.3 Domain Model Construction (Class Diagram)

The class diagram visualizes the static structure of the Petit Suivi system, displaying the primary object classes, their attributes, and their relational associations. This model forms the concrete blueprint upon which our database migrations and entity relationships are ultimately built.

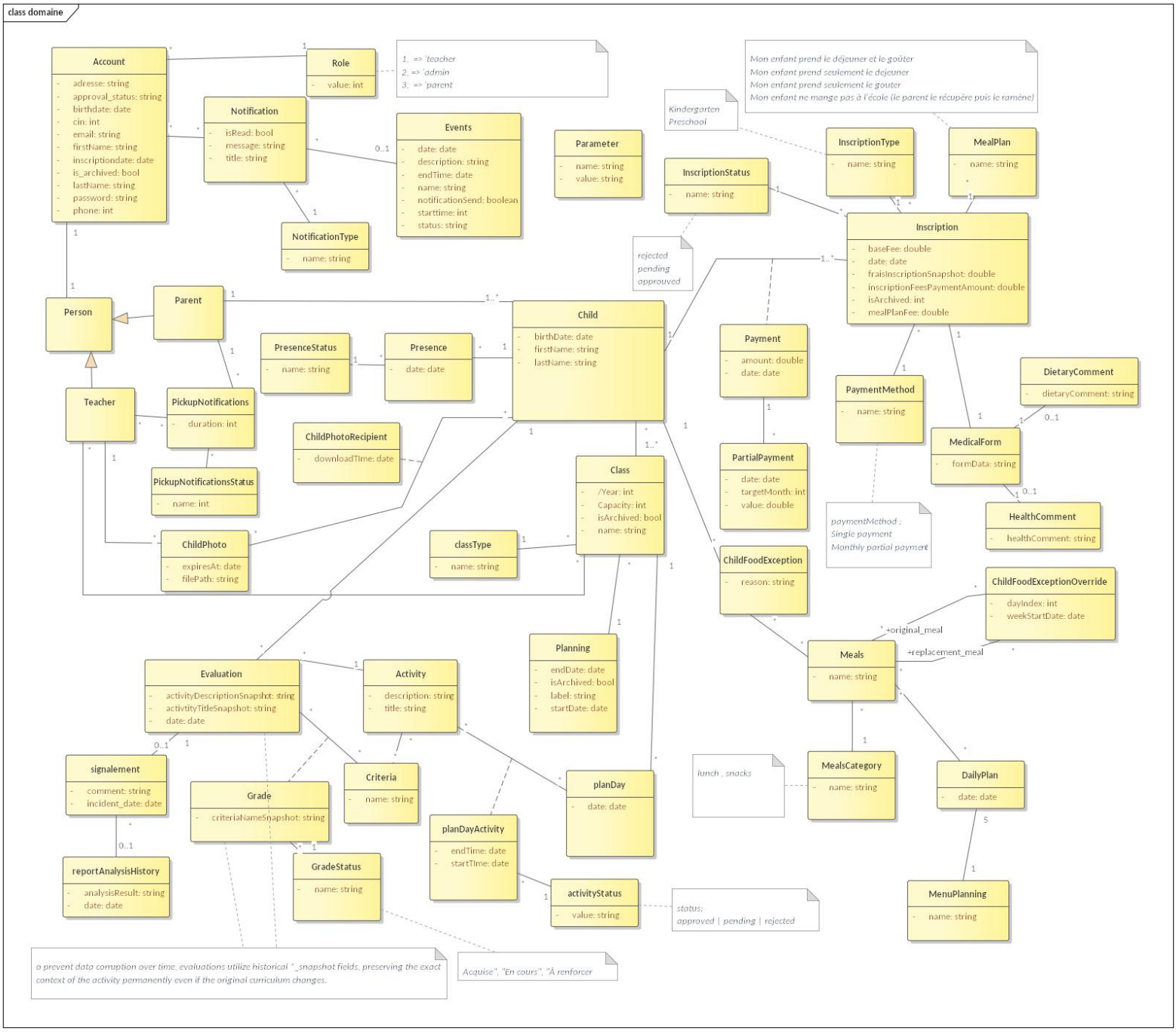


Figure 3.6: Global Class Diagram for Petit Suivi

3.3.1 Detailed Class Definitions

User & Access Management – Class Diagram Fragment

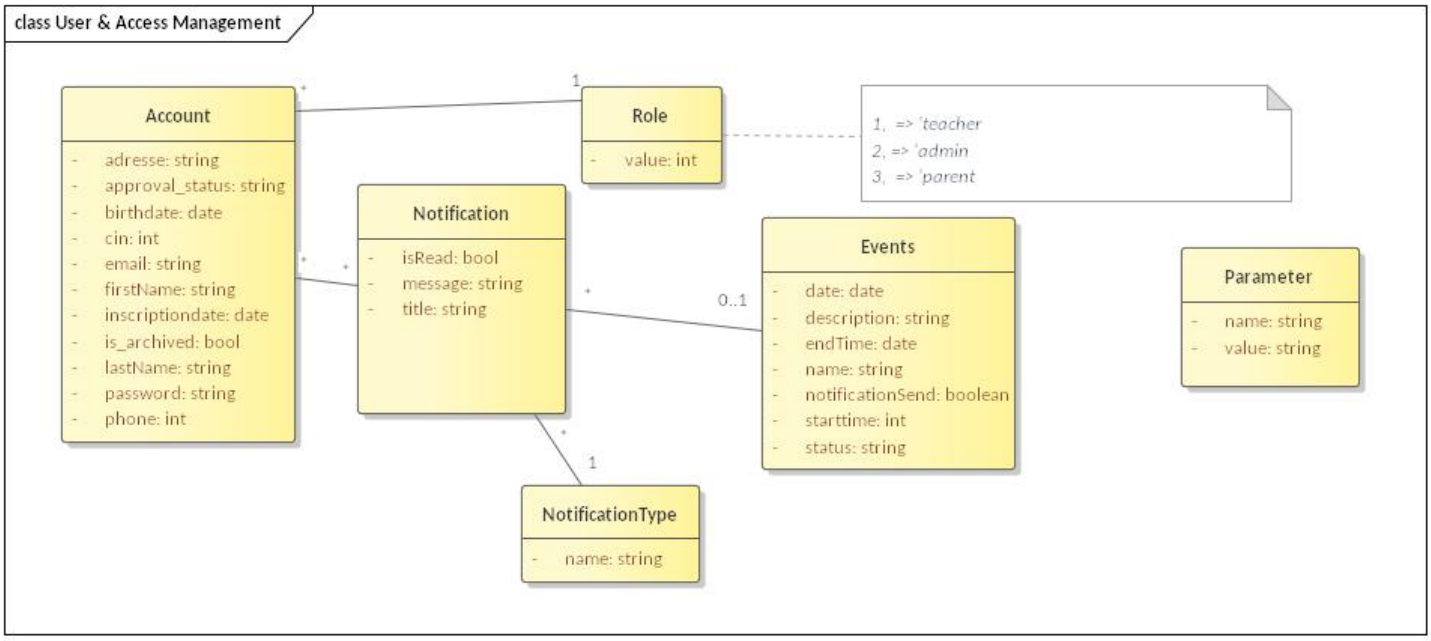


Figure 3.7: Class Diagram: User & Access Management

Class: Role

#	Attribute Name	Description
1	**value**	Numeric value used to identify the role assigned to an account (1: teacher, 2: admin, 3: parent).

Class: Account

#	Attribute Name	Description
1	**adresse**	Stores the account holder's postal address.
2	**approval_status**	Indicates whether the account has been approved.
3	**birthdate**	Stores the account holder's date of birth.
4	**cin**	Stores the national identity number linked to the account.
5	**email**	Stores the account email address.
6	**firstName**	Stores the account holder's first name.
7	**inscriptiondate**	Records the date when the account was created or registered.
8	**is_archived**	Indicates whether the account is archived.
9	**lastName**	Stores the account holder's last name.
10	**password**	Stores the encrypted password associated with the account.
11	**phone**	Stores the phone number associated with the account.

Class: Notification

#	Attribute Name	Description
1	**isRead**	Indicates whether the notification has been read.
2	**message**	Stores the main notification message shown to the user.
3	**title**	Stores the notification title.

Class: Notificationtype

#	Attribute Name	Description
1	**name**	Stores the notification type name (e.g., alert, message).

Class: Events

#	Attribute Name	Description
1	**date**	Stores the date of the event.
2	**description**	Stores the detailed description of the event.
3	**endTime**	Stores the event ending time.
4	**name**	Stores the event title or name.
5	**notificationSend**	Indicates whether a notification should be sent for the event.
6	**starttime**	Stores the event starting time.
7	**status**	Stores the current status of the event.

Class: Parameter

#	Attribute Name	Description
1	**name**	Stores the parameter name.
2	**value**	Stores the value associated with the parameter.

Registration & Finance – Class Diagram Fragment

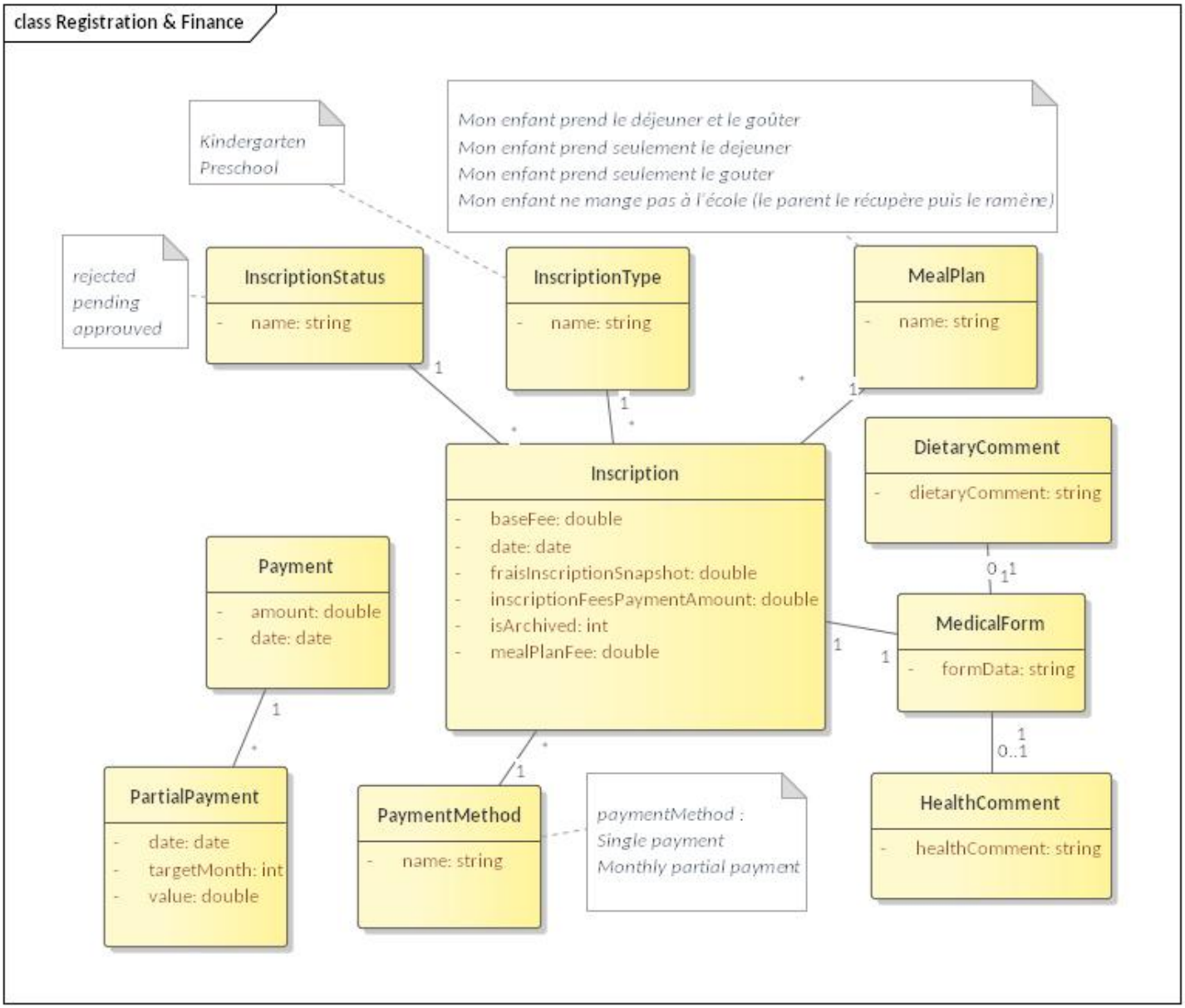


Figure 3.8: Class Diagram: Registration & Finance

Class: Dietarycomment

#	Attribute Name	Description
1	**dietaryComment**	Stores the dietary comment .

Class: Medicalform

#	Attribute Name	Description
1	**formData**	Stores the medical form data associated with the inscription.

Class: Healthcomment

#	Attribute Name	Description
1	**healthComment**	Stores the health-related .

Class: Partialpayment

#	Attribute Name	Description
1	**date**	Stores the date of the partial payment.
2	**targetMonth**	Identifies the month covered by the partial payment.
3	**value**	Stores the amount paid in the partial payment.

Class: Payment

#	Attribute Name	Description
1	**amount**	Stores the total payment amount.
2	**date**	Stores the payment date.

Class: Inscription

#	Attribute Name	Description
1	**baseFee**	Stores the base fee of the inscription.
2	**date**	Stores the inscription date.
3	**fraisInscriptionSnapshot**	Stores the registration fee snapshot captured at inscription time.
4	**inscriptionFeesPaymentAmount**	Stores the amount paid for inscription fees.
5	**isArchived**	Indicates whether the inscription is archived.
6	**mealPlanFee**	Stores the fee associated with the selected meal plan.

Class: Paymentmethod

#	Attribute Name	Description
1	**name**	Stores the name of the payment method (e.g., Single payment, Monthly partial payment).

Class: Inscriptiontype

#	Attribute Name	Description
1	**name**	Stores the inscription type name (e.g., Kindergarten, Preschool).

Class: Mealplan

#	Attribute Name	Description
1	**name**	Stores the meal plan name (e.g., Déjeuner et goûter, Seulement déjeuner, etc.).

Class: Inscriptionstatus

#	Attribute Name	Description
1	**name**	Stores the inscription status name (e.g., rejected, pending, approved).

People & Logistics – Class Diagram Fragment

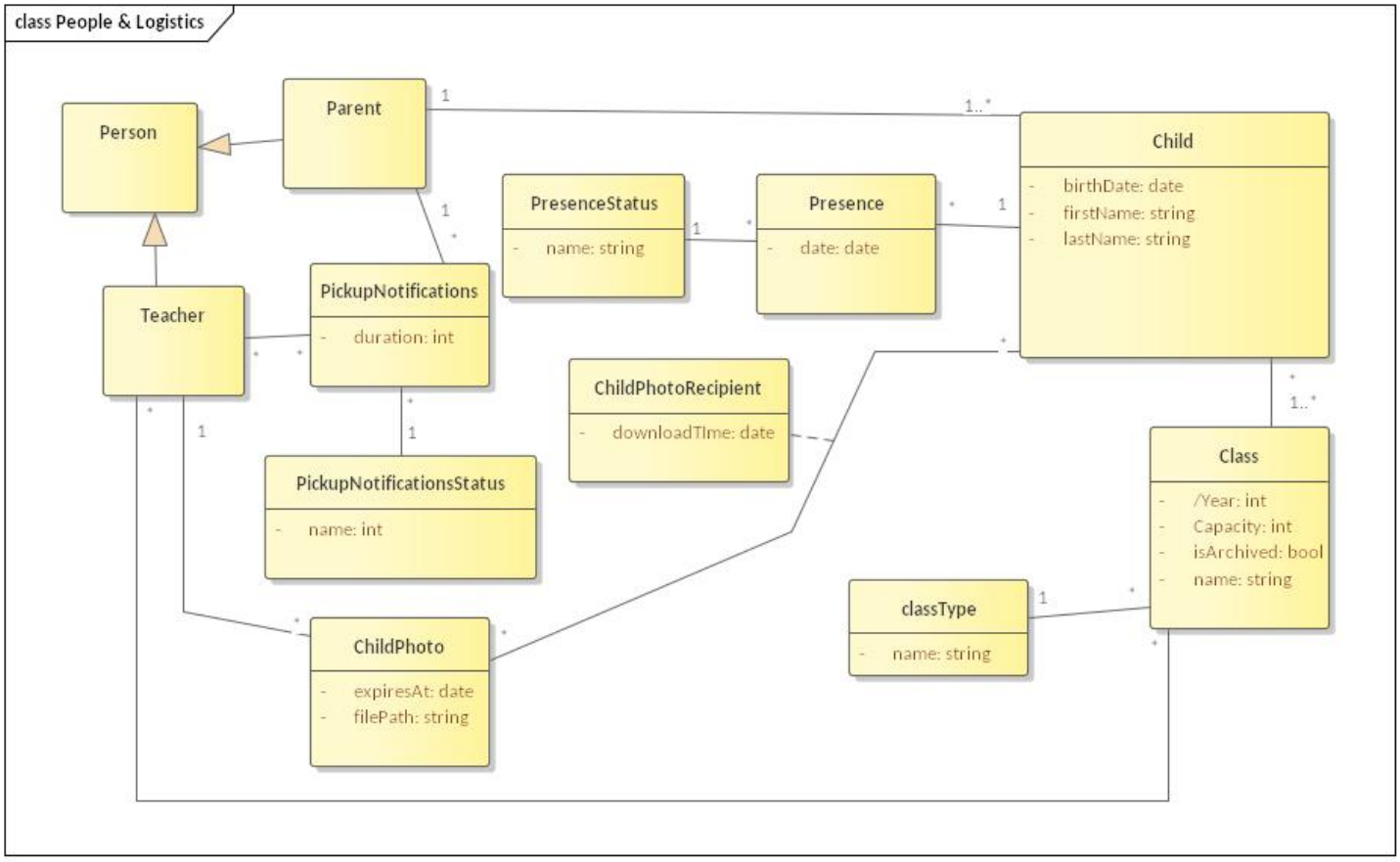


Figure 3.9: Class Diagram: People & Logistics

Class: Presencestatus

#	Attribute Name	Description
1	**name**	Stores the presence status name.

Class: Presence

#	Attribute Name	Description
1	**date**	Stores the date of the recorded presence.

Class: Childphoto

#	Attribute Name	Description
1	**expiresAt**	Stores the expiration date of the shared child photo.
2	**filePath**	Stores the file path of the child photo.

Class: Child

#	Attribute Name	Description
1	**birthDate**	Stores the child's date of birth.
2	**firstName**	Stores the child's first name.
3	**lastName**	Stores the child's last name.

Class: Childphotorecipient

#	Attribute Name	Description
1	**downloadTime**	Stores the date when the recipient downloaded the child photo.

Class: Pickupnotifications

#	Attribute Name	Description
1	**duration**	Stores the duration during which the pickup notification remains valid.

Class: Pickupnotificationsstatus

#	Attribute Name	Description
1	**name**	Stores the pickup notification status value.

Class: Class

#	Attribute Name	Description
1	**Capacity**	Stores the maximum number of children allowed in the class.
2	**isArchived**	Indicates whether the class is archived.
3	**name**	Stores the class name.
4	**Year**	Stores the academic year associated with the class.

Class: Classtype

#	Attribute Name	Description
1	**name**	Stores the class type name.

Nutrition & Health – Class Diagram Fragment

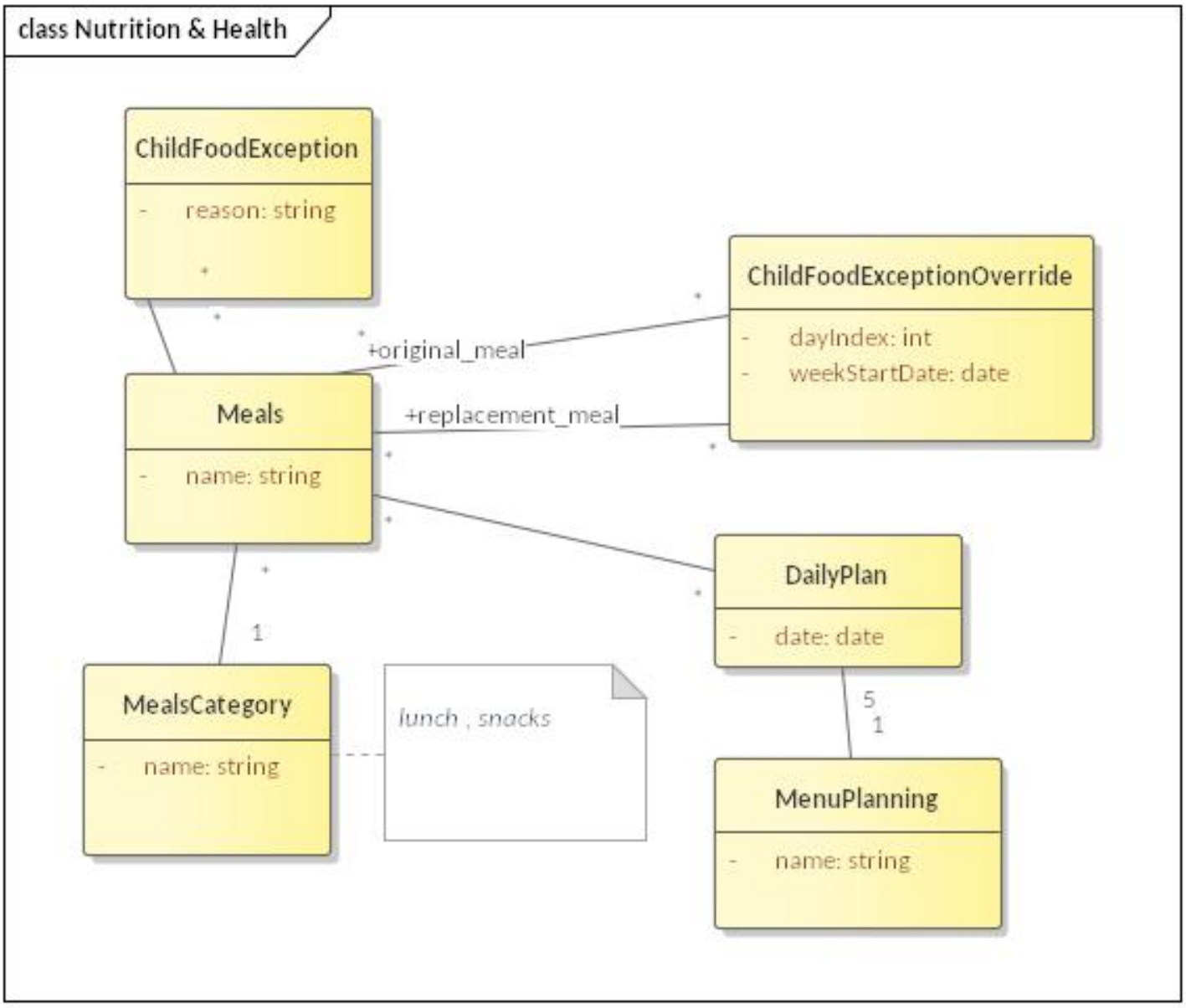


Figure 3.10: Class Diagram: Nutrition & Health

Class: Childfoodexception

#	Attribute Name	Description
1	**reason**	Stores the reason for the child's food exception.

Class: Dailyplan

#	Attribute Name	Description
1	**date**	Stores the date associated with the daily plan.

Class: Menuplanning

#	Attribute Name	Description
1	**name**	Stores the menu planning name.

Class: Meals

#	Attribute Name	Description
1	**name**	Stores the meal name.

Class: Mealscategory

#	Attribute Name	Description
1	**name**	Stores the meal category name.

Class: Childfoodexceptionoverride

#	Attribute Name	Description
1	**dayIndex**	Stores the day index concerned by the food exception override.
2	**weekStartDate**	Stores the start date of the week for the override.

Educational Planning – Class Diagram Fragment

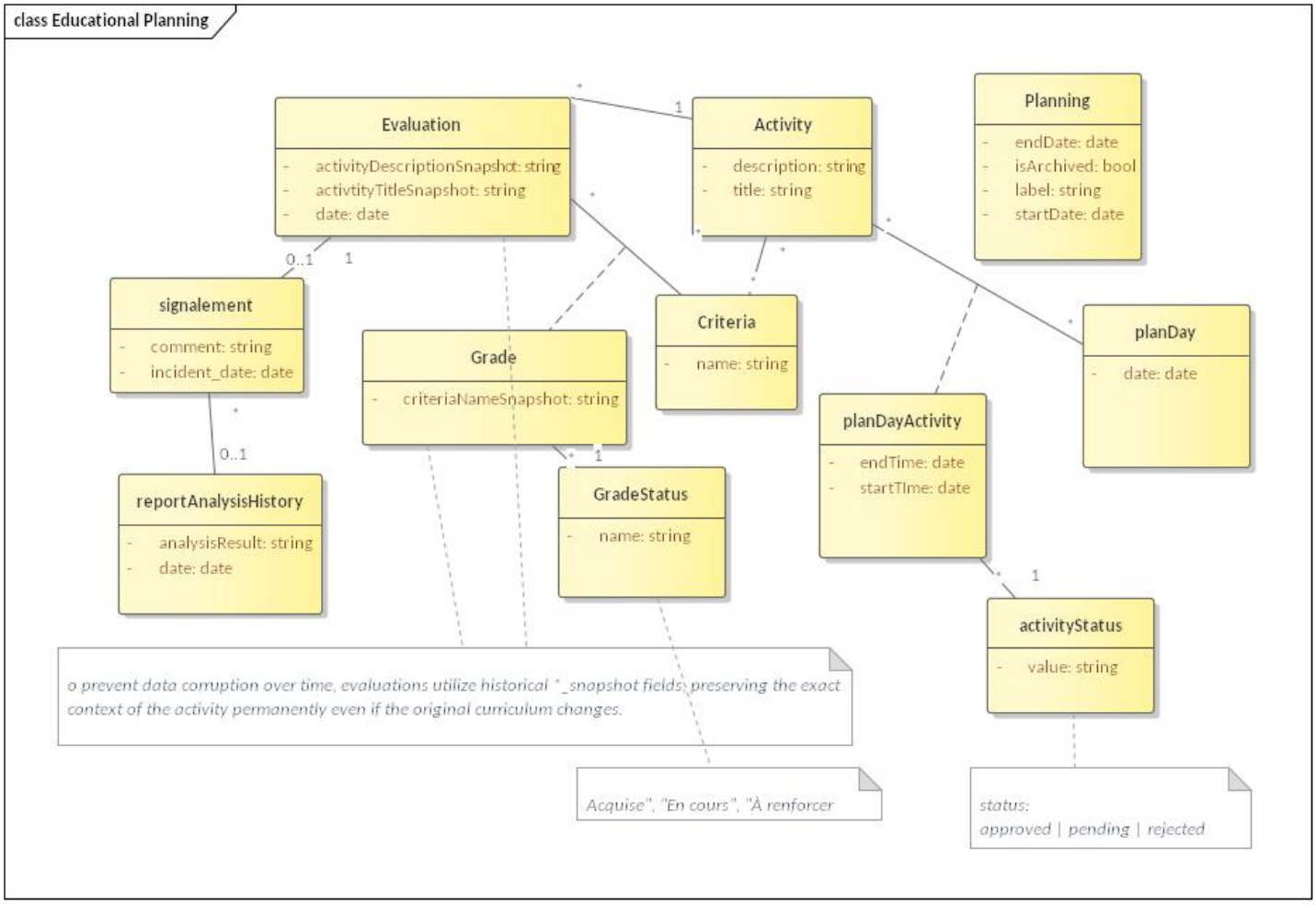


Figure 3.11: Class Diagram: Educational Planning

Class: Planning

#	Attribute Name	Description
1	**endDate**	Stores the ending date of the planning period.
2	**isArchived**	Indicates whether the planning is archived.
3	**label**	Stores the label of the planning.
4	**startDate**	Stores the starting date of the planning period.

Class: Planday

#	Attribute Name	Description
1	**date**	Stores the date associated with the plan day.

Class: Plandayactivity

#	Attribute Name	Description
1	**endTime**	Stores the ending time of the planned activity.
2	**startTime**	Stores the starting time of the planned activity.

Class: Activitystatus

#	Attribute Name	Description
1	**value**	Stores the value of the activity status (e.g., approved, pending, rejected).

Class: Activity

#	Attribute Name	Description
1	**description**	Stores the activity description.
2	**title**	Stores the activity title.

Class: Criteria

#	Attribute Name	Description
1	**name**	Stores the criteria name.

Class: Evaluation

#	Attribute Name	Description
1	**activityDescriptionSnapshot**	Stores the activity description snapshot saved in the evaluation.
2	**activityTitleSnapshot**	Stores the activity title snapshot saved in the evaluation.
3	**date**	Stores the evaluation date.

Class: Grade

#	Attribute Name	Description
1	**criteriaNameSnapshot**	Stores the criteria name snapshot saved in the grade.

Class: Gradestatus

#	Attribute Name	Description
1	**name**	Stores the grade status name (e.g., Acquise, En cours, À renforcer).

Class: Signalement

#	Attribute Name	Description
1	**comment**	Stores the comment associated with the incident report.
2	**incident_date**	Stores the date of the reported incident.

Class: Reportanalysishistory

#	Attribute Name	Description
1	**analysisResult**	Stores the generated report analysis result.
2	**date**	Stores the date of the analysis record.

3.4 Future Software Architecture

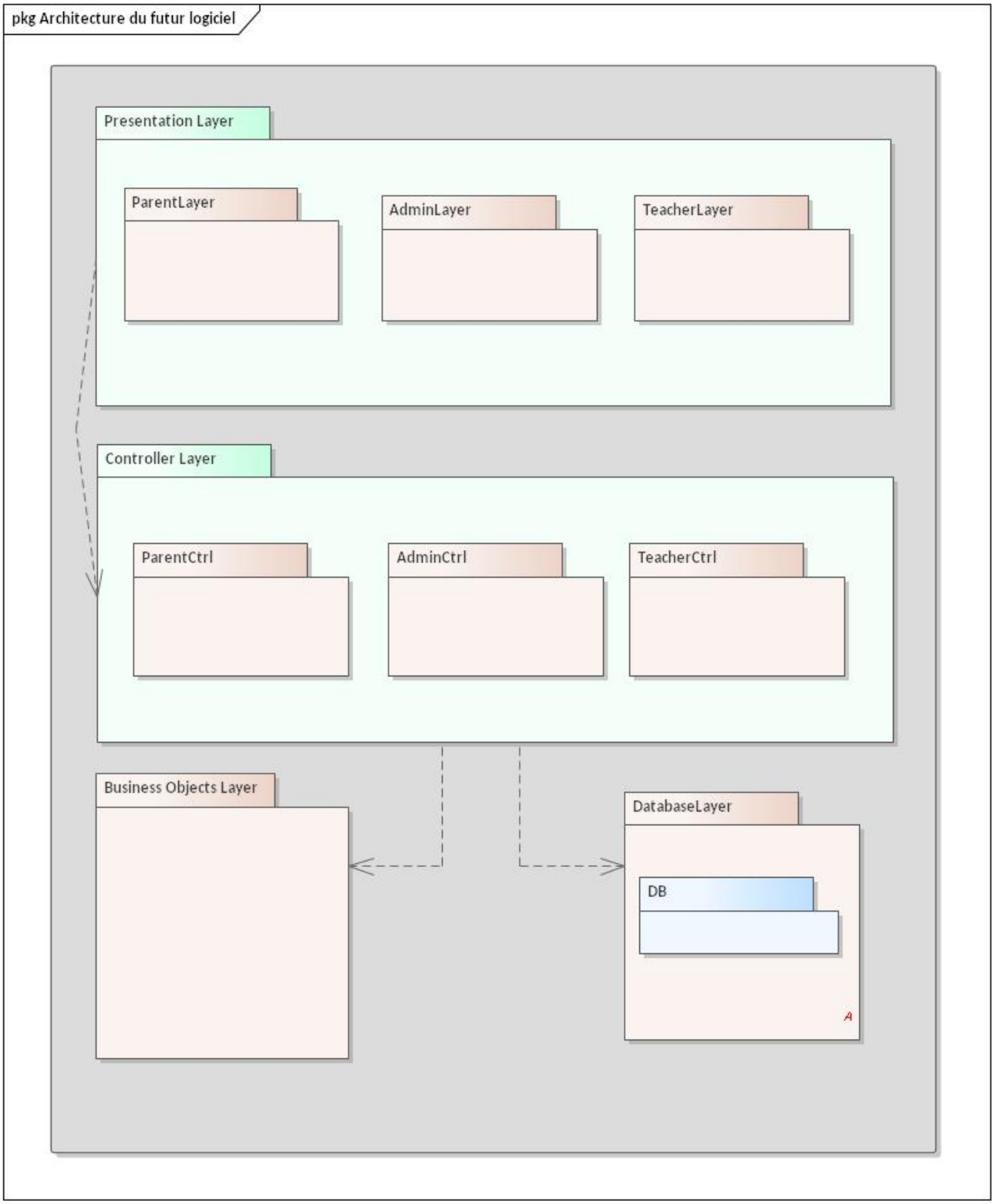


Figure 3.12: Future Software Architecture

3.5 Physical Data Schemas

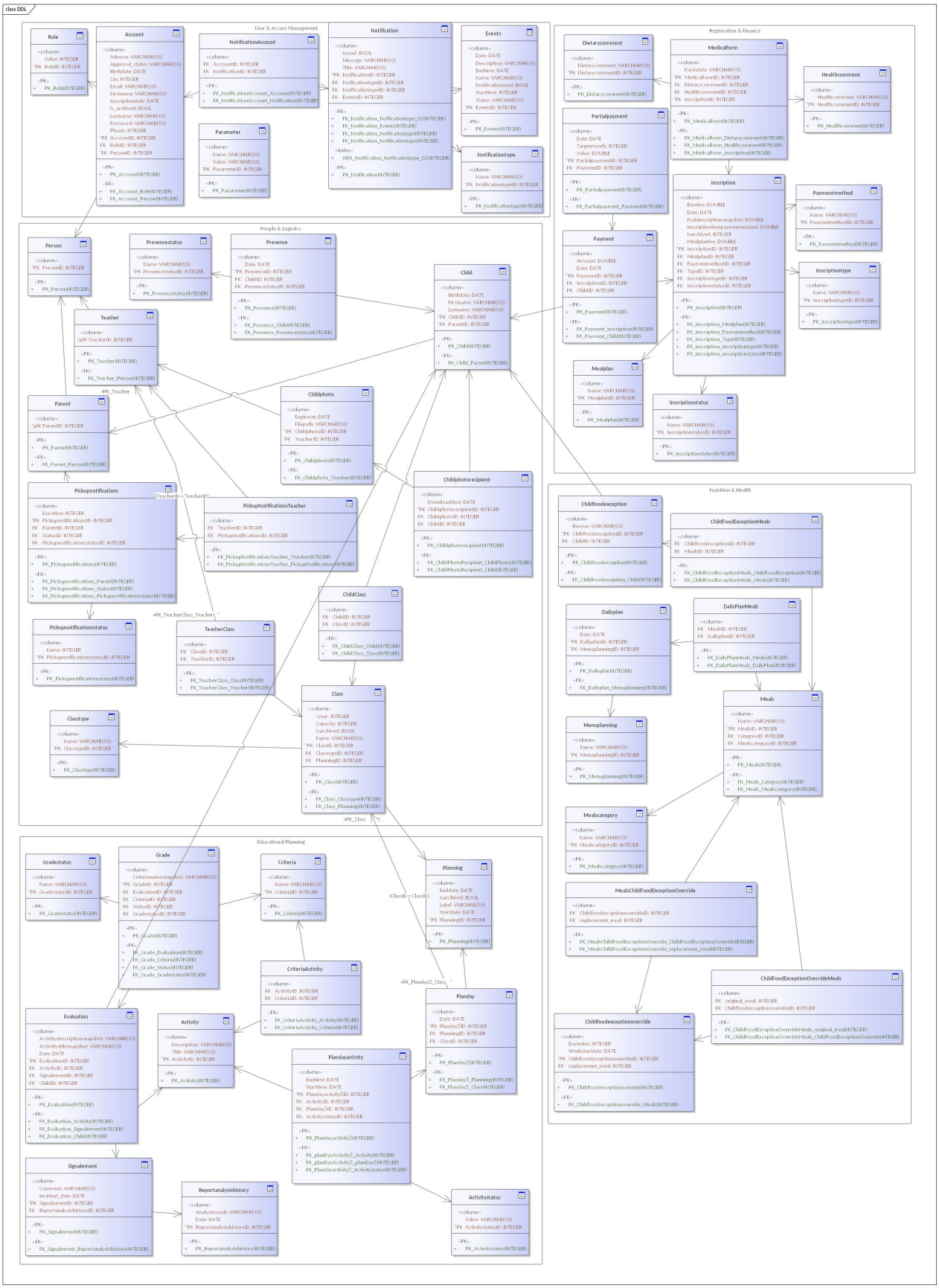


Figure 3.13: Physical Data Schemas

3.5.1 User & Access Management

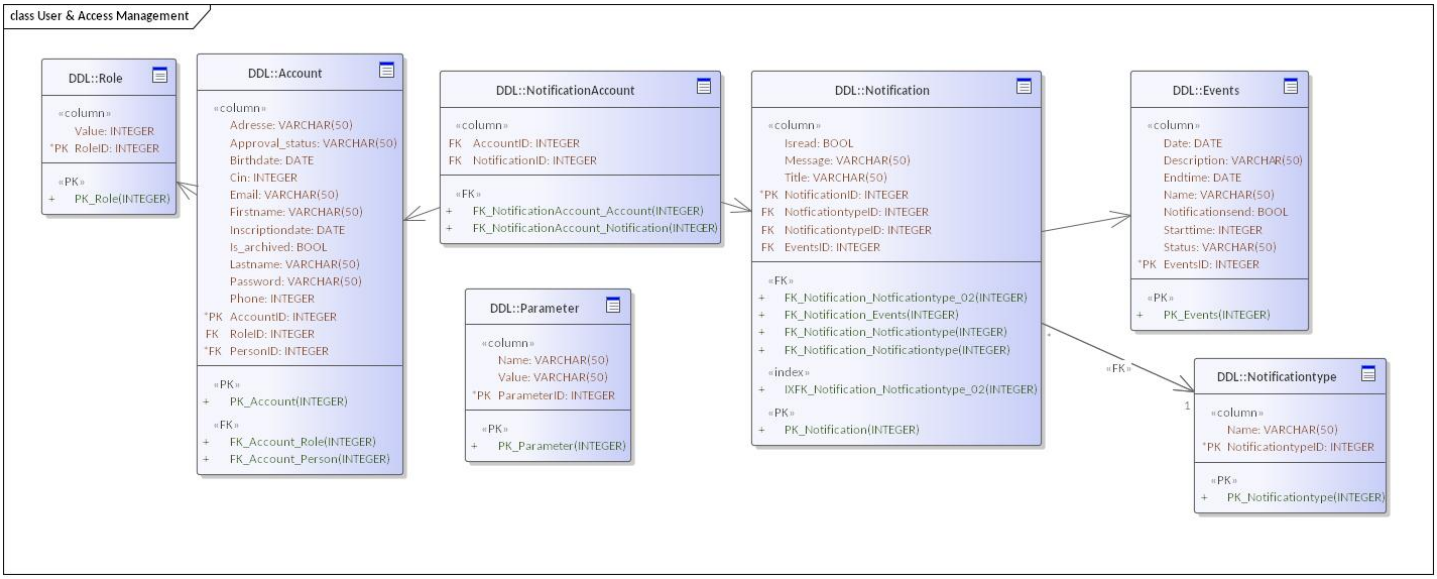


Figure 3.14: Physical Data Schema: User & Access Management

- **Role** (RoleID, Value)
- **Account** (AccountID, Adresse, Approval_status, Birthdate, Cin, Email, Firstname, Inscriptiondate, Is_archived, Lastname, Password, Phone, RoleID#, PersonID#)
- **NotificationAccount** (AccountID#, NotificationID#)
- **Notification** (NotificationID, Isread, Message, Title, NotificationtypeID#, EventsID#)
- **Events** (EventsID, Date, Description, Endtime, Name, Notificationsend, Starttime, Status)
- **Parameter** (ParameterID, Name, Value)
- **Notificationtype** (NotificationtypeID, Name)

3.5.2 Registration & Finance

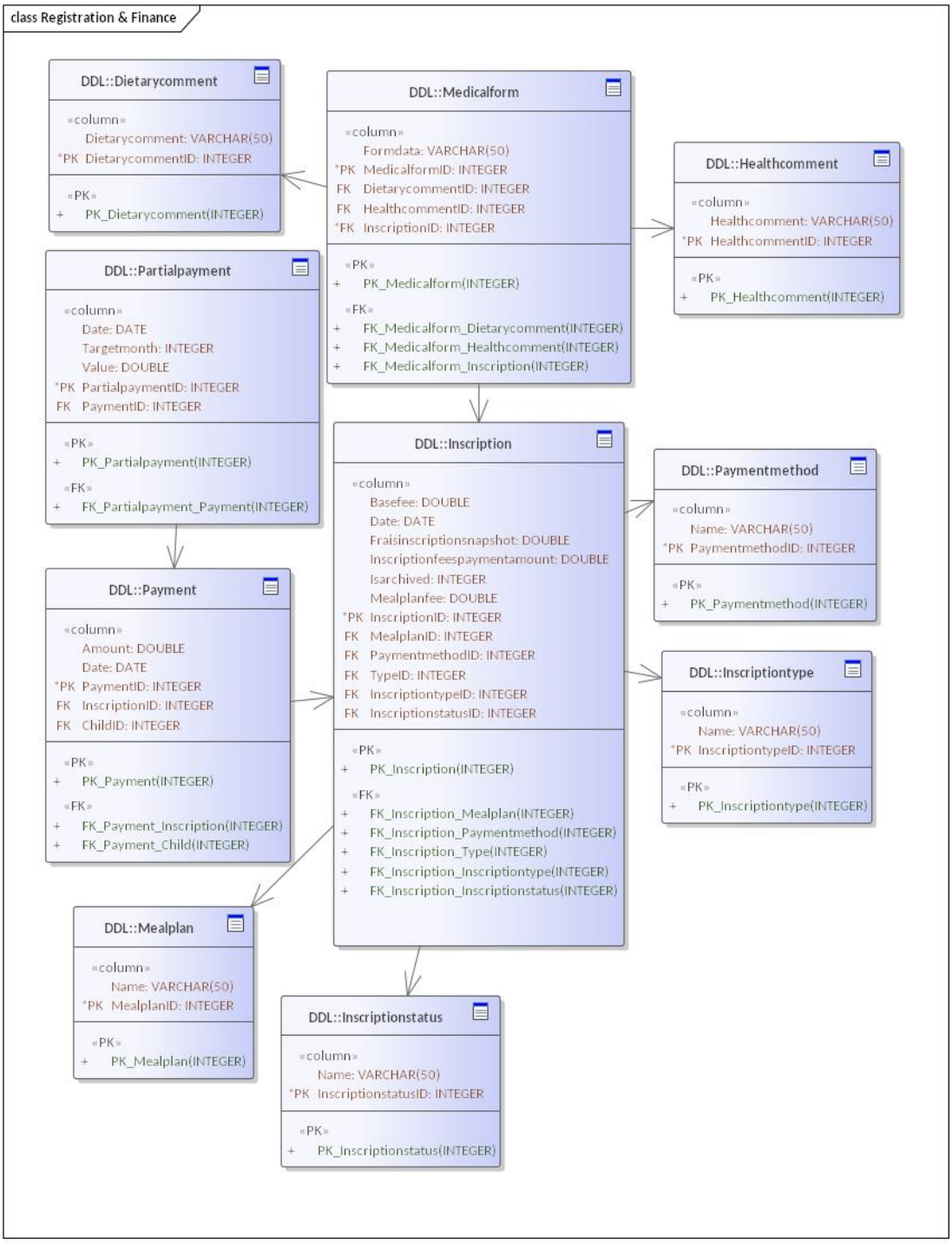


Figure 3.15: Physical Data Schema: Registration & Finance

- **Dietarycomment** (DietarycommentID, Dietarycomment)
- **Medicalform** (MedicalformID, Formdata, DietarycommentID#, HealthcommentID#, InscriptionID#)
- **Healthcomment** (HealthcommentID, Healthcomment)
- **Partialpayment** (PartialpaymentID, Date, Targetmonth, Value, PaymentID#)
- **Payment** (PaymentID, Amount, Date, InscriptionID#, ChildID#)

- **Inscription** (InscriptionID, Basefee, Date, Fraisinscriptionsnapshot, Inscriptionfeespaymentamount, Isarchived, Mealplanfee, MealplanID#, PaymentmethodID#, TypeID#, InscriptiontypeID#, InscriptionstatusID#)
- **Paymentmethod** (PaymentmethodID, Name)
- **Inscriptiontype** (InscriptiontypeID, Name)
- **Mealplan** (MealplanID, Name)
- **Inscriptionstatus** (InscriptionstatusID, Name)

3.5.3 People & Logistics

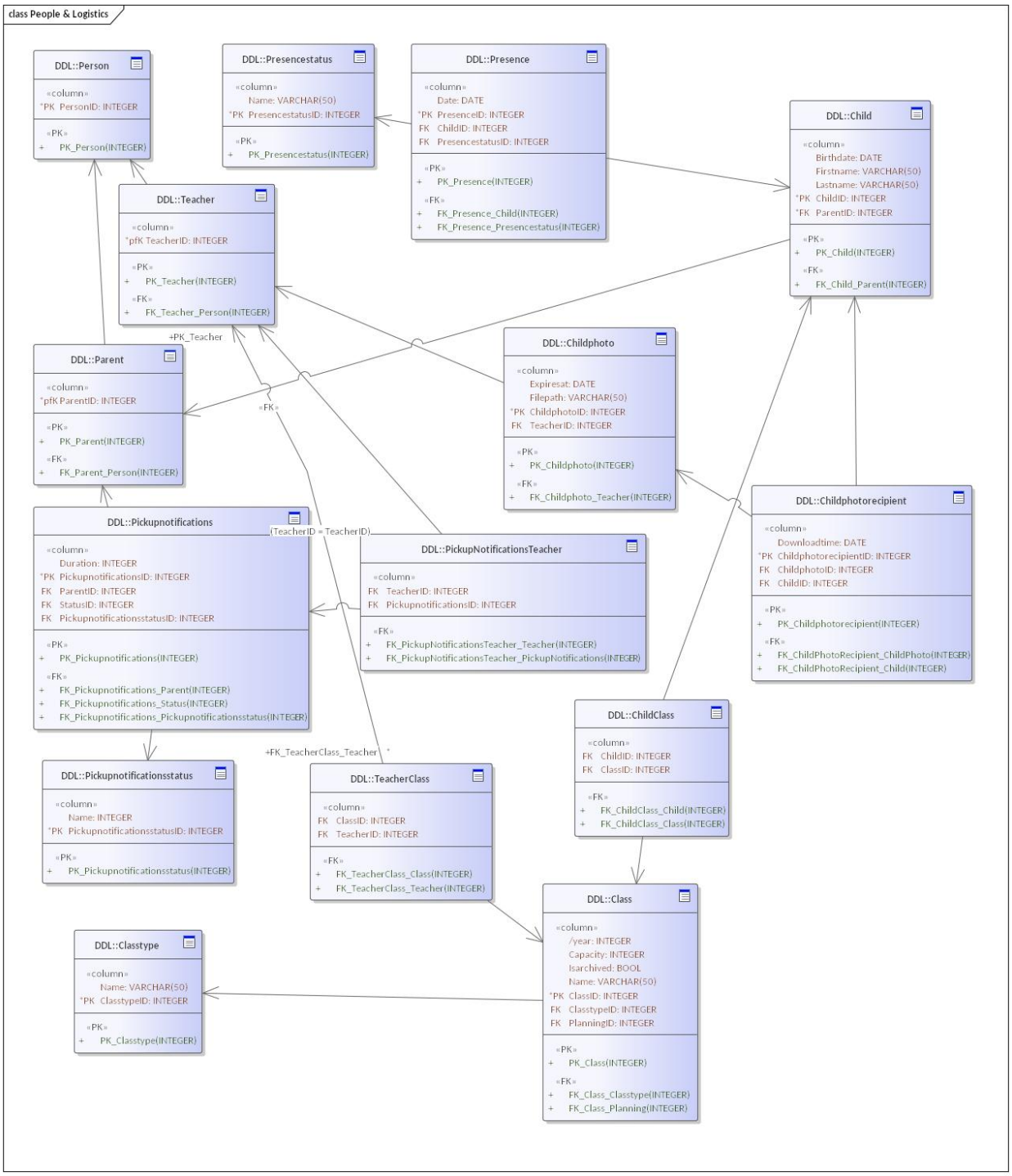


Figure 3.16: Physical Data Schema: People & Logistics

- **Person** (PersonID)
- **Presencestatus** (PresencestatusID, Name)
- **Presence** (PresenceID, Date, ChildID#, PresencestatusID#)
- **Teacher** (TeacherID, PersonID#)
- **Parent** (ParentID, PersonID#)
- **Childphoto** (ChildphotoID, Expiresat, Filepath, TeacherID#)
- **Child** (ChildID, Birthdate, Firstname, Lastname, ParentID#)
- **Childphotorecipient** (ChildphotorecipientID, Downloadtime, ChildphotoID#, ChildID#)
- **Pickupnotifications** (PickupnotificationsID, Duration, ParentID#, StatusID#, PickupnotificationsstatusID#)
- **PickupNotificationsTeacher** (TeacherID#, PickupnotificationsID#)
- **Pickupnotificationsstatus** (PickupnotificationsstatusID, Name)
- **TeacherClass** (ClassID#, TeacherID#)
- **ChildClass** (ChildID#, ClassID#)
- **Class** (ClassID, Capacity, Isarchived, Name, year, ClasstypeID#, PlanningID#)
- **Classtype** (ClasstypeID, Name)

3.5.4 Nutrition & Health

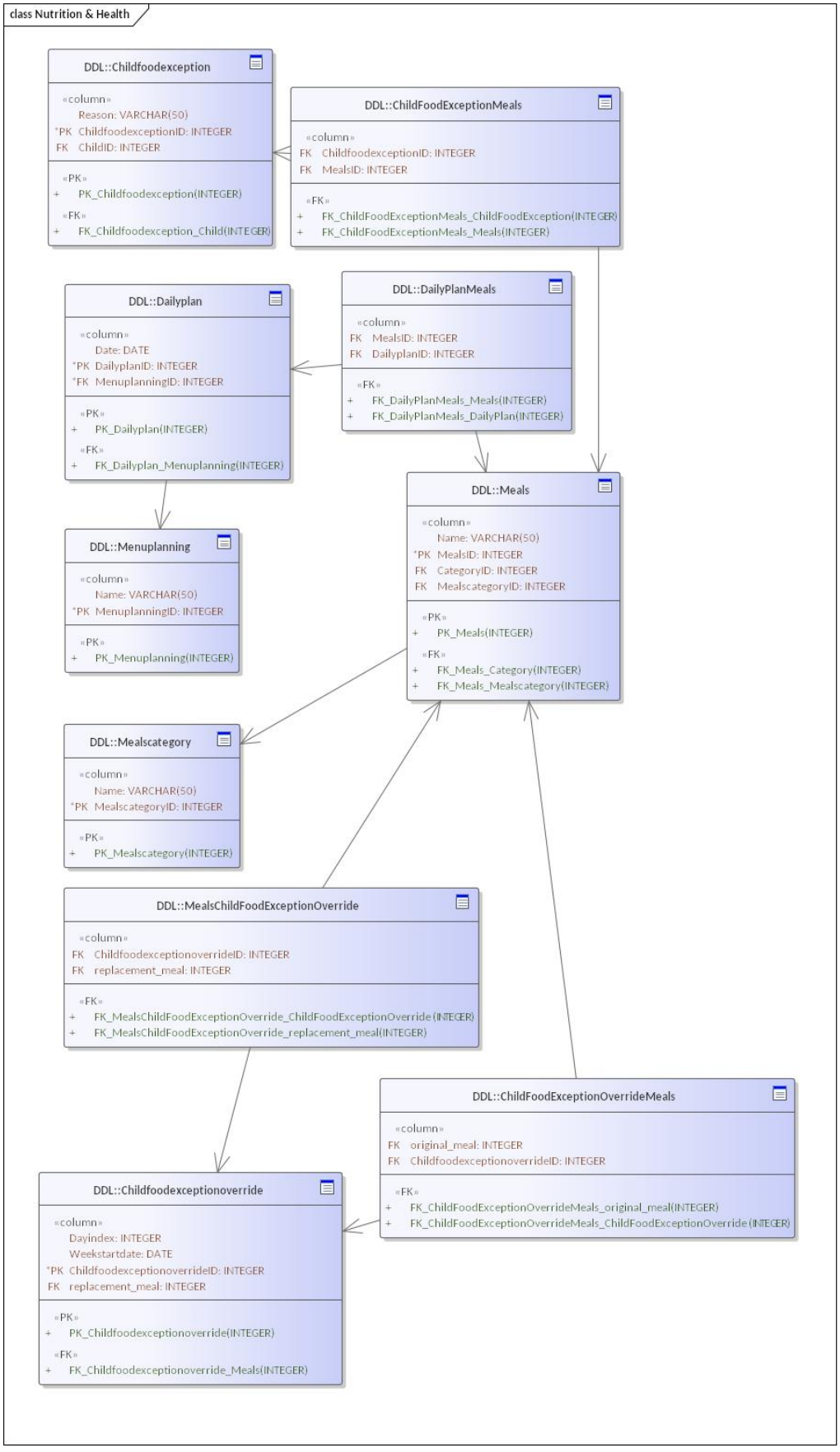


Figure 3.17: Physical Data Schema: Nutrition & Health

- **Childfoodexception** (ChildfoodexceptionID, Reason, ChildID#)
- **ChildFoodExceptionMeals** (ChildfoodexceptionID#, MealsID#)
- **Dailyplan** (DailyplanID, Date, MenuplanningID#)
- **DailyPlanMeals** (MealsID#, DailyplanID#)
- **Menuplanning** (MenuplanningID, Name)
- **Meals** (MealsID, Name, CategoryID#, MealscategoryID#)
- **Mealscategory** (MealscategoryID, Name)
- **Childfoodexceptionoverride** (ChildfoodexceptionoverrideID, Dayindex, Weekstartdate, replacement_meal#)
- **MealsChildFoodExceptionOverride** (ChildfoodexceptionoverrideID#, replacement_meal#)
- **ChildFoodExceptionOverrideMeals** (original_meal#, ChildfoodexceptionoverrideID#)

3.5.5 Educational Planning

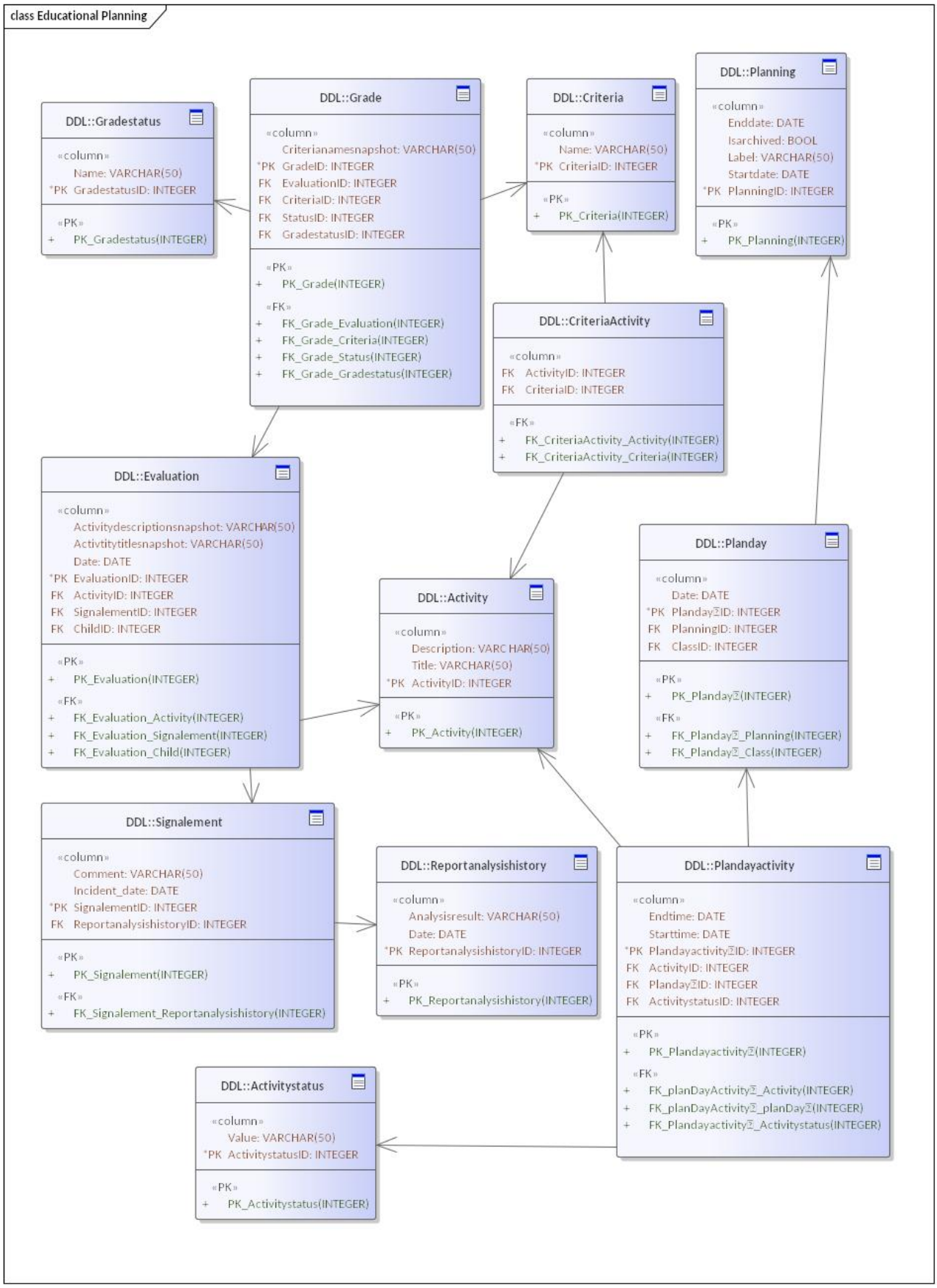


Figure 3.18: Physical Data Schema: Educational Planning

- **Planning** (PlanningID, Enddate, Isarchived, Label, Startdate)
- **Planday** (PlandayID, Date, PlanningID#, ClassID#)
- **Plandayactivity** (PlandayactivityID, Endtime, Starttime, ActivityID#, PlandayID#, ActivitystatusID#)
- **Activity** (ActivityID, Description, Title)
- **CriteriaActivity** (ActivityID#, CriteriaID#)
- **Criteria** (CriteriaID, Name)
- **Evaluation** (EvaluationID, Activitydescriptionsnapshot, Activitytitlesnapshot, Date, ActivityID#, SignalementID#, ChildID#)
- **Grade** (GradeID, Criterianamesnapshot, EvaluationID#, CriteriaID#, StatusID#, GradestatusID#)
- **Gradestatus** (GradestatusID, Name)
- **Signalement** (SignalementID, Comment, Incident_date, ReportanalysishistoryID#)
- **Reportanalysishistory** (ReportanalysishistoryID, Analysisresult, Date)
- **Activitystatus** (ActivitystatusID, Value)

3.6 Conclusion

Through the formalization of the use cases with sequence diagrams and the structural definition provided by the class diagram, this chapter established a solid architectural foundation for the Petit Suivi system. This analysis ensures overall system coherence and provides a clear transition to the technical implementation phase presented in the next chapter.

Chapter 4

Implementation

4.1 Introduction

This chapter is dedicated to the realization of the **Petit Suivi** platform. It bridges the gap between our abstract modeling phase and the final, tangible product. We begin by defining the development environment, detailing the hardware specifications and the suite of software and technologies selected for developing the diverse components of the platform.

4.2 Development Environment

The development environment consists of the combination of hardware and software resources required to effectively build, test, and deploy the application.

4.2.1 Hardware Environment

- **Operating System:** Arch Linux (64-bit).
- **Processor:** High-performance multi-core processor.
- **Memory (RAM):** 16 GB.
- **Storage:** 512 GB SSD.

4.2.2 Software Environment and Technologies

Backend API

Laravel (PHP) and MySQL.

Frontend Web

React.js for the administrative dashboard.

Mobile Application

Flutter (Dart) for Parents and Teachers.

AI Microservice

Node.js, Express, and GitHub Copilot SDK.

4.2.3 Deployment and Infrastructure Tools

Neovim, Postman, and GitHub CI/CD pipelines.

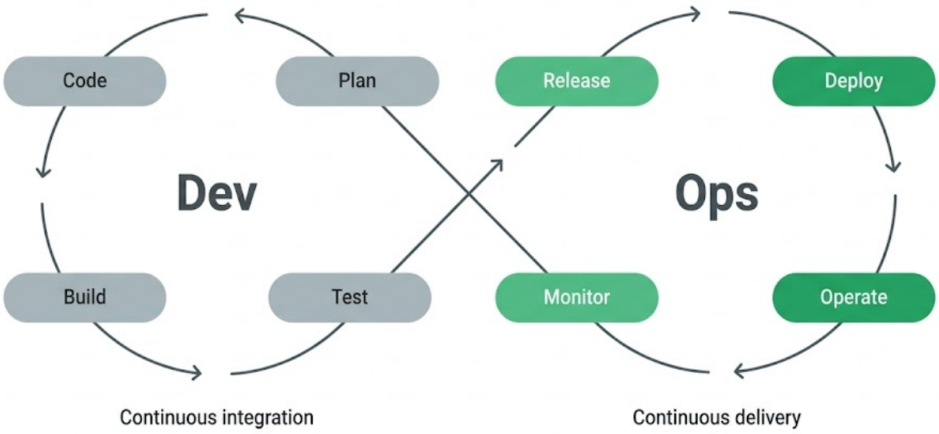


Figure 4.1: CI/CD Pipeline Configuration

4.3 Technical Implementation of Use Cases

4.3.1 Parent Use Cases

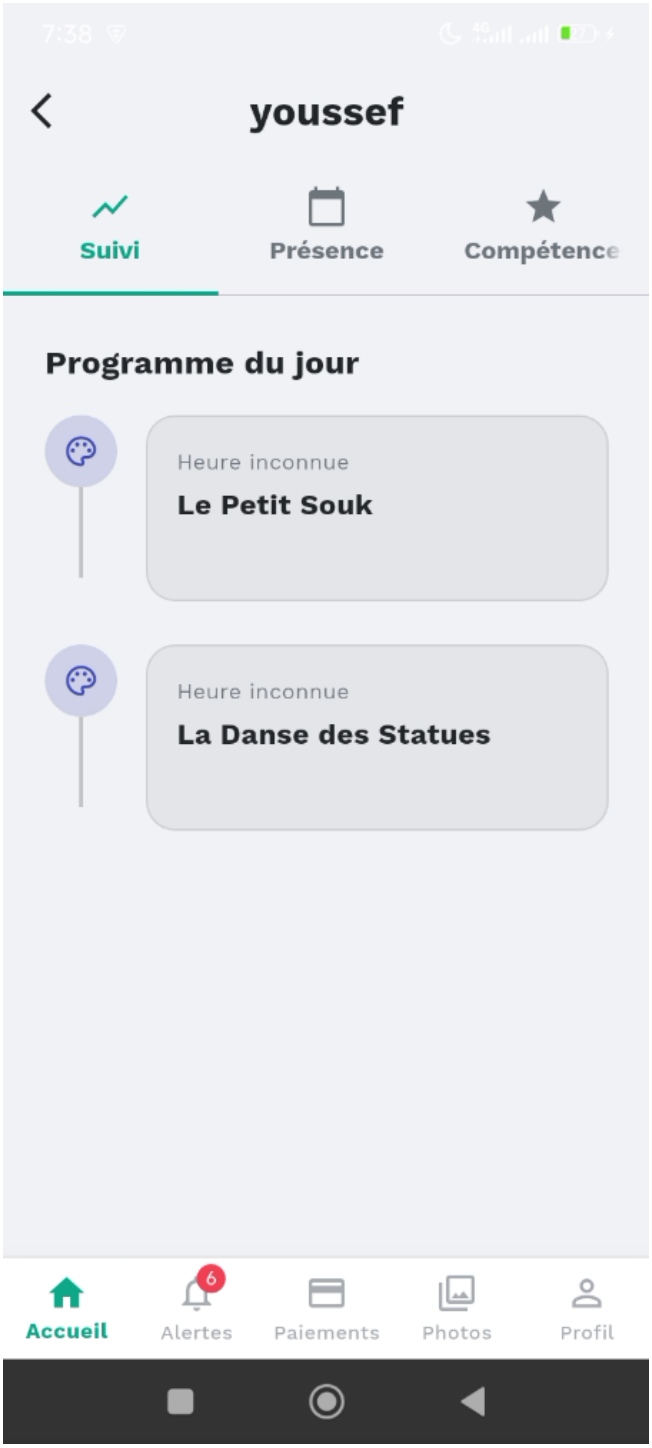
Use Case name: Track Children



Description: This interface allows the parent to view their children's profiles and send real-time pickup notifications to the respective teachers. By notifying the school in advance, the parent ensures the child is ready upon arrival, effectively eliminating wait times and streamlining the dismissal process.

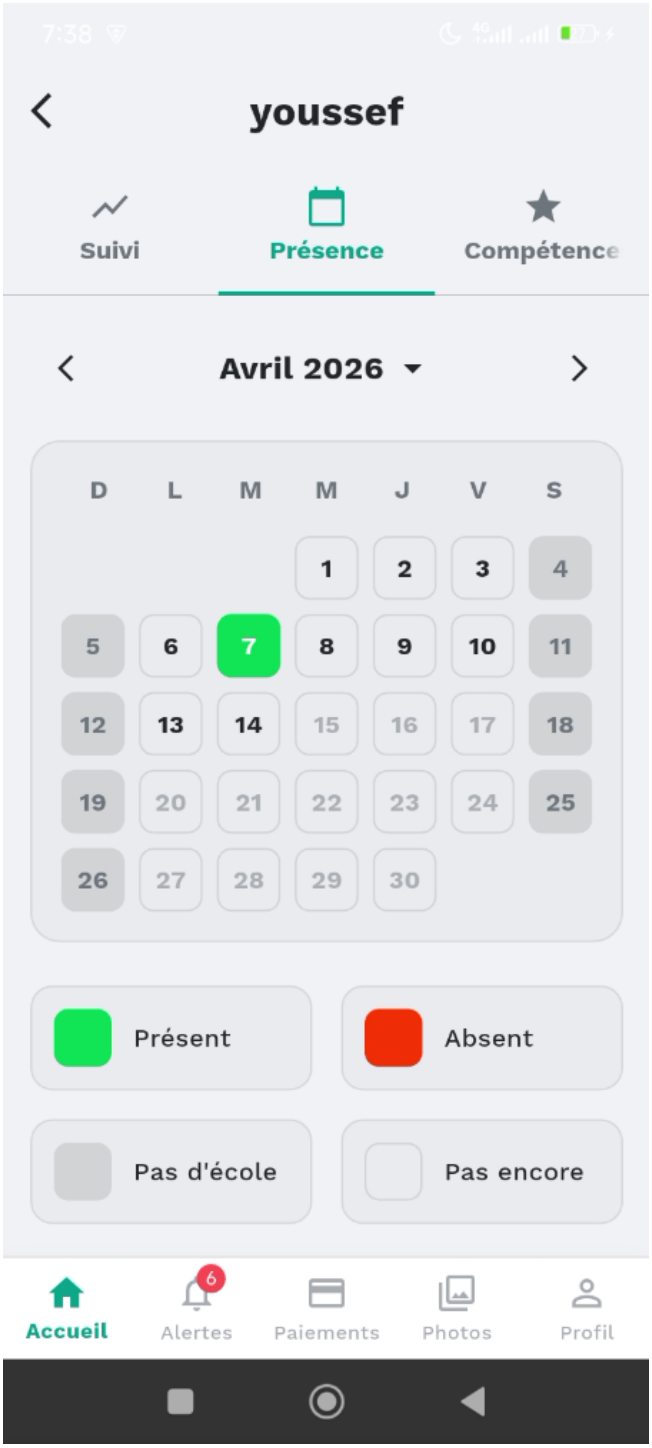
Use Case name: Track Children

Scenario: Consult today's scheduled activities for a child



Use Case name: Track Children

Scenario: Check daily attendance status of a child



Use Case name: Track Children

Scenario: Review acquired competences of a child



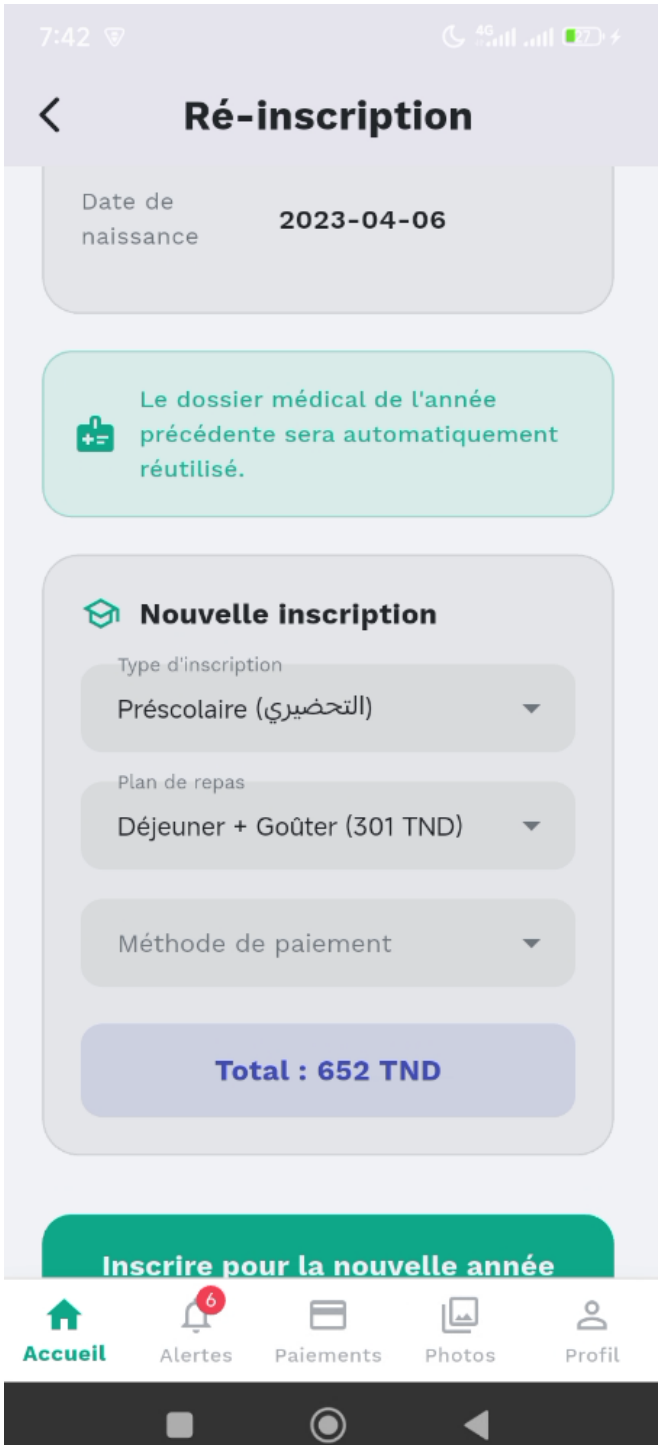
Use Case name: View Photos

Scenario: Browse photo gallery of a child



Use Case name: Register Parent & Children

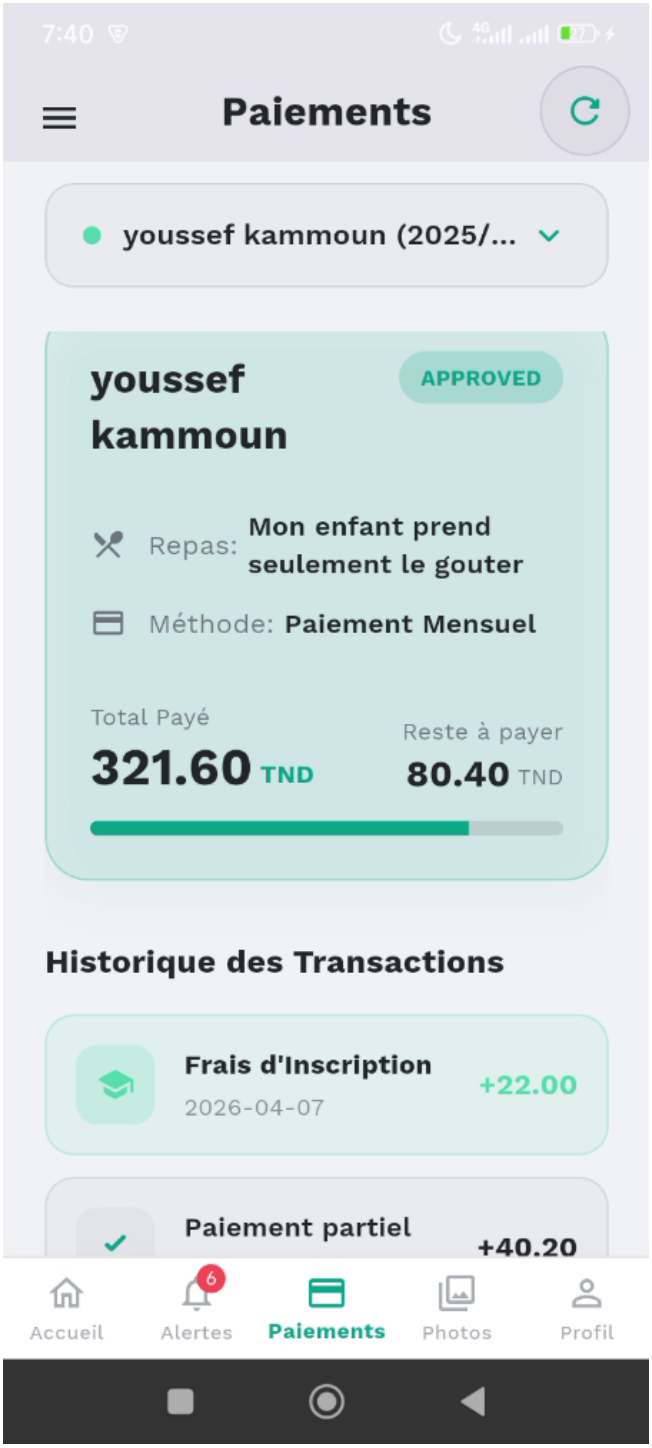
Scenario: Reinscribe an existing child for a new academic year (existing account)



Description: This interface enables parents to efficiently re-enroll their existing children for the upcoming academic year. It allows the user to select new enrollment types, meal plans, and payment methods, while automatically calculating the total fees to ensure transparency before final submission.

Use Case name: Review Payment History

Scenario: Consult monthly payment records of a child



Description: This interface provides parents with a comprehensive overview of the financial status for each child's enrollment. It displays specific registration details, including selected meal plans and payment methods, while tracking the total amount paid versus the remaining balance.

4.3.2 Teacher Use Cases

Use Case name: View All Activities

Scenario: Browse all planned activities of a class for the current academic year



Use Case name: Send Photos

Scenario: Upload and share photos for a specific child, an entire class, or a selected group of children



Description: This interface allows authorized Teachers to share photos with parents by targeting specific classrooms. The process is organized into three steps:

- **Class Selection:** Selecting the target classroom.

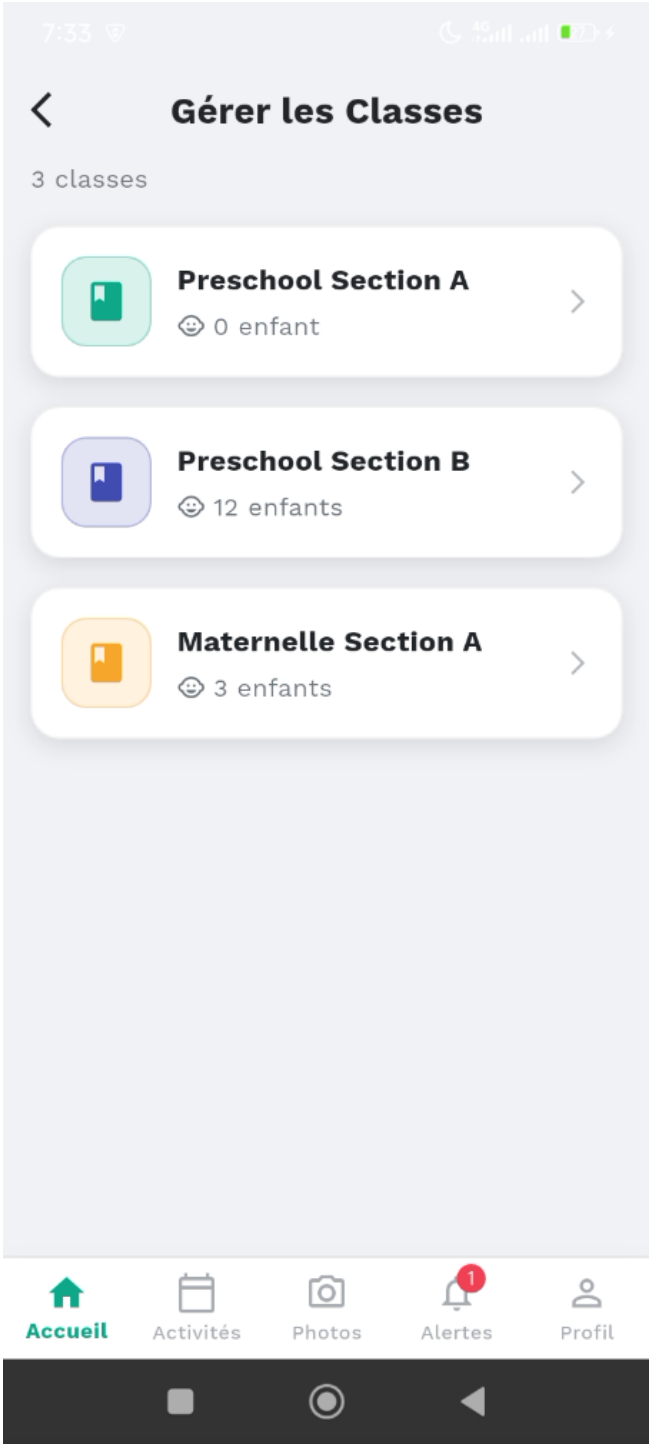
- **Recipient Identification:** Identifying the specific children to receive the media.
- **Media Upload:** Uploading the photos to the system.

Additionally, the user can set an expiration duration for the shared content, ensuring that media is only available for a defined period to maintain data privacy and storage efficiency.

Use Case name: Send Photos

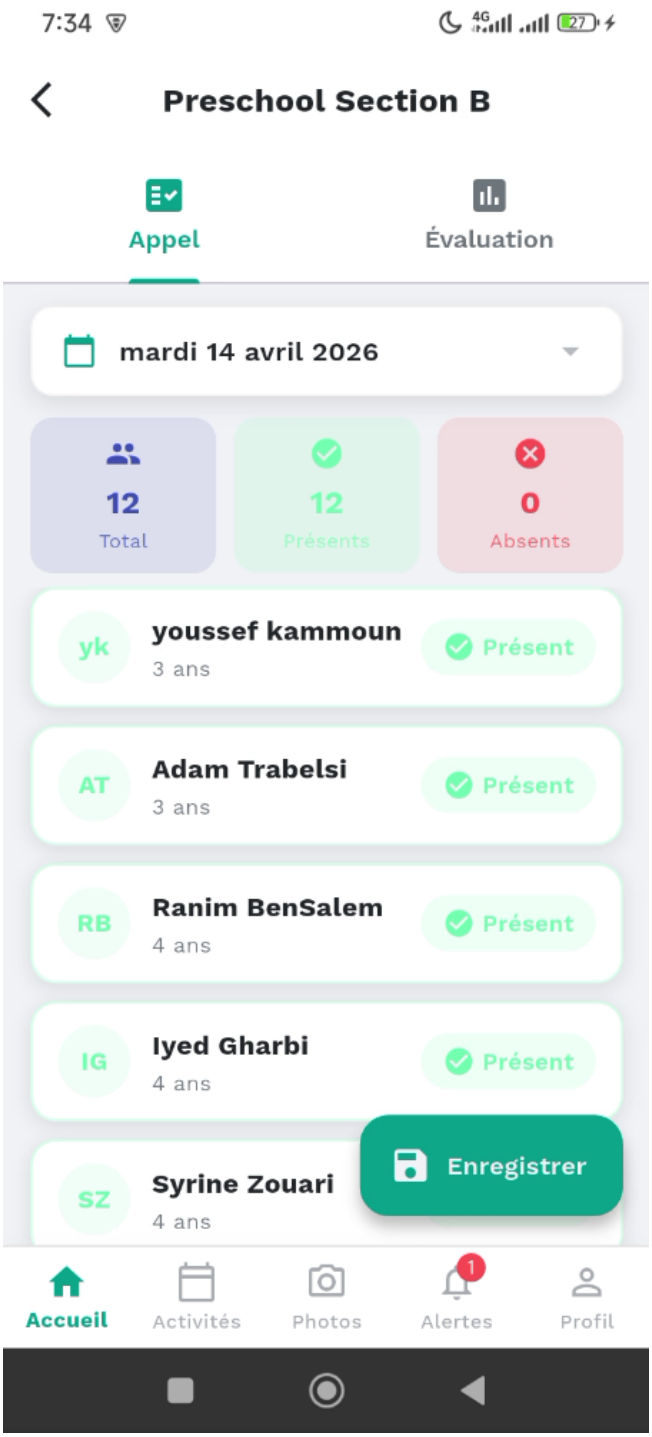
```
return ListView(
  physics: const AlwaysScrollableScrollPhysics(),
  padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
  children: [
    SectionCard(
      title: '1) Classe',
      child: ClassSelector(controller: _controller),
    ), // SectionCard
    const SizedBox(height: 12),
    SectionCard(
      title: '2) Enfants destinataires',
      child: ChildrenSelector(controller: _controller),
    ), // SectionCard
    const SizedBox(height: 12),
    SectionCard(
      title: '3) Photo',
      child: PhotoUpload(controller: _controller),
    ), // SectionCard
    const SizedBox(height: 18),
    RecentShares(controller: _controller),
  ],
); // ListView
```

Use Case name: Manage Classes



Use Case name: Manage Class Attendance

Scenario: Record daily attendance for all children in a class



Use Case name: Evaluate Child Competences

Scenario: Assess each competence of an activity as acquired or needs reinforcement



4.3.3 Parent and Teacher Shared Use Cases

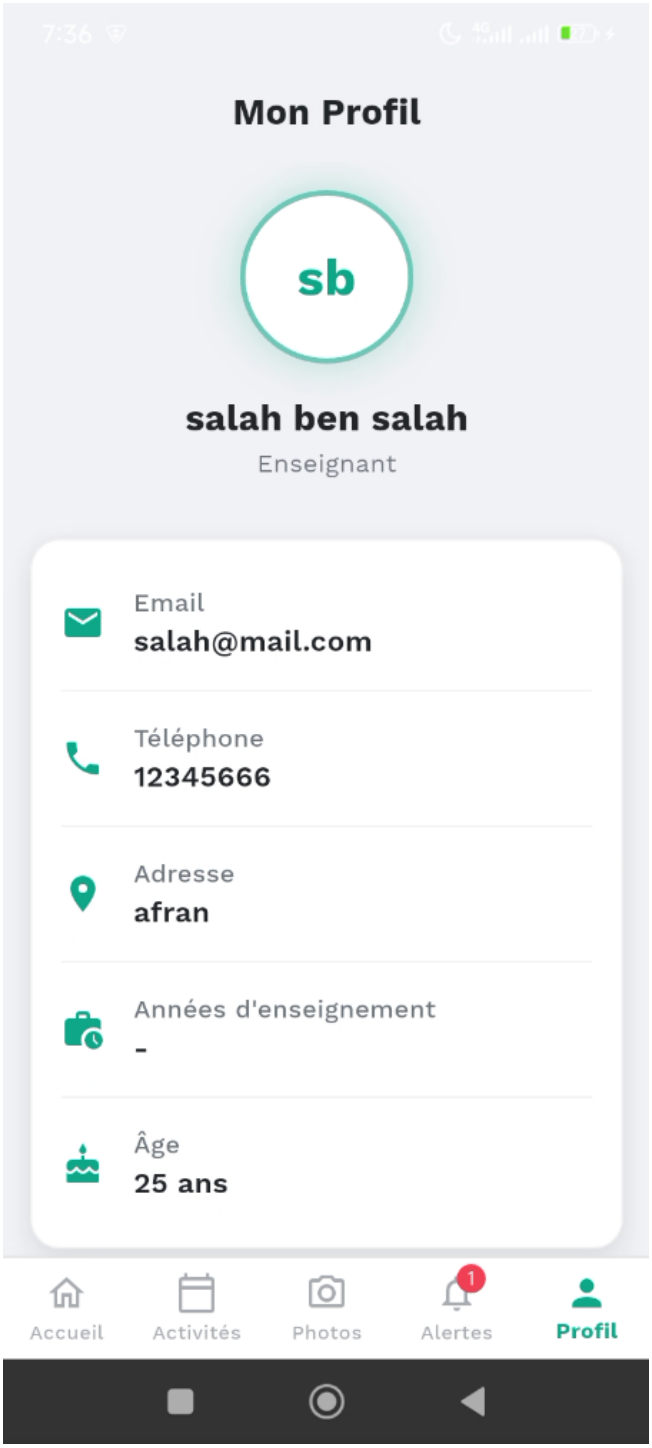
Use Case name: Manage Parent Profile

Scenario: View personal profile details



Use Case name: Manage Teacher Profile

Scenario: View personal profile details



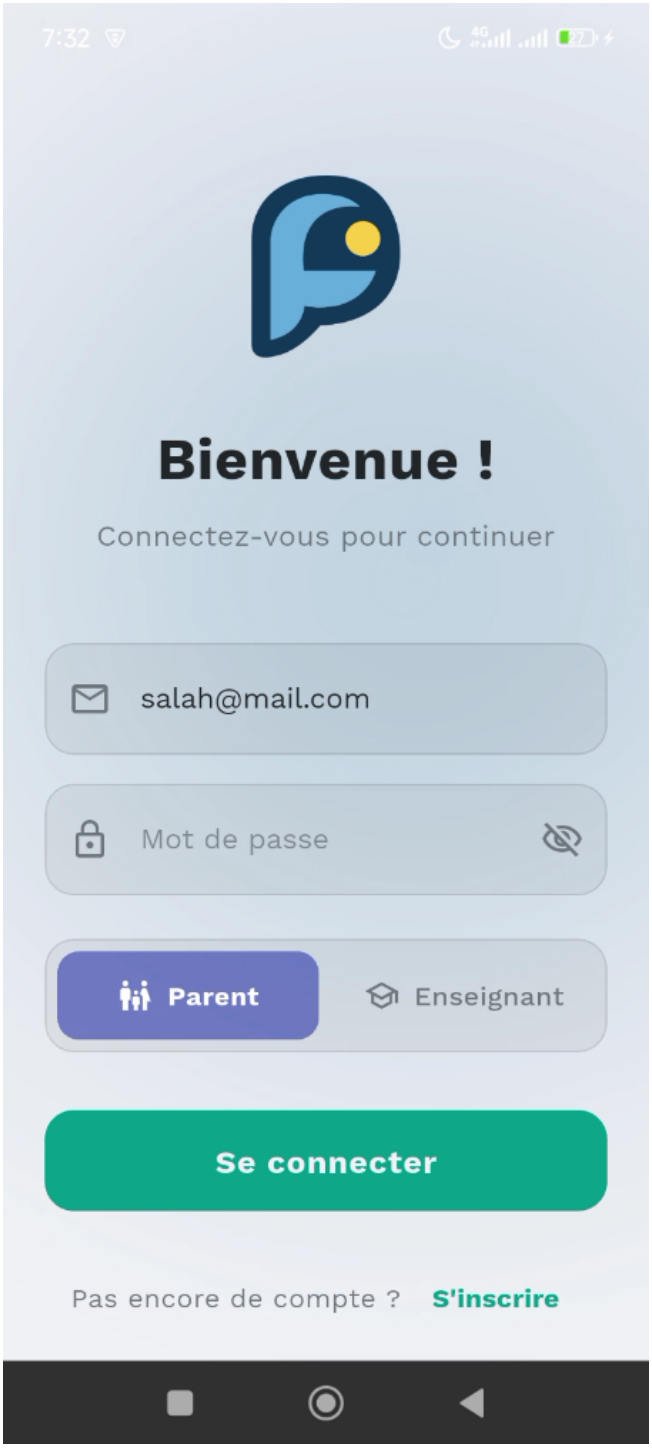
Use Case name: View Notifications

Scenario: Browse and read received notifications



Use Case name: Authenticate

Scenario: Log in to the mobile application via email and password

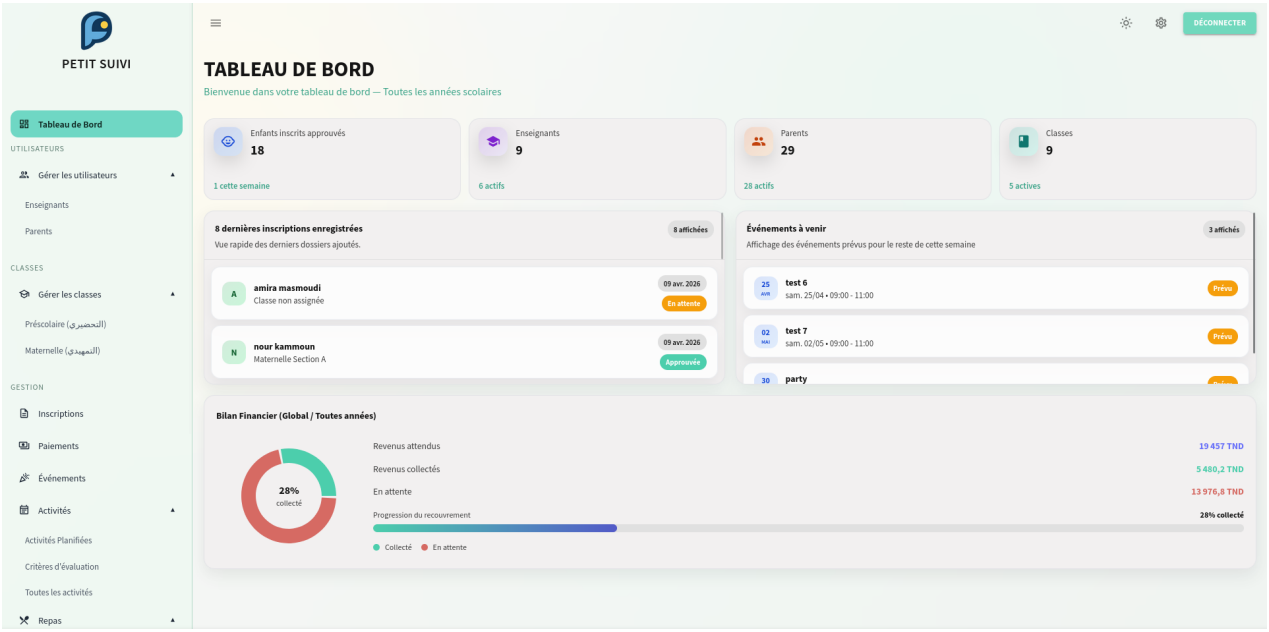


4.3.4 Administrator Use Cases

Description: This interface allows the administrator to view real-time statistics, including active users and pending registration requests.

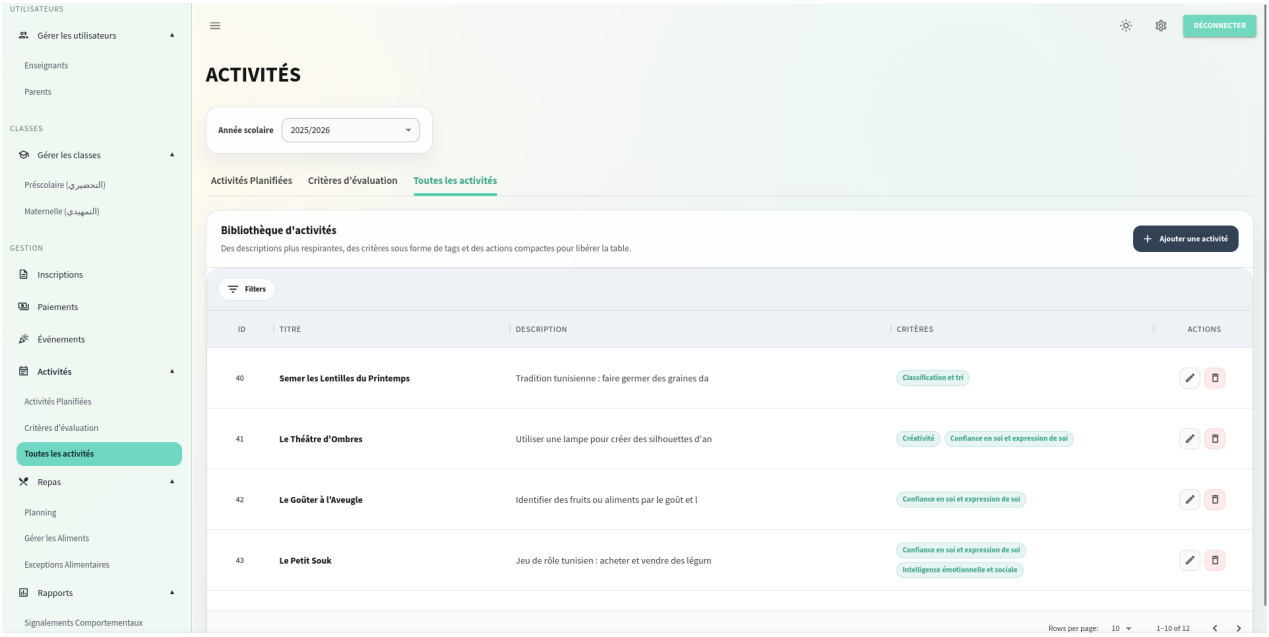
Use Case name: Access Dashboard

Scenario: View aggregated statistics across all academic years



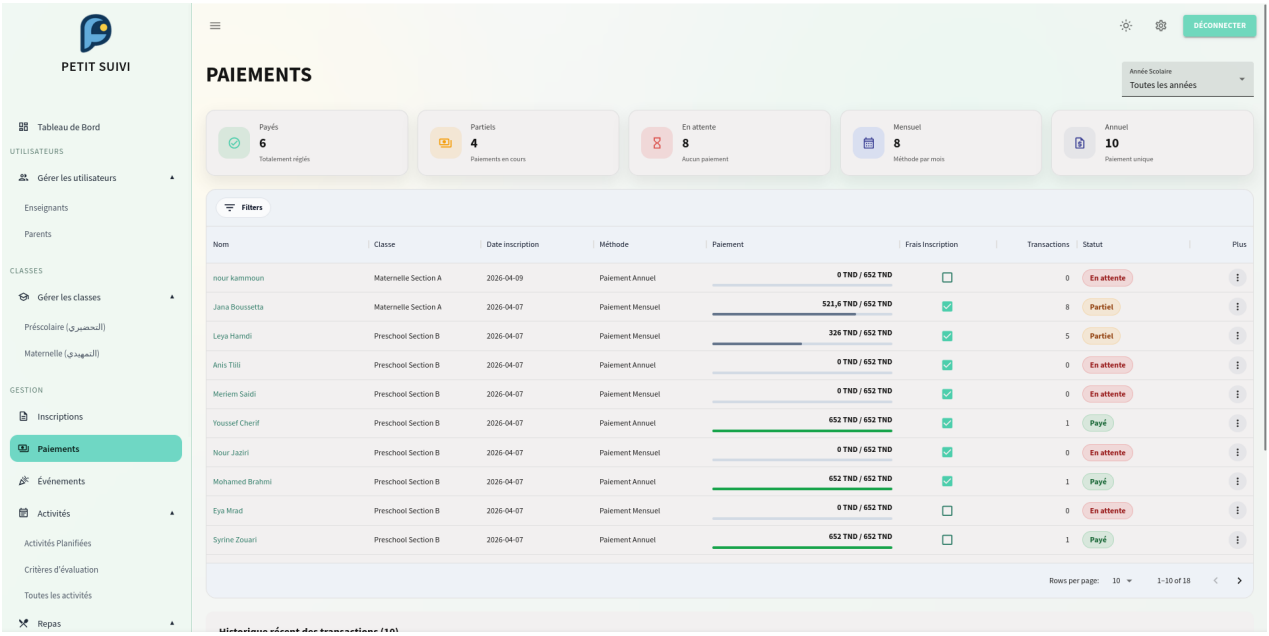
Use Case name: Manage Activities

Scenario: Create a new activity



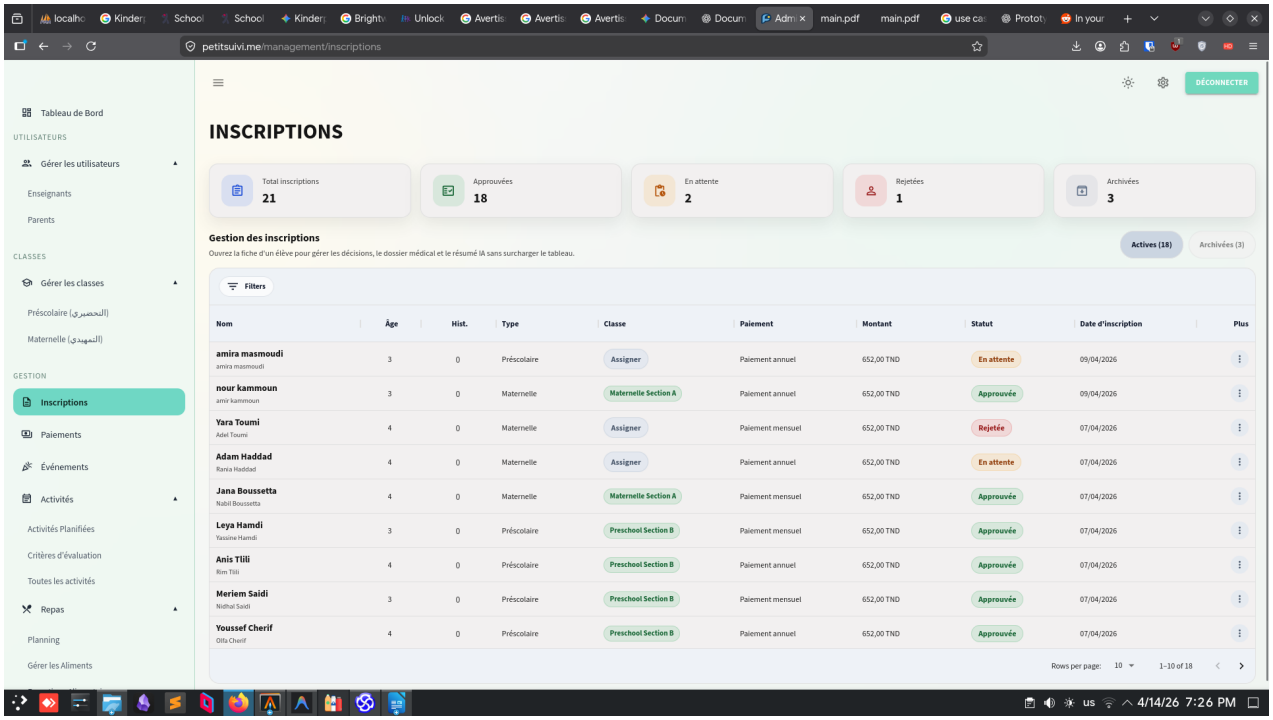
Use Case name: Manage Payments

Scenario: Record a monthly payment as received



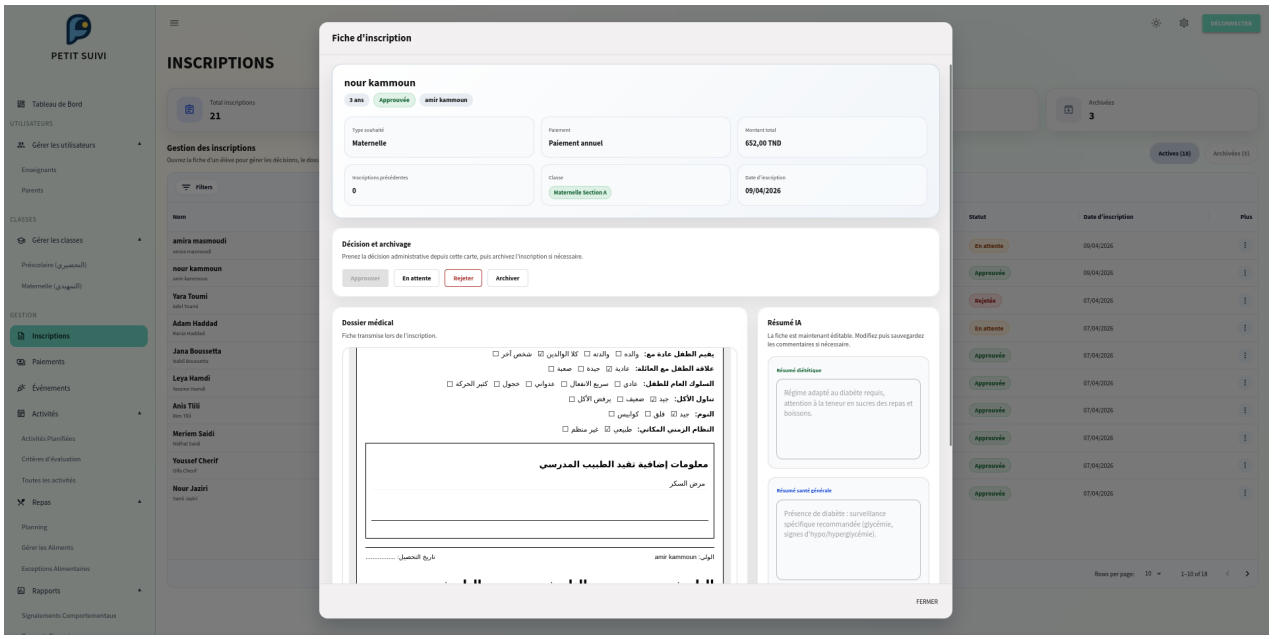
Use Case name: Manage Inscriptions

Scenario: View pending inscriptions list



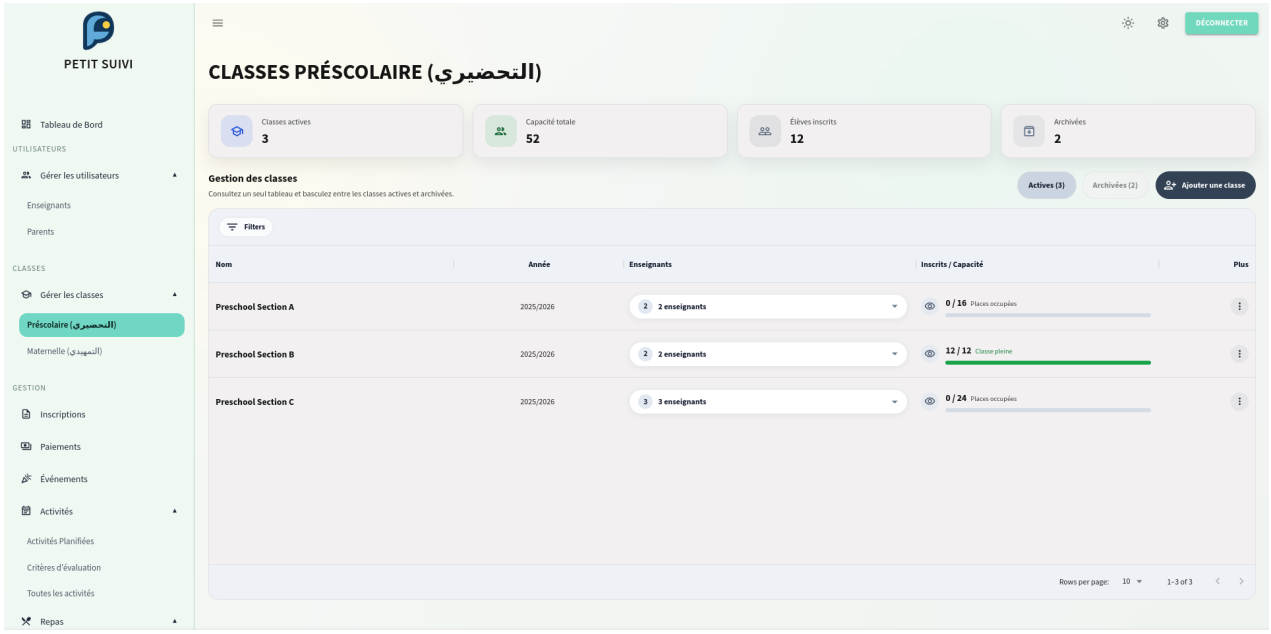
Use Case name: Manage Inscriptions

Scenario: Approve a pending inscription request



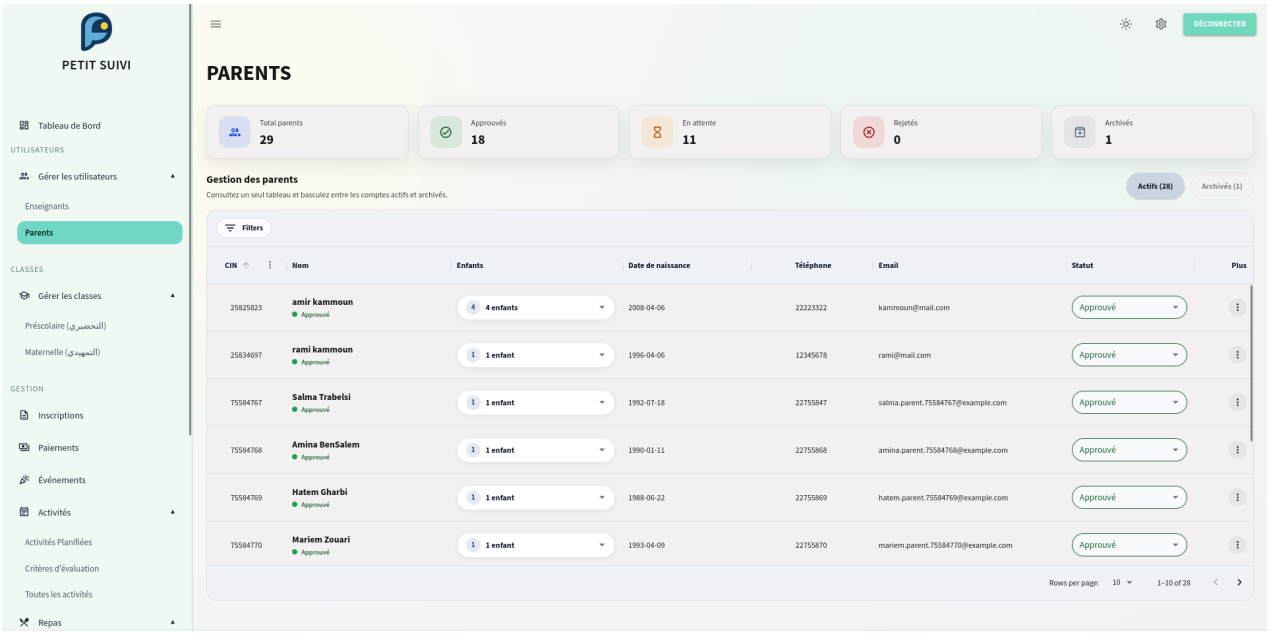
Use Case name: Manage Classes

Scenario: Create a new class



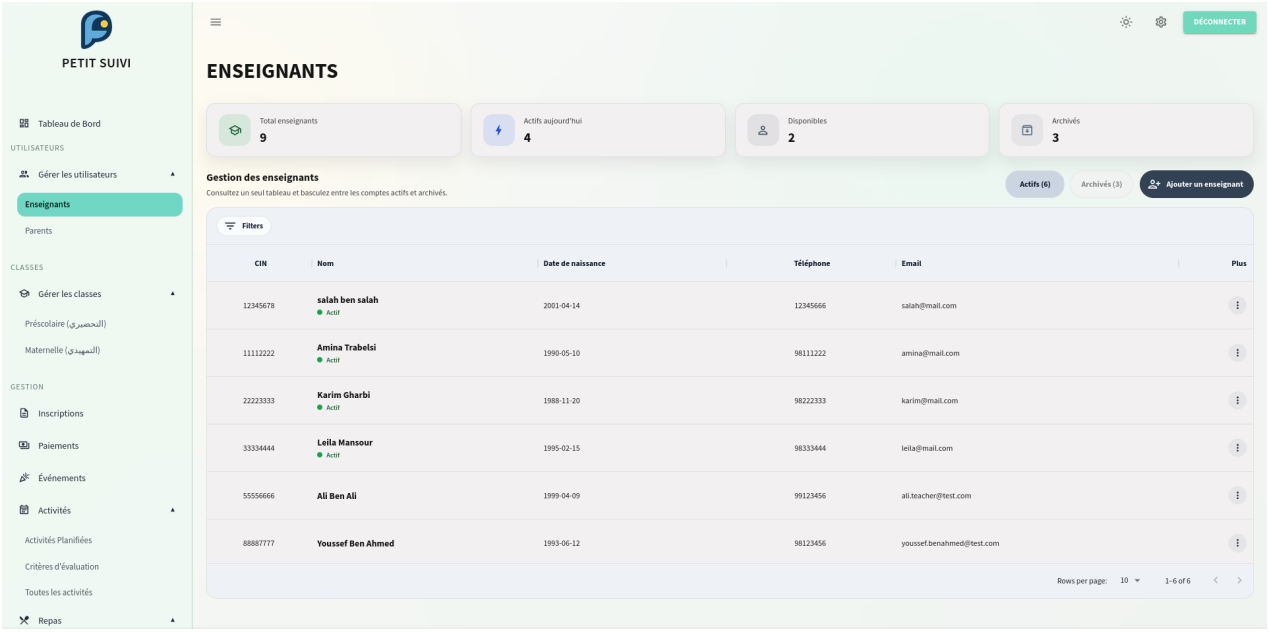
Use Case name: Manage Users

Scenario: Approve a pending parent account



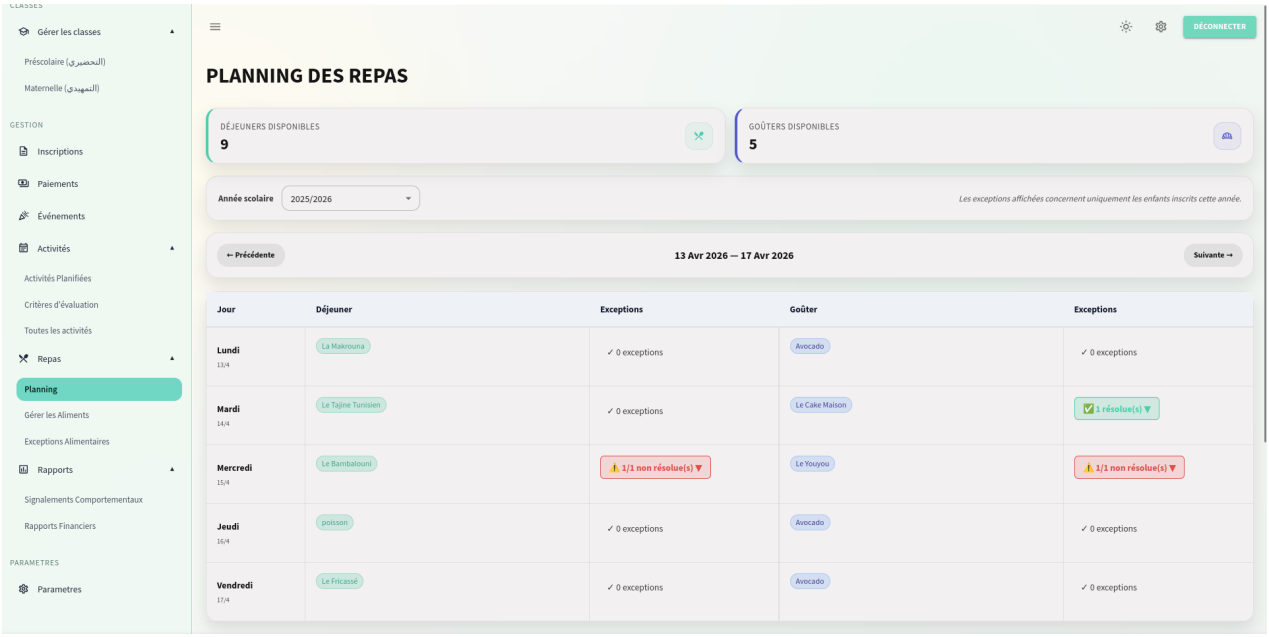
Use Case name: Manage Users

Scenario: Create a new teacher account



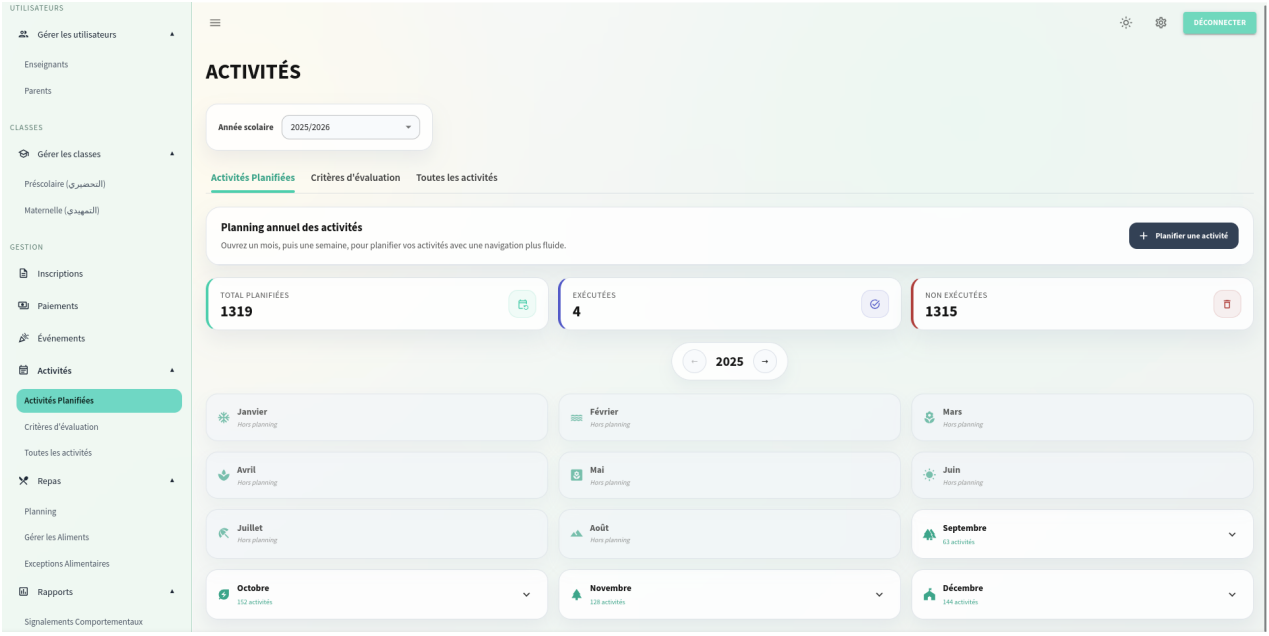
Use Case name: Orchestrate Meal Schedules

Scenario: Compose a weekly meal plan



Use Case name: Manage Planned Activities

Scenario: Assign or unassign activities for each day across the academic year



Description: **Data Integrity Protocol: Scheduling Constraints & Temporal Collision Detection**

Core Identification

- **API Route:** `POST /api/plannings/{id}/activities`
- **Controller:** `PlannedActivityController`
- **Function:** `store`

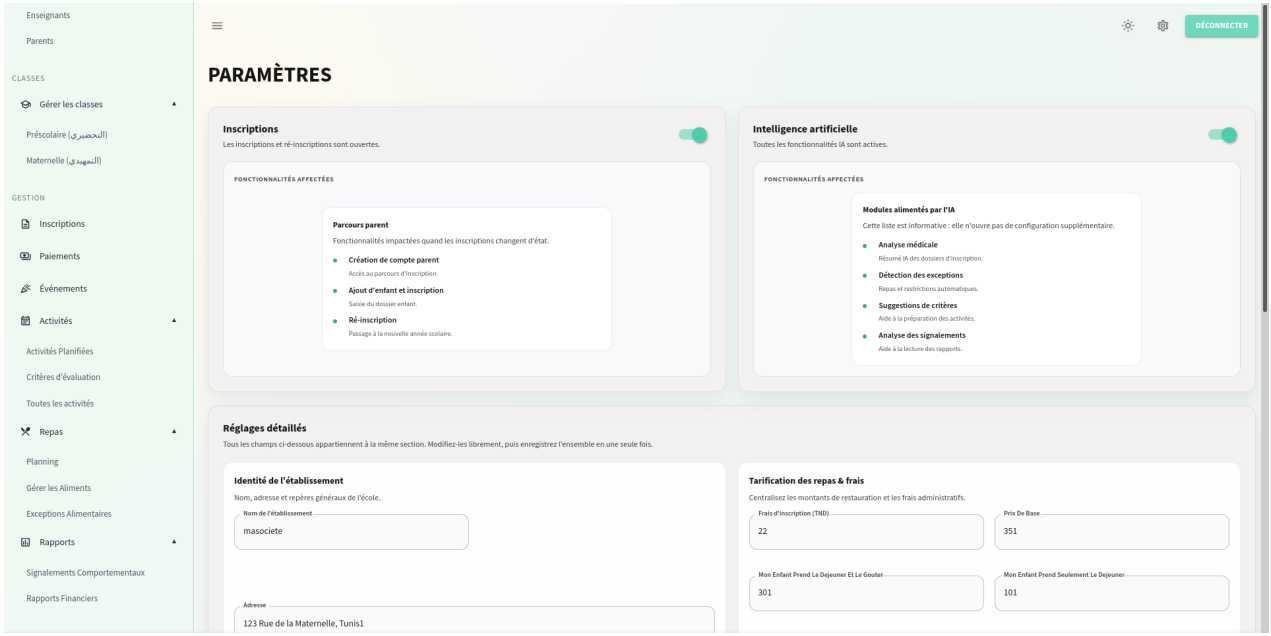
```php
$dayOfWeek = \Carbon\Carbon::parse($request->date)->dayOfWeek;
if ($dayOfWeek === \Carbon\Carbon::SATURDAY || $dayOfWeek === \Carbon\Carbon::SUNDAY) {
    return response()->json([
        'success' => false,
        'message' => 'Les activités ne peuvent être planifiées que du lundi au vendredi.'
    ], 422);
}
```

```php
// Early check for time overlaps if times are provided
if ($request->start_time && $request->end_time) {
    foreach ($request->class_ids as $classId) {
        $overlap = \Illuminate\Support\Facades\DB::table('Plandayactivity')
            ->join('Planday', 'Plandayactivity.PlandayID', '=', 'Planday.PlandayID')
            ->join('Class', 'Planday.ClassID', '=', 'Class.ClassID')
            ->join('Activity', 'Plandayactivity.ActivityID', '=', 'Activity.ActivityID')
            ->where('Planday.Date', $request->date)
            ->where('Planday.ClassID', $classId)
            ->whereNotNull('Plandayactivity.Starttime')
            ->whereNotNull('Plandayactivity.Endtime')
            ->where('Plandayactivity.Starttime', '<', $request->end_time)
            ->where('Plandayactivity.Endtime', '>', $request->start_time)
            ->first();

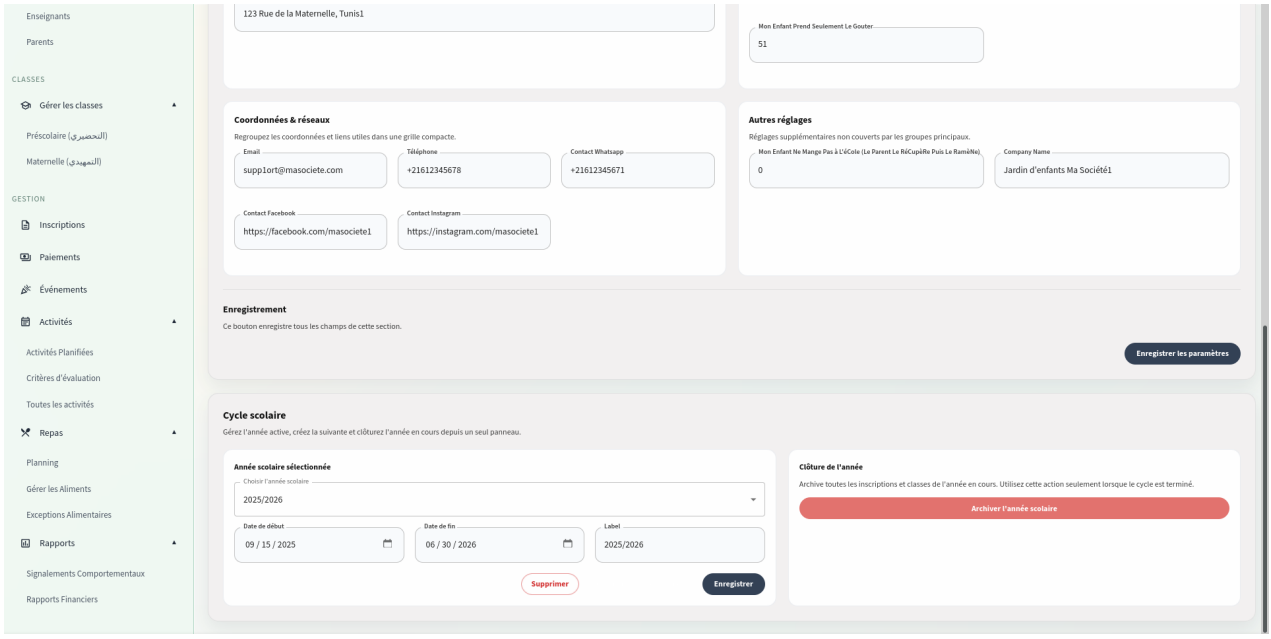
        if ($overlap) {
            return response()->json([
                'success' => false,
                'message' => 'L\'activité chevauche une autre activité ("' . $overlap->Title
            ], 409);
        }
    }
}
```

Use Case name: Configure Parameters

Scenario: Update system configuration parameter values

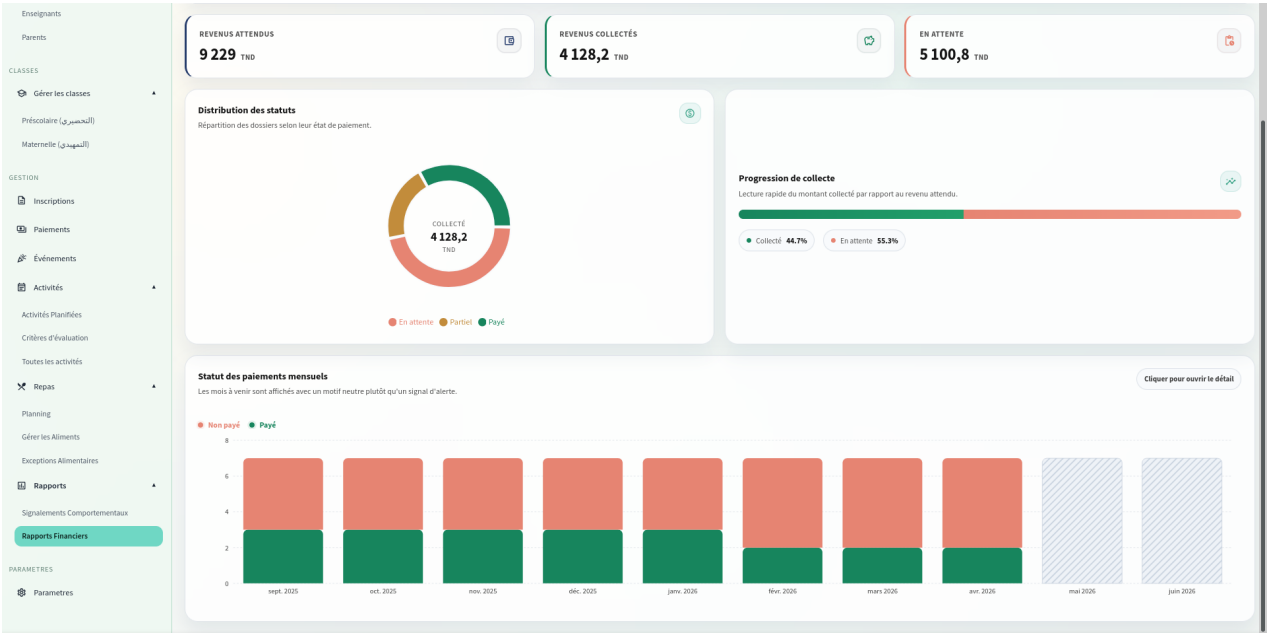


Use Case name: Configure Parameters

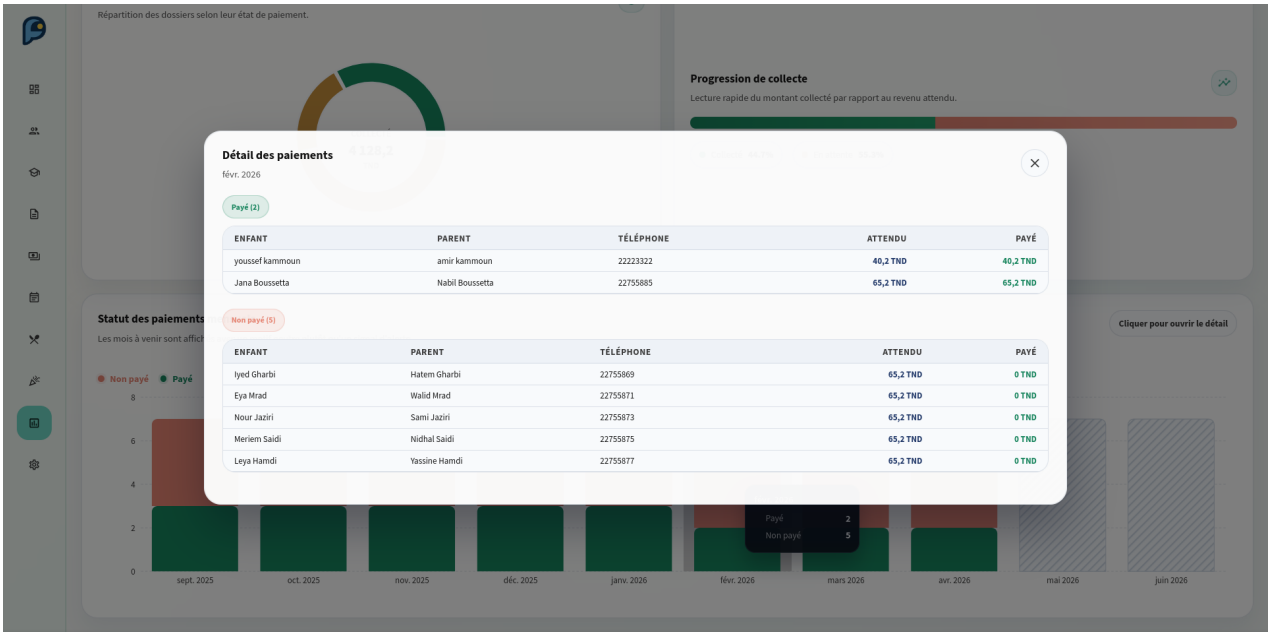


Use Case name: Generate Reports

Scenario: Analyze financial reports (RAPPORTS FINANCIERS)



Use Case name: Generate Reports



4.3.5 System API Endpoints

Section	Endpoint	Method
Authentication & Profile		
Admin Login	/admin/login	POST
Admin Logout	/admin/logout	POST
Teacher Login	/api/login/teacher	POST
Parent Login	/api/login/parent	POST
Register User	/api/register	POST
Check Email Availability	/api/register/check-email	POST
Change Password	/api/password/change	POST
User & Student Management		
Manage Teachers	/admin/teachers	GET, POST
Teacher Details	/admin/teachers/{cin}	PUT, DELETE
Archive Teacher	/admin/teachers/{cin}/toggle-archive	PATCH
Manage Parents	/admin/parents	GET
Parent Details	/admin/parents/{cin}	PUT, DELETE
Approve Parent	/admin/parents/{cin}/approval-status	PATCH
Manage Children	/api/parents/{cin}/children	GET, POST
View Child Photos	/api/children/{id}/photos	GET
View Child Payments	/api/parents/{cin}/payments	GET
School Infrastructure		
Manage Classes	/admin/classes	GET, POST
Class Details	/admin/classes/{id}	PUT, DELETE
Manage Plannings	/admin/plannings	GET, POST
Archive School Year	/admin/plannings/archive-current-year	POST
Teacher Class View	/api/teachers/{cin}/classes/by-planning	GET

Table 4.1: System API Endpoints - Part 1

Section	Endpoint	Method
Activities & Pedagogical Tracking		
Manage Activities	/admin/activities	GET, POST, PUT, DELETE
Manage Criteria	/admin/criteria	GET, POST, DELETE
AI Activity Suggestions	/admin/ai/suggest-activity-criteria	POST
List Activities	/api/teachers/{cin}/classes/{id}/activities	GET
Update Activity Status	/api/teachers/{cin}/classes/{id}/activities/{aid}/status	PATCH
Bulk Evaluation	/api/evaluations/bulk	POST
View Evaluations	/api/children/{id}/evaluations/date/{date}	GET
Nutrition & Food Management		
Manage Meals	/admin/meals	GET, POST, DELETE
Weekly Menu	/admin/meals/by-week/{date}	GET
Menu Planning	/admin/menu-plannings	GET, POST
Food Exceptions	/admin/food-exceptions	GET, POST, DELETE
Communication & Notifications		
Report Incident	/api/signalements	POST
View Incidents	/api/children/{id}/signalements	GET
Mark Incident Read	/api/children/{id}/signalements/read	PATCH
Parent Notifications	/api/parents/notifications	GET
Pickup Request	/api/notifications/pickup	POST
Pickup Queue	/api/teacher/pickup-notifications	GET
Complete Pickup	/api/teacher/pickup-notifications/{id}/complete	PATCH
Media & Assets		
Upload Photos	/api/teacher/photos	POST (Multipart)
Retrieve Photos	/api/teacher/photos	GET

Table 4.2: System API Endpoints - Part 2

4.4 Conclusion

This chapter outlined the technical realization of the platform, showcasing the transition from architecture to a live, secure implementation.

General Conclusion

In this project, we successfully developed and deployed **Petit Suivi**, an integrated educational management application designed to facilitate and streamline the complex administrative tasks, pedagogical tracking, and communication processes within early childhood preschool institutions.

In the first chapter of this work, we presented the general project context, identified the limitations of existing solutions, and outlined our primary mission to bridge these gaps. The second chapter focused on identifying the different actors within our system and documenting their interactions through comprehensive use cases. This foundation allowed us to transition to the system design phase in Chapter 3, where we modeled the architecture and database logic. Finally, Chapter 4 constituted the core practical implementation of our project, detailing the hardware specifications and the presentation of the modern technologies utilized.

This end-of-studies project provided a profound opportunity to consolidate the theoretical knowledge acquired throughout our academic training by applying it to cutting-edge technologies. This experience allowed us to master precise UML modeling and dive deep into multi-platform development. Building the mobile application with the Flutter SDK presented distinct challenges in state management and cross-platform fluid UI design, but successfully overcoming these obstacles was highly rewarding. Furthermore, developing a highly reactive administrative dashboard using React.js and integrating Artificial Intelligence utilities via a Node.js microservice constituted a major part of our practical technical contribution.

Crucially, our work allowed us to discover the intricacies of modern distributed architectures. We learned how to build and integrate a robust Laravel API hosted on a remote server ecosystem, utilizing REST parameters and JSON formatting to seamlessly manage secure data communication between heterogeneous environments—namely, the Dart-based mobile client, the JavaScript web dashboard, and the centralized MySQL database. We also practically explored modern DevOps practices, including continuous integration and deployment pipelines.

The daily involvement in this project alongside stakeholders was a unique occasion to cultivate our communication skills in a professional environment, ensuring the technical features perfectly matched real-world nursery workflows. It was a profoundly enriching experience on all fronts. Detailed screenshots explaining the practical operation and user interfaces of our application are presented in a separate manual annex of our project.

Although the primary objectives of the Petit Suivi project have been fully achieved, the application we developed is built with scalability in mind. In the future, this system could be enriched by more advanced features, such as deep machine-learning data analytics for predicting child behavioral trends, integrating local electronic payment gateways, or adding biometric attendance tracking, effectively rendering it even more interactive and comprehensive.

Bibliography

[1] Mrs. Imen Ben Said, Mrs. Emna Feki. *Laravel Course*.

[2] Mrs. Mounira Ben Abdallah Kriaa, Mrs. Nahla Zaaboub Haddar, Mrs. Samira Bendbabis. *UML Course*.

[3] Mr. Fahmi Bargui, Mrs. Wahiba Dammak. *SQL Course (Triggers)*.

[4] Mrs. Khouloud Boukadi. *Flutter Course*.

[5] Mr. Slim Kallel. *REST API Course*.

Webography

[W1] Laravel – Official Documentation.
https://laravel.com/docs/13.x

[W2] Flutter – Official Documentation.
https://docs.flutter.dev/

[W3] React – Official Documentation.
https://react.dev/learn

[W4] Sparx Systems – Enterprise Architect User Guide.
https://sparxsystems.com/enterprise_architect_user_guide/17.1/welcome/index.html

[W5] GitHub – Code Hosting and Version Control Platform.
https://github.com